



ALUMNO:

Alexis Yahir Alvarez León

MATERIA:
Sistemas Operativos

Solución Ejercicios de Shell Script

"ICO"

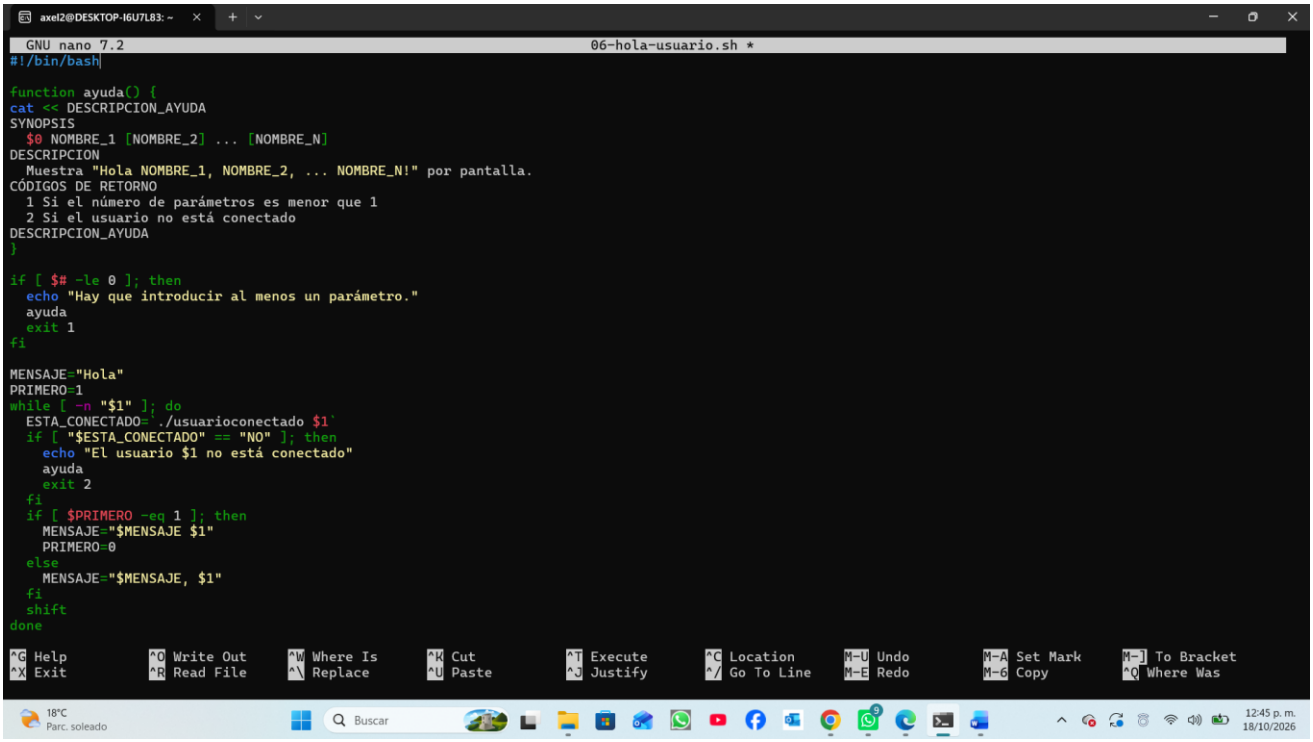
"COMPARTE LA FELICIDAD"

18/10/2026

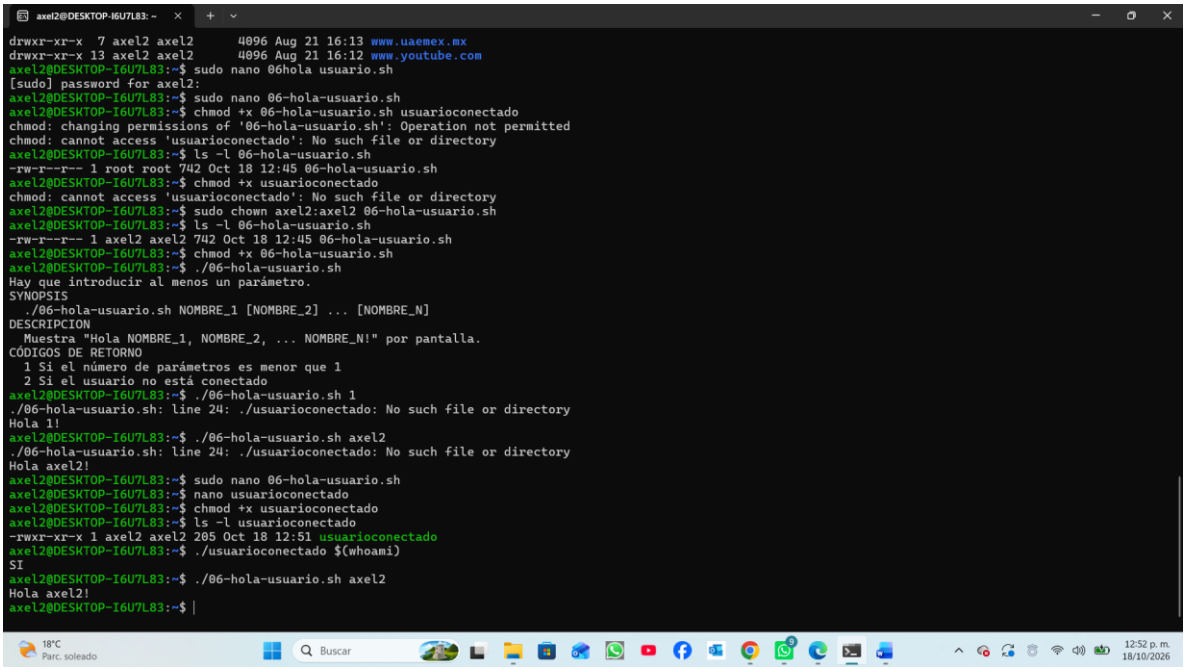
En esta tarea es ejecutar todos los scripts de un pdf que viene en este link https://asanzdiego.github.io/curso-nivelacion-bigdata-2017/ejercicios/ejercicios-shell-script-solucion.pdf
donde son 40 ejercicios y en este documento se vana describir cada de uno ejecutados y mostrados de que hacen al momento de su ejecución

Pero empezaremos desde el numero 9 en adelante ya que los anteriores 8 ya se realizaron en clase, así que comencemos

El numero 9 es :Modificar el script '06-hola-usuario.sh' para que llame a 'usuarioconectado'

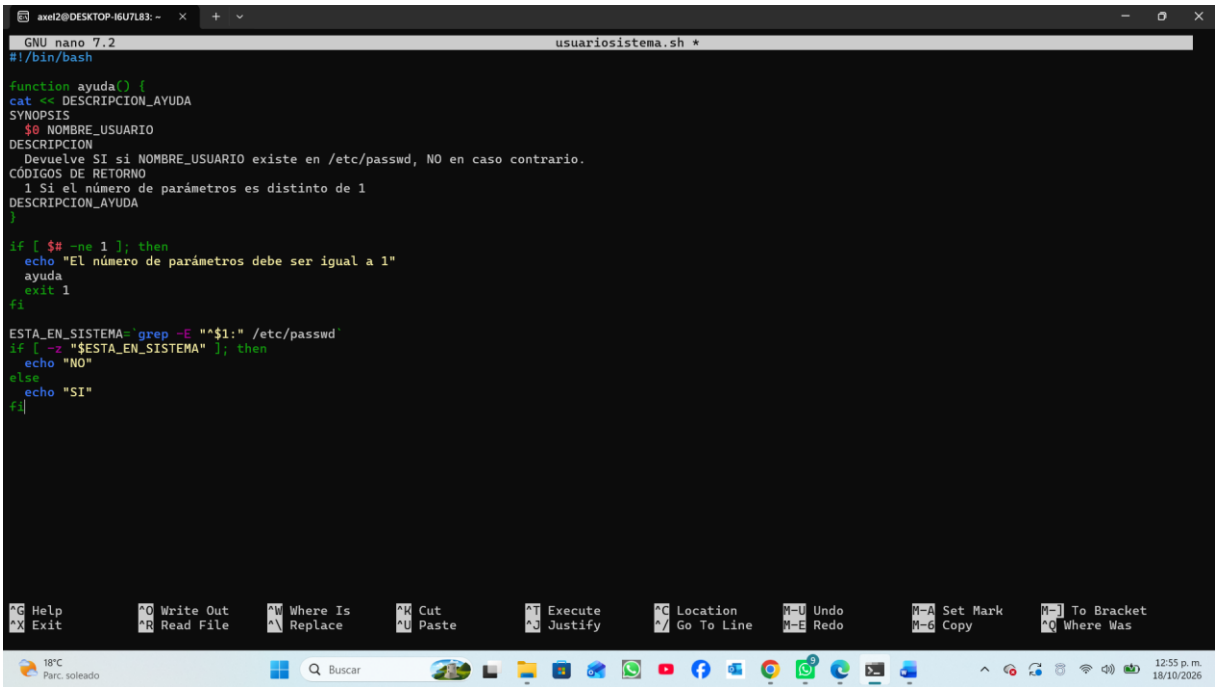


Ahora al ejecutar sale esto jeje

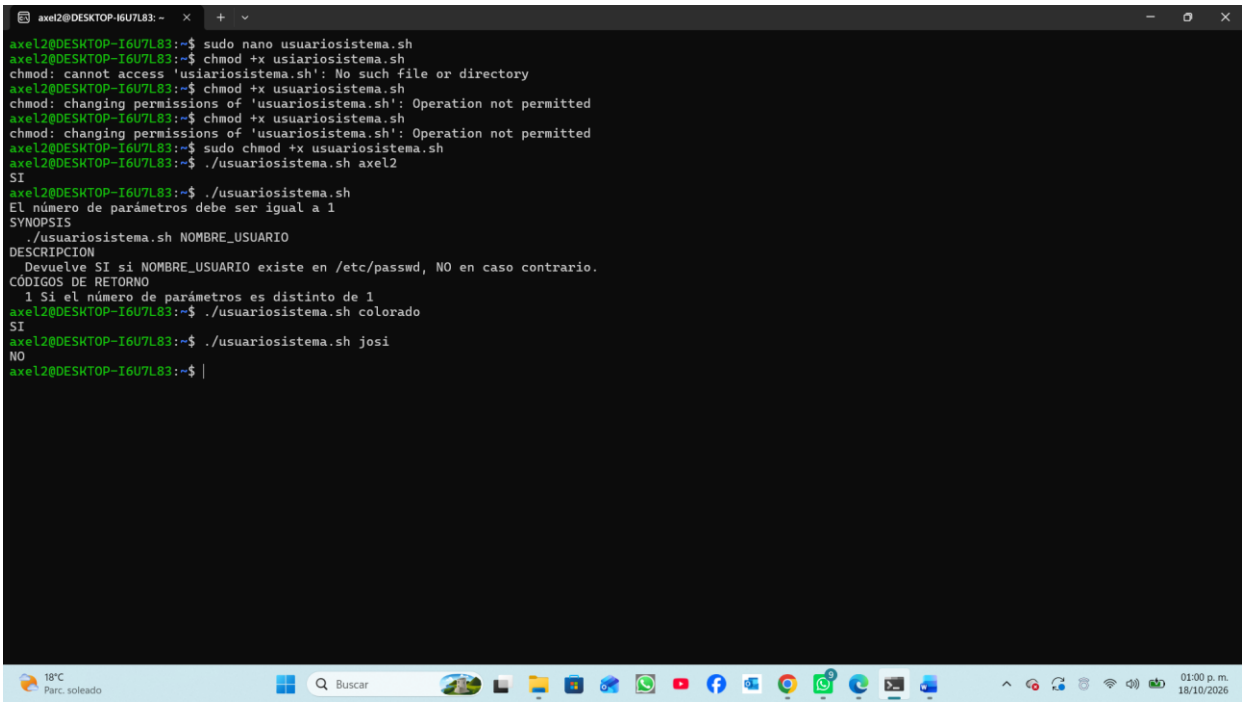


Vemos que hice algunas cosas porque no tenia creado el 6 lo tuvve que volver a hacer pero al ejecutar el paso 9 y poner mi usuario sale hola axel2 que es mi usuario, pasemos el ejercicio 10

EJERCICIO 10

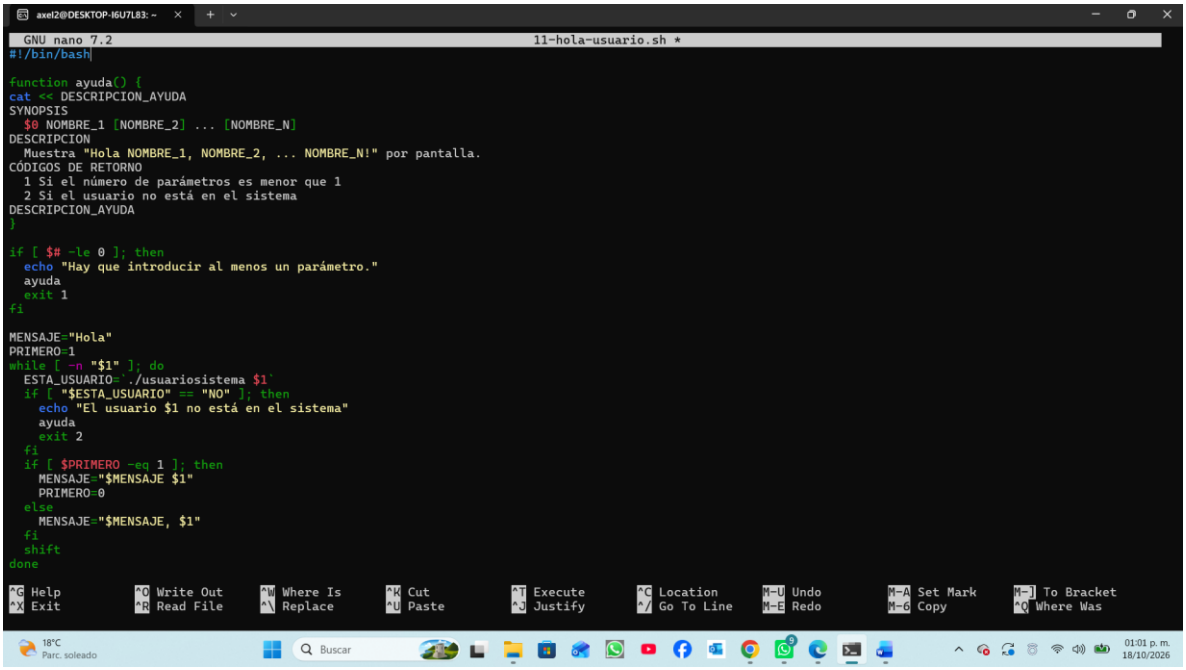


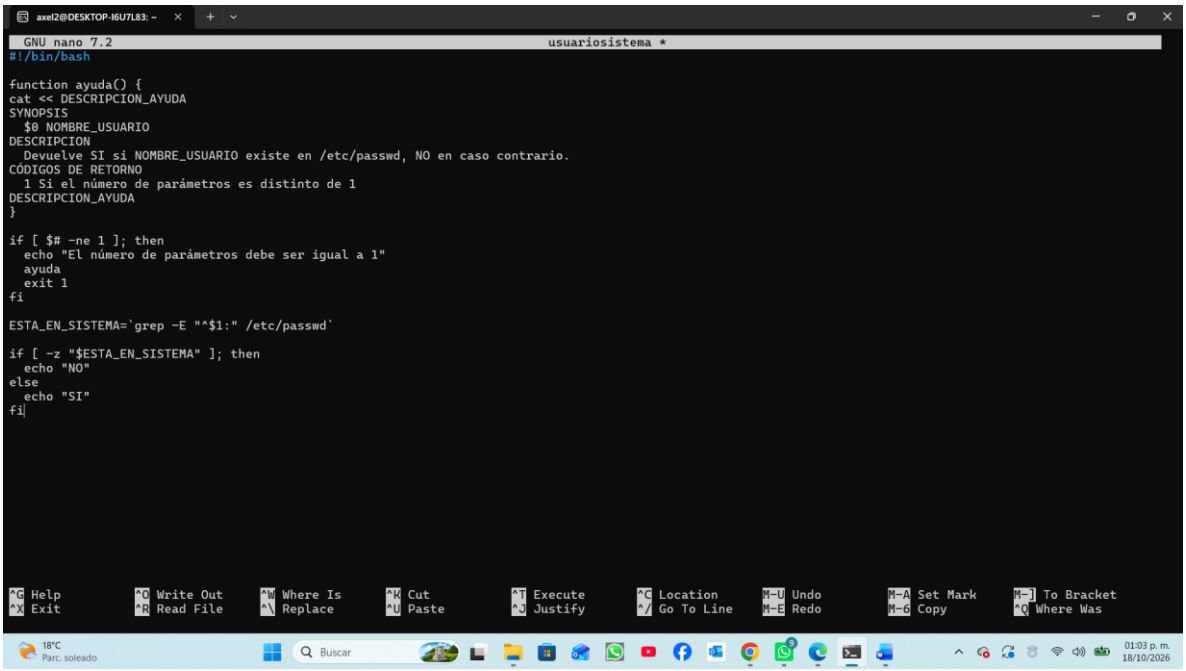
De este ejercicio 10 es para ver si si retorna, coincide el usuario del sistema o no, vamos a ejecutarlo



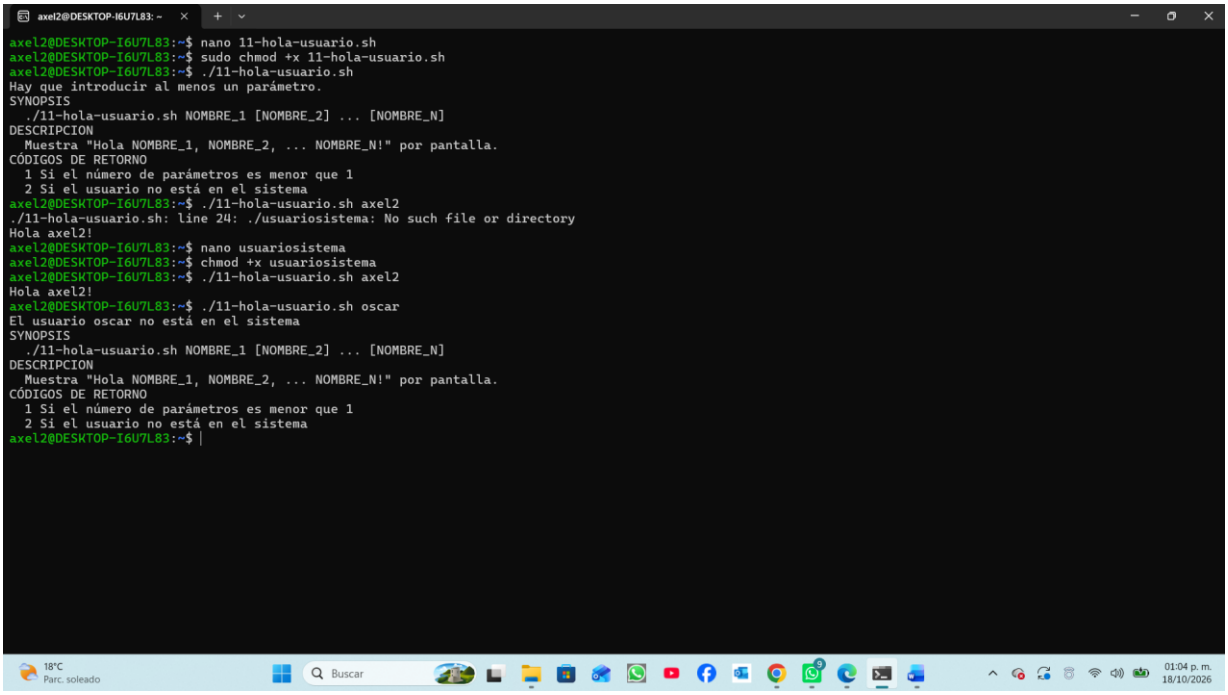
Viendo la imagen se le debe dar permisos con chmod al script para que se ejecute correctamente cosa que en los demás scripts se va esta repitiendo, ya que es un paso importante y para no estar diciendo en todos los ejercicios que se tiene que dar permisos, pero en la ejecución se ve que ya al poner mi usuario y otro de mis usuarios creados pone la palabra que SI ya que si están en mi sistema y si pongo uno que no dira que NO en este caso se ve en la imagen

EJERCICIO 11



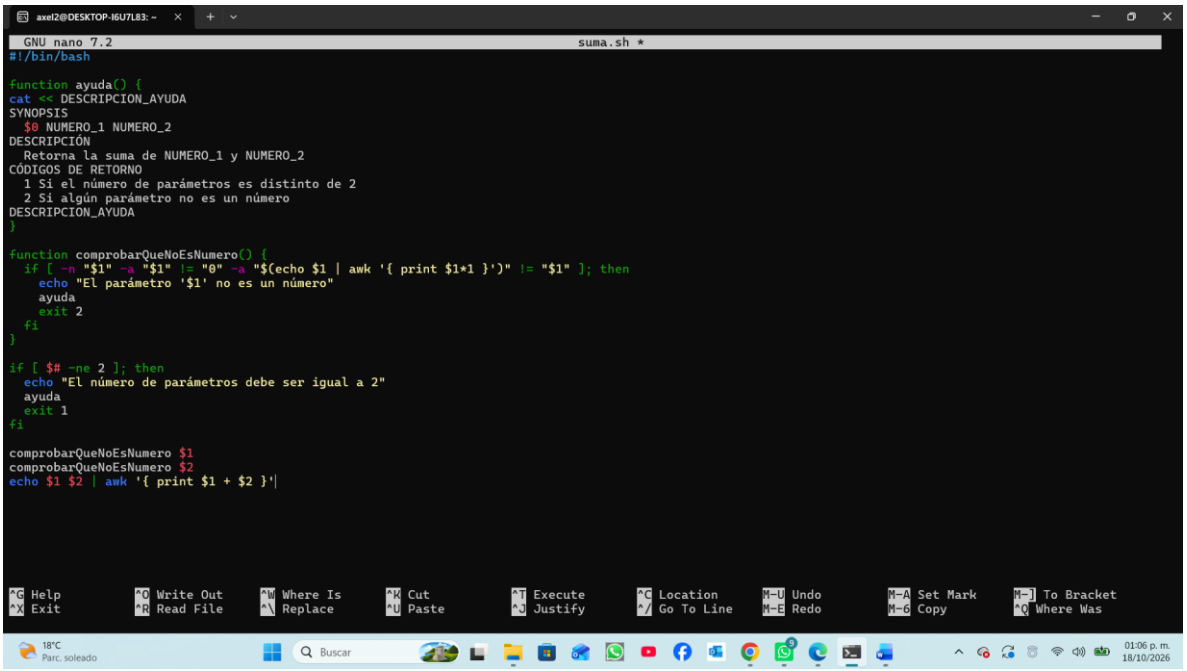


En este se modifica también el 6 para que llamen al usuariosistema y se pone la imagen de modificada del 6

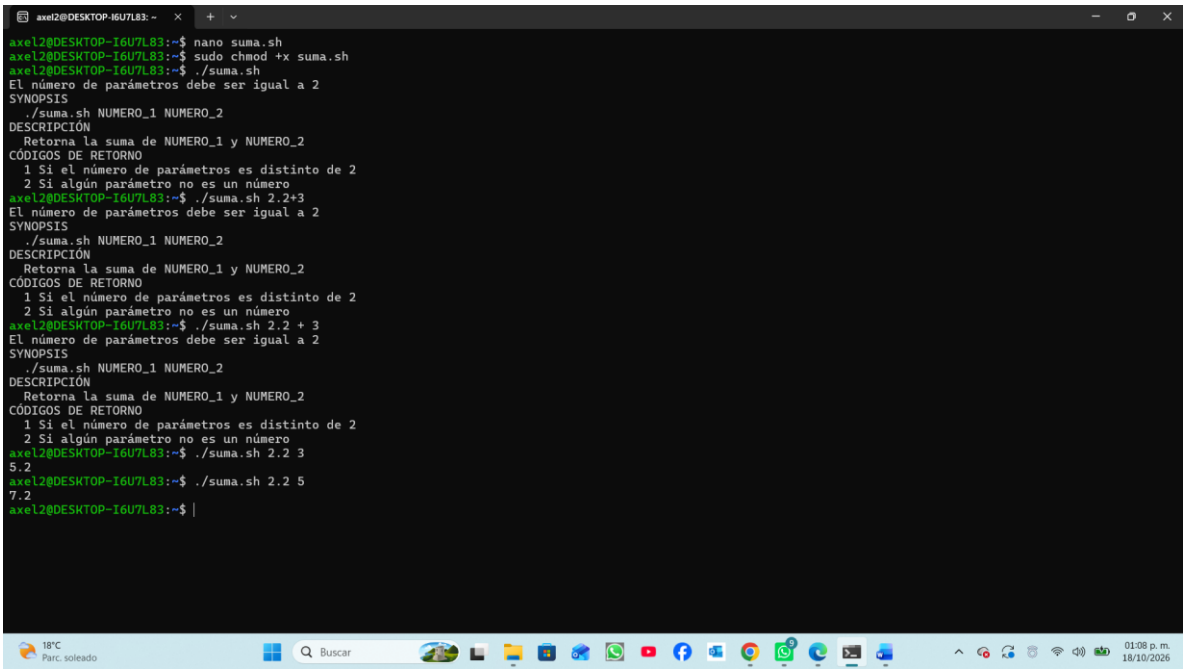


Al ejecutar se que ve nuevamente con mi usuario si funciona pero con otro que no funciona entonces aparece ese mensaje de que no esta

EJERCICIO 12

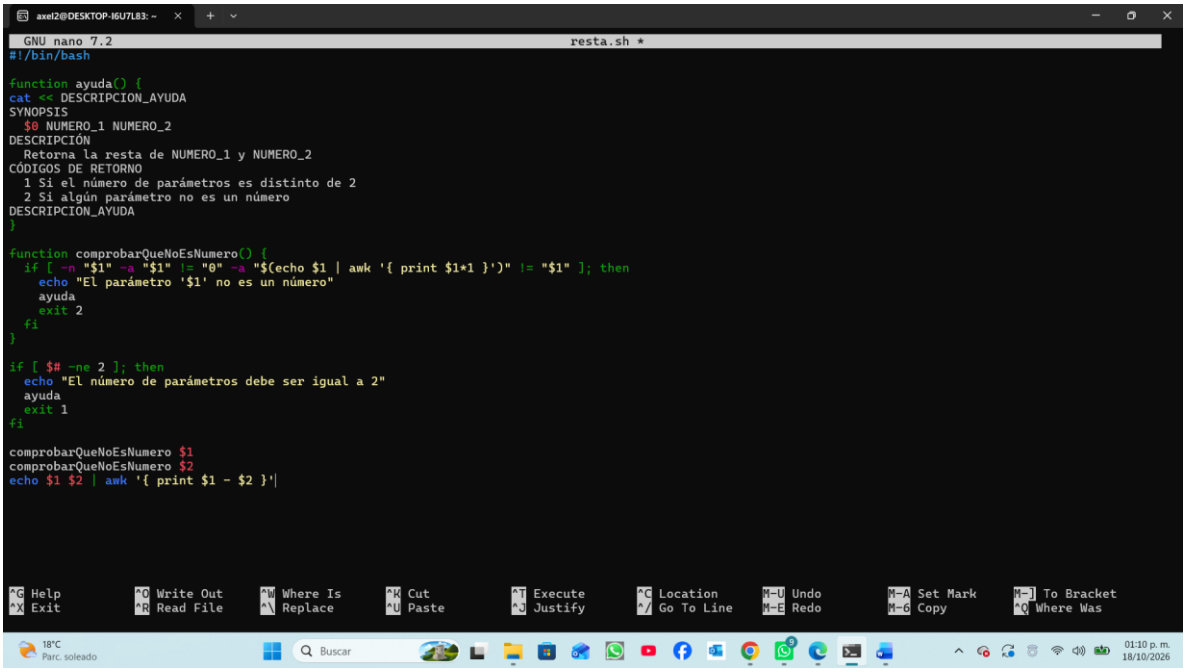


En este se hará un script de una suma donde sumara números decimales

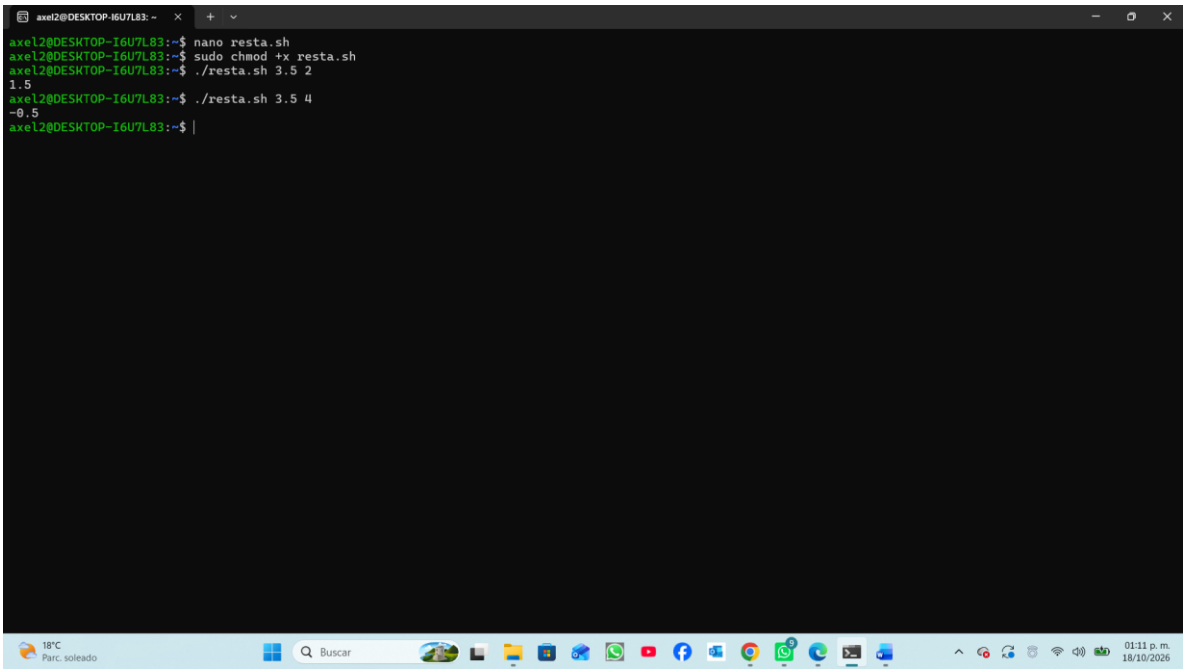


Ahora al ejecutar se ve que si funciona nomas que solo poner los números sin el signo de + y da el resultado de este suma jeje

EJERCICIO 13

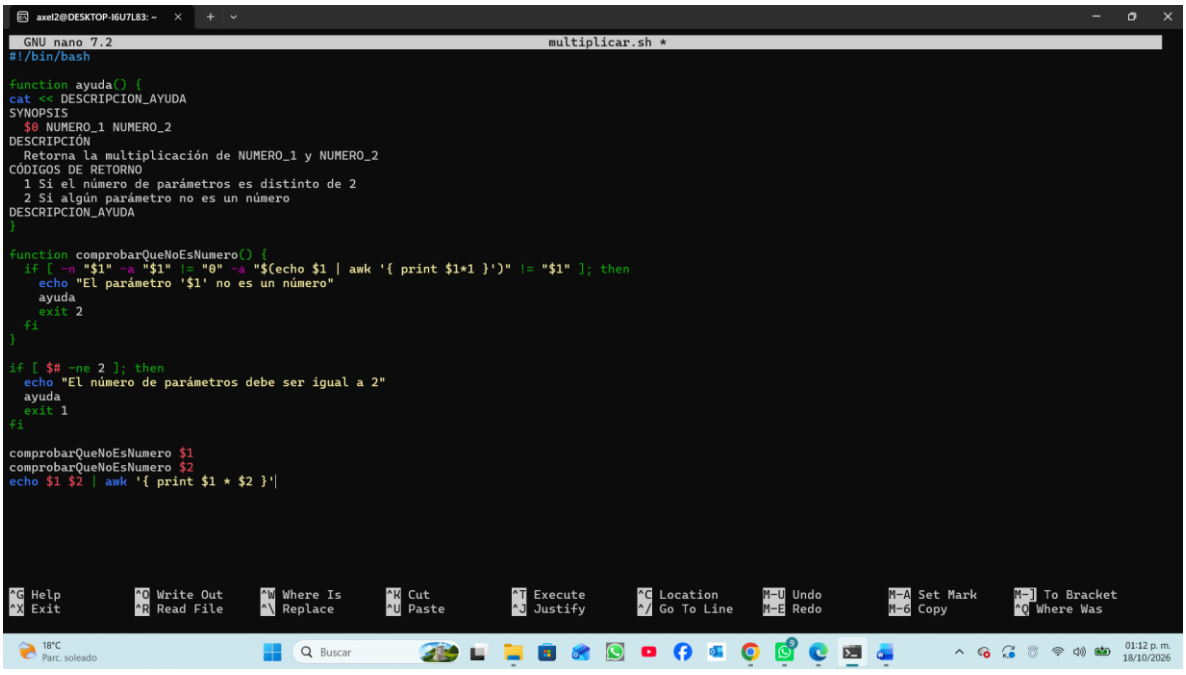


Aquí será lo mismo que el anterior solo que esta vez será una resta vamos a ejecutar y ver que funciona

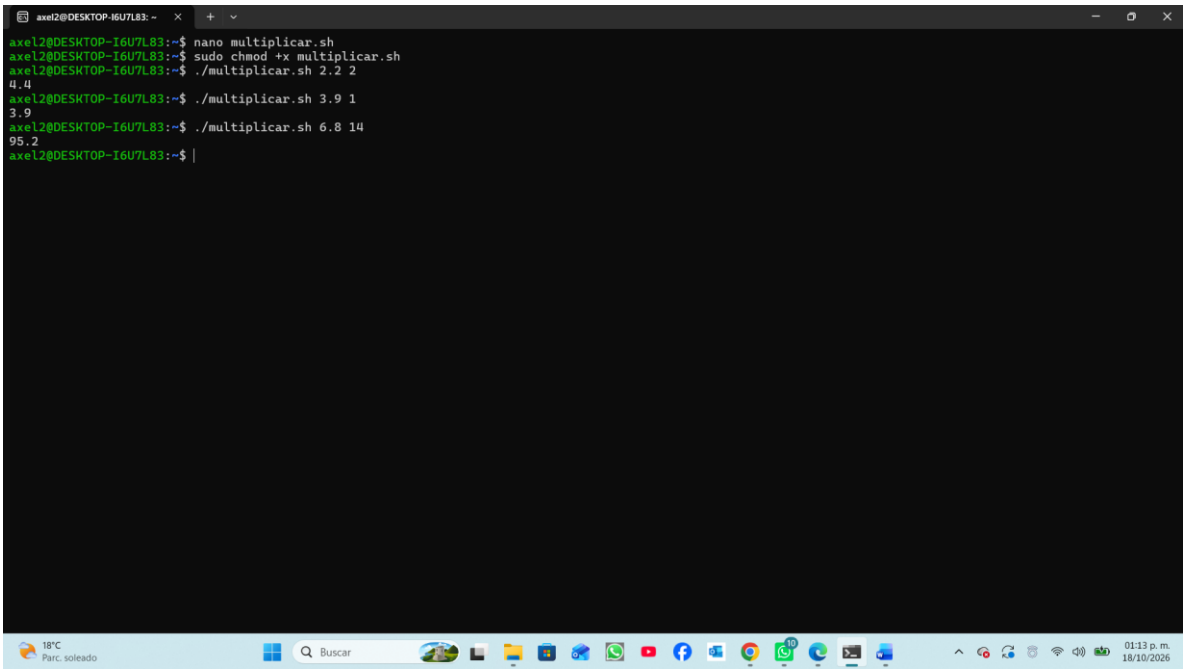


Y al ejecutar y dar permisos vemos que si funciona esa resta correctamente

EJERCICIO 14

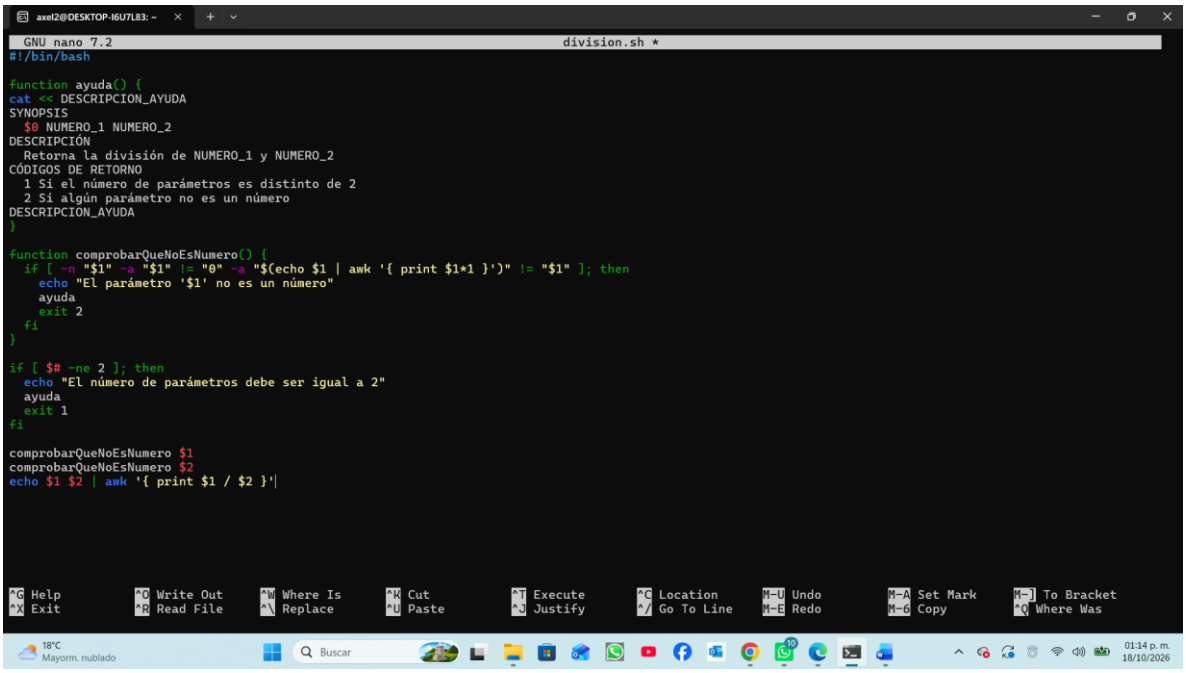


del ejercicio 14 será ahora una multiplicación y se utiliza este script , vamos a ejecutarlo

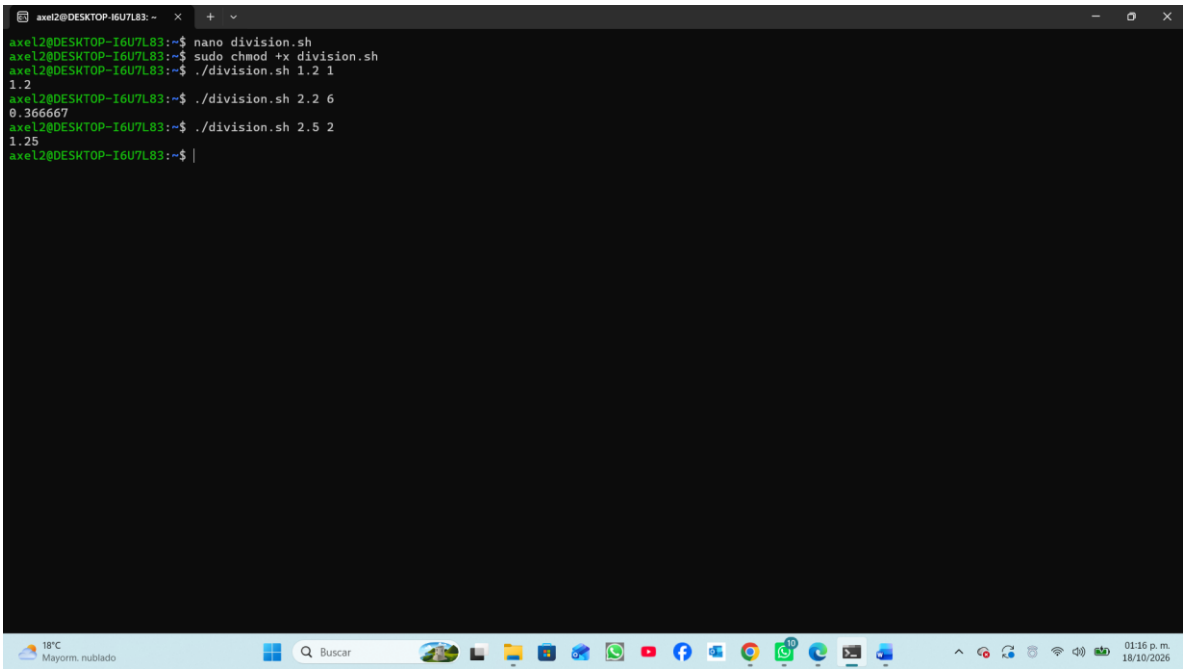


Vemos que si funciona este script de multiplicar pasemos al que sigue el 15

EJERCICIO 15



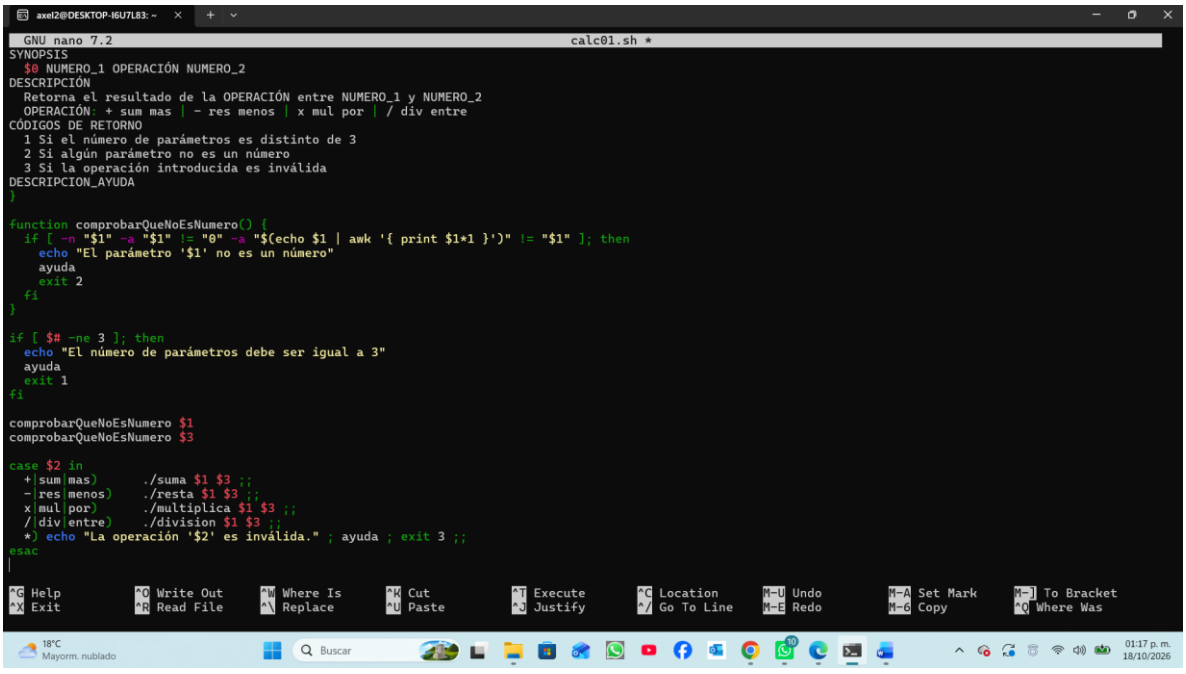
Ahora siguiendo con las operaciones vamos con la división, veremos que pasa al ejecutarlo, vamos a ellos



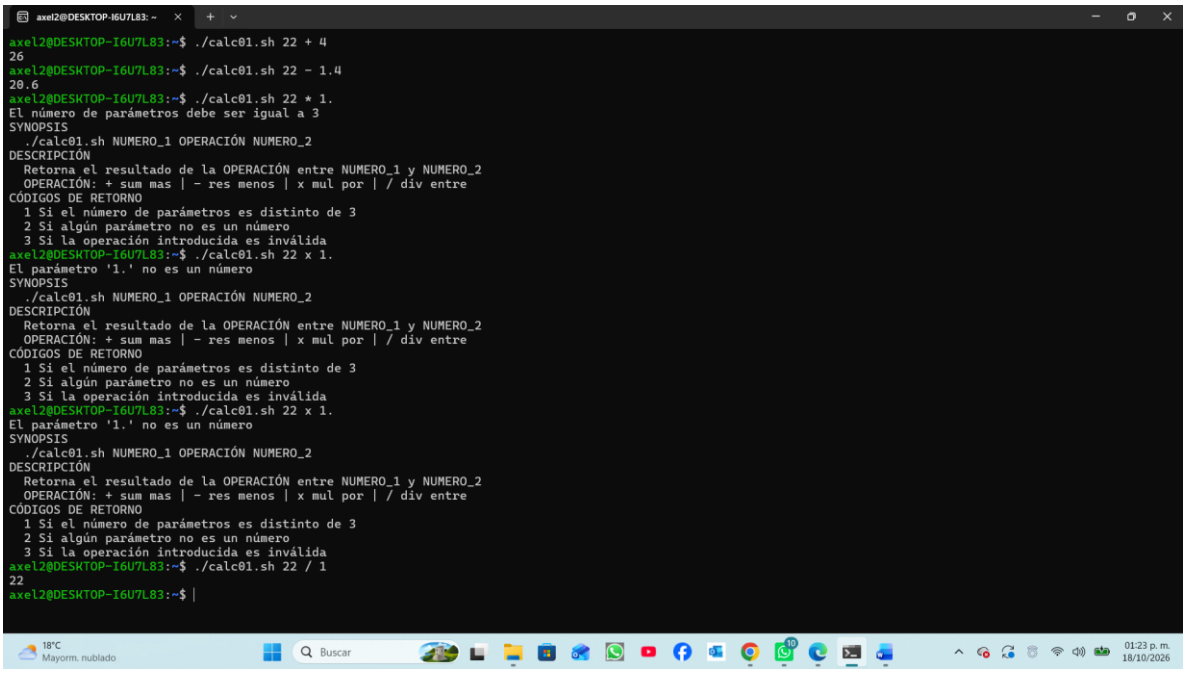
Vemos que al ejecutar si se ve que funciono esta división vamos bien el ejercicio 15

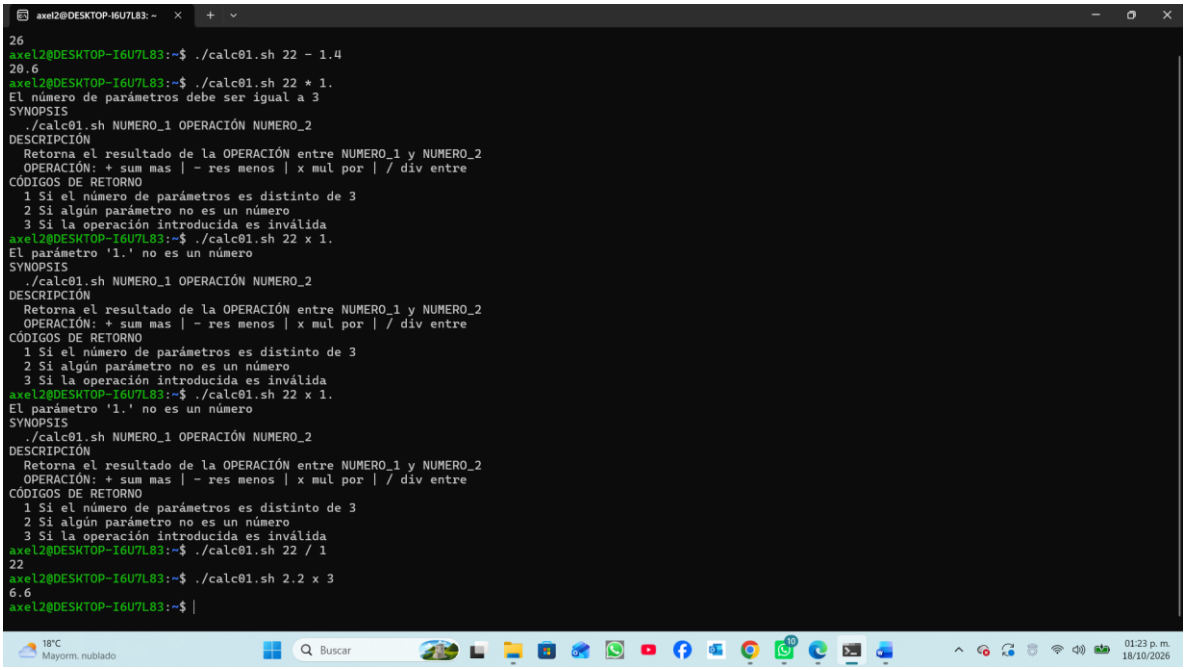
EJERCICIO 16





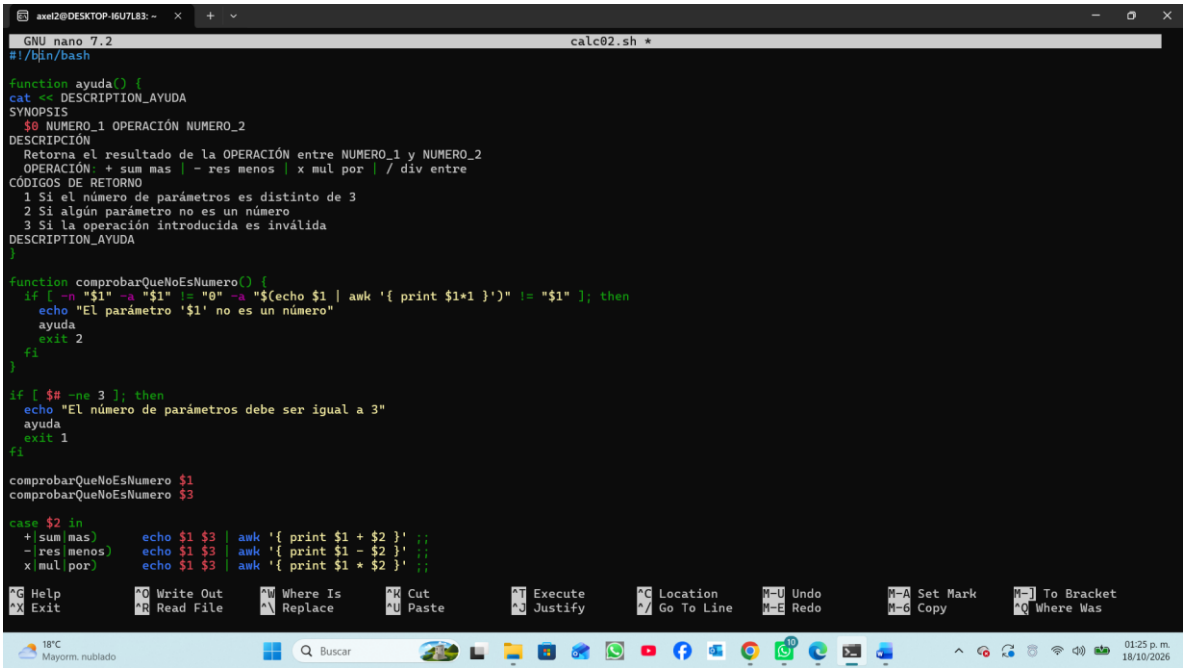
En este ahora se juntan las 4 operaciones anteriores en uno solo, y vamos ahora ejecutarlo haber que sucede jeje

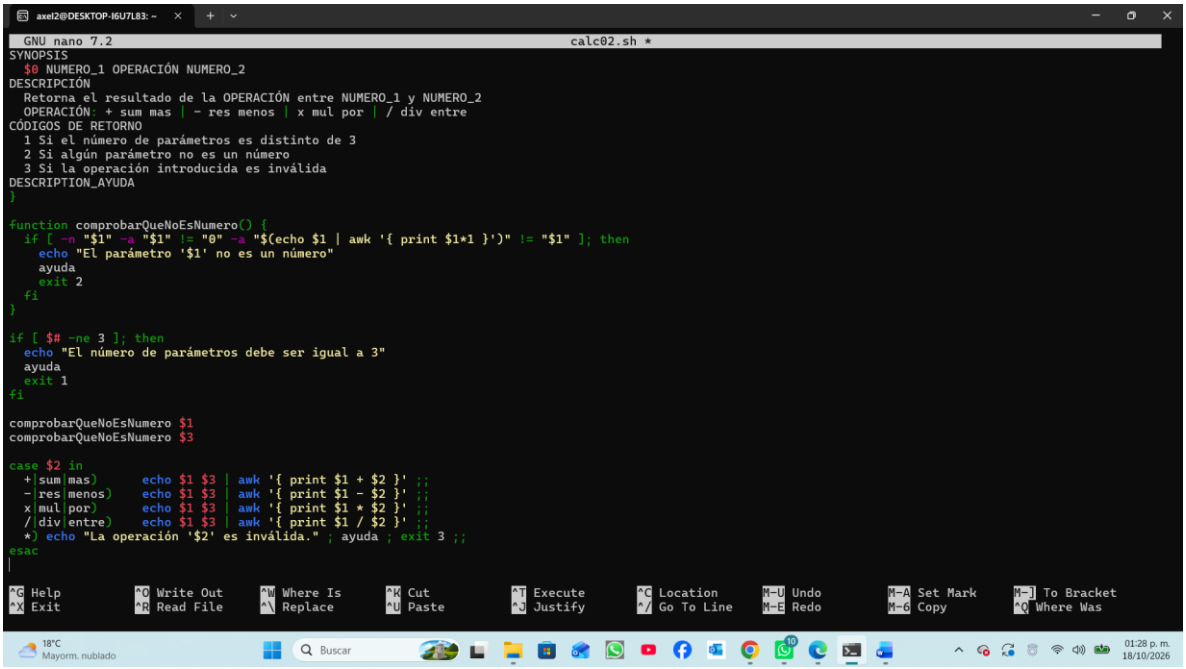




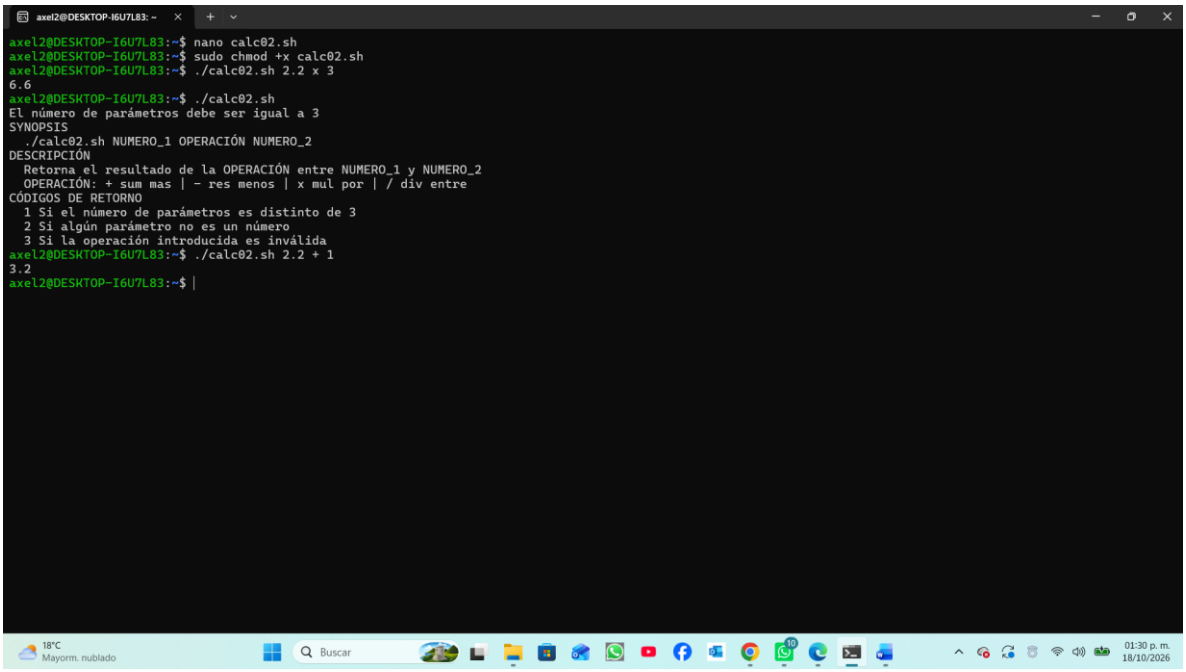
Vemos que si funciona esta de juntar todas las operaciones jeje. Tuve un problema con la multiplicación puse la x y a funciono la multiplicación vamos con el siguiente}

EJERCICIO 17



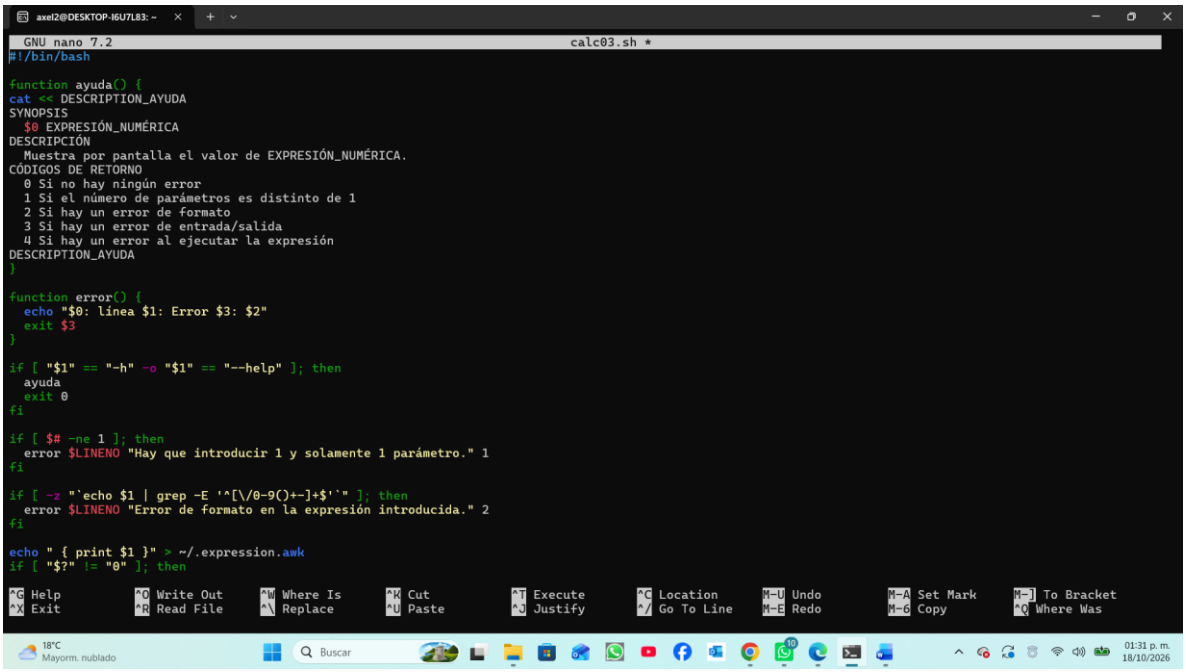


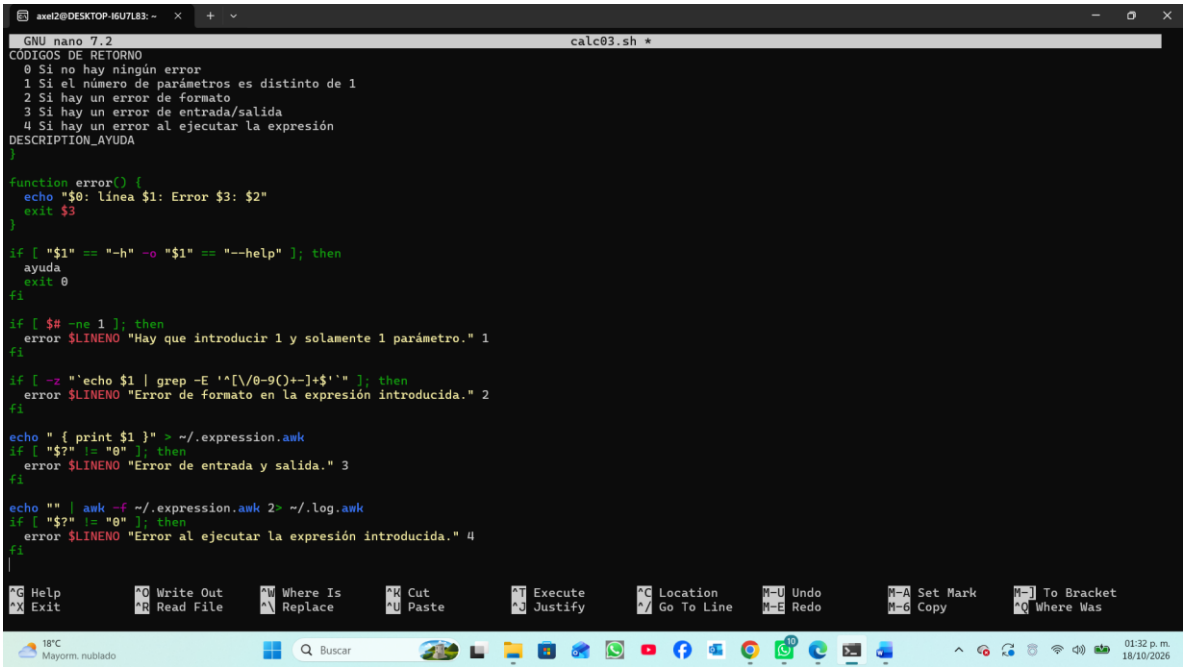
De este es un idem sin llamar a los scripts ahora veremos como funciona



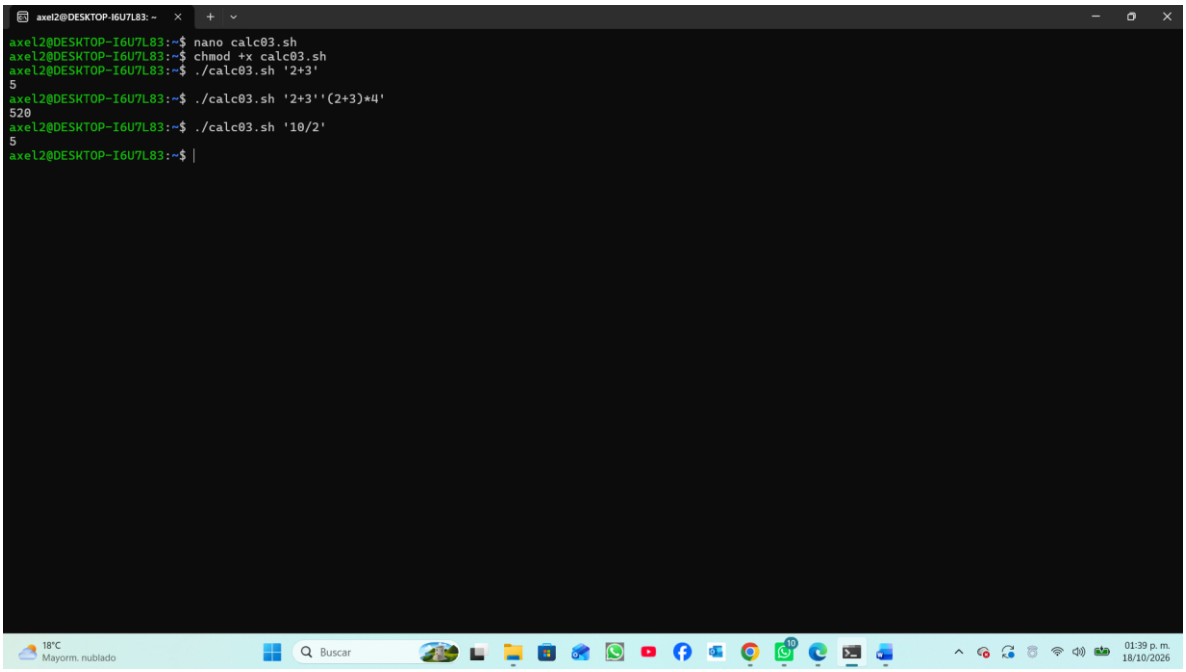
Vemos que si funciona con esas 2 operaciones y no depende de las otras que habíamos creado antes jejeje

EJERCICIO 18





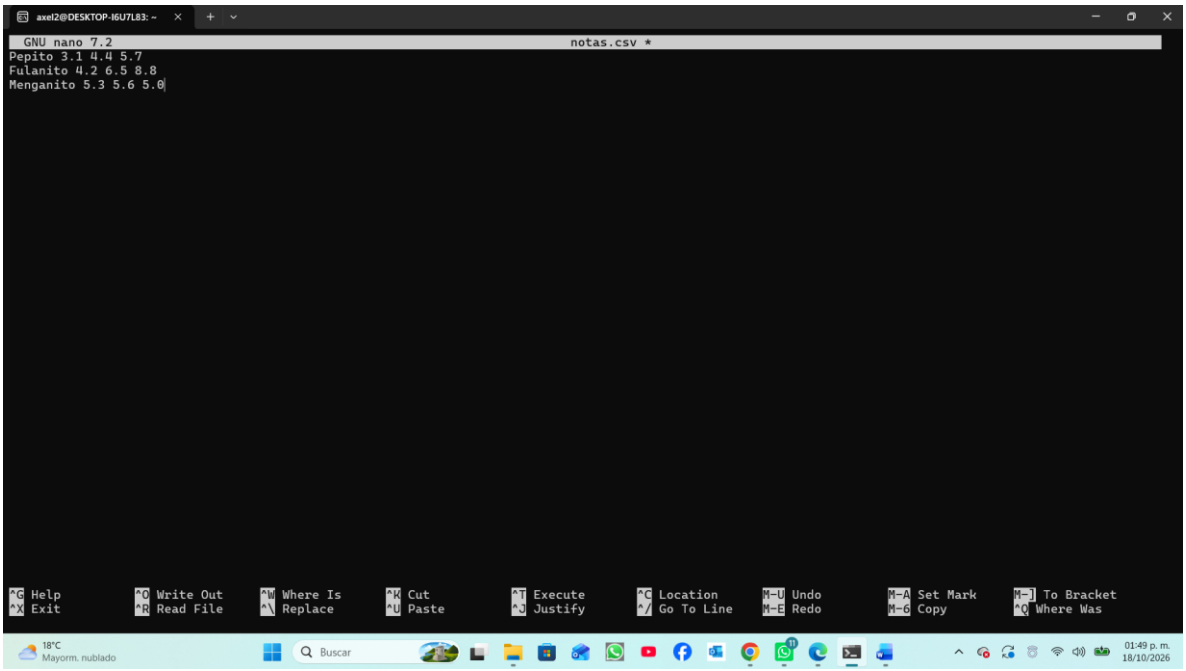
Ahora de este ejercicio lo que pide hacer es el calcular una expresión numérica pasada por parámetro, vamos a ejecutarla



Y vemos que si funciona y comillas otros signos si función al ejecutar este script si da la operación que se esta aplicando

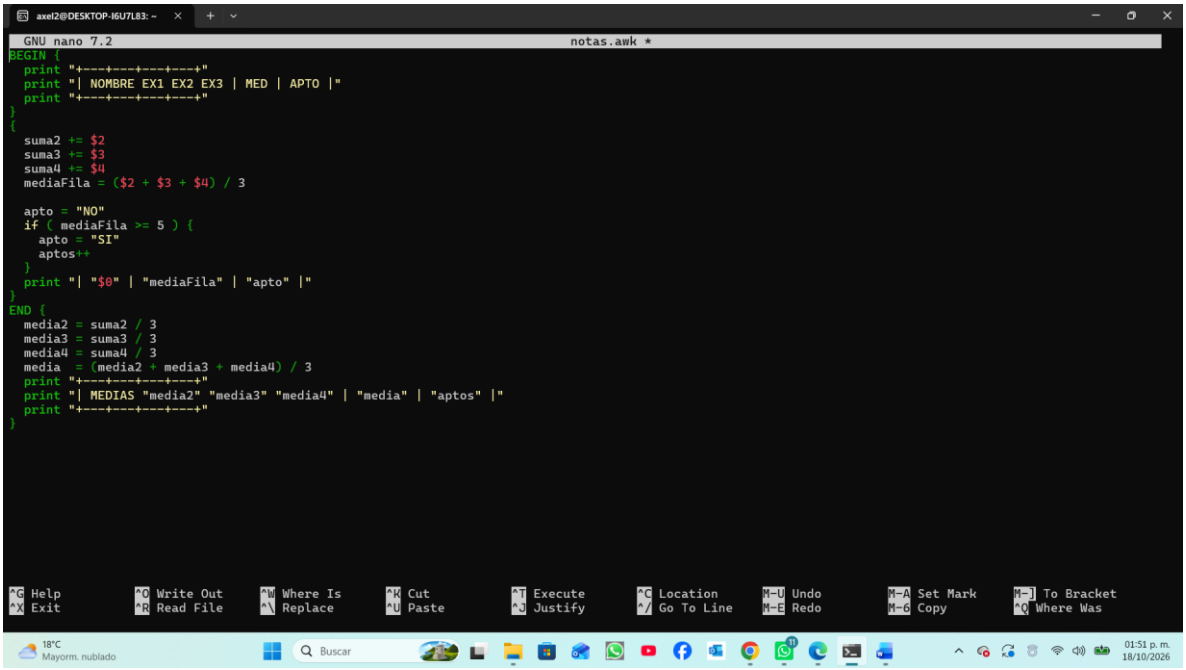
EJERCICICO 19

En este se va crear un archivo notas.csv que va contener lo siguiente:

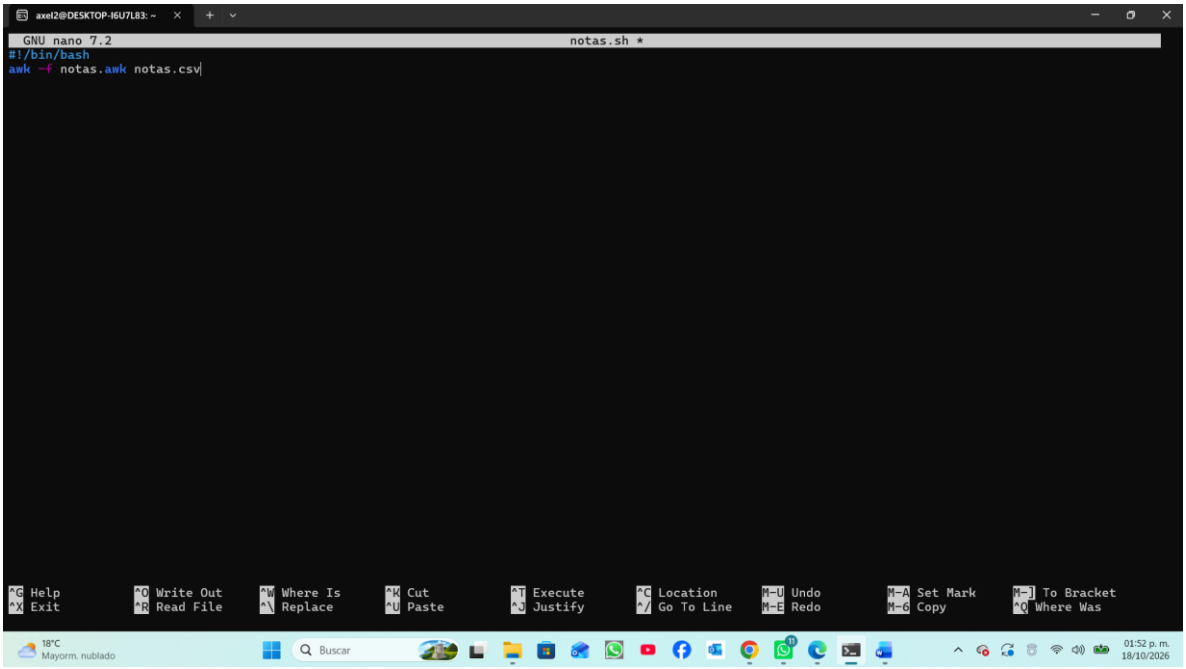


Ya con esto hecho podemos avanzar al ejercicio 20

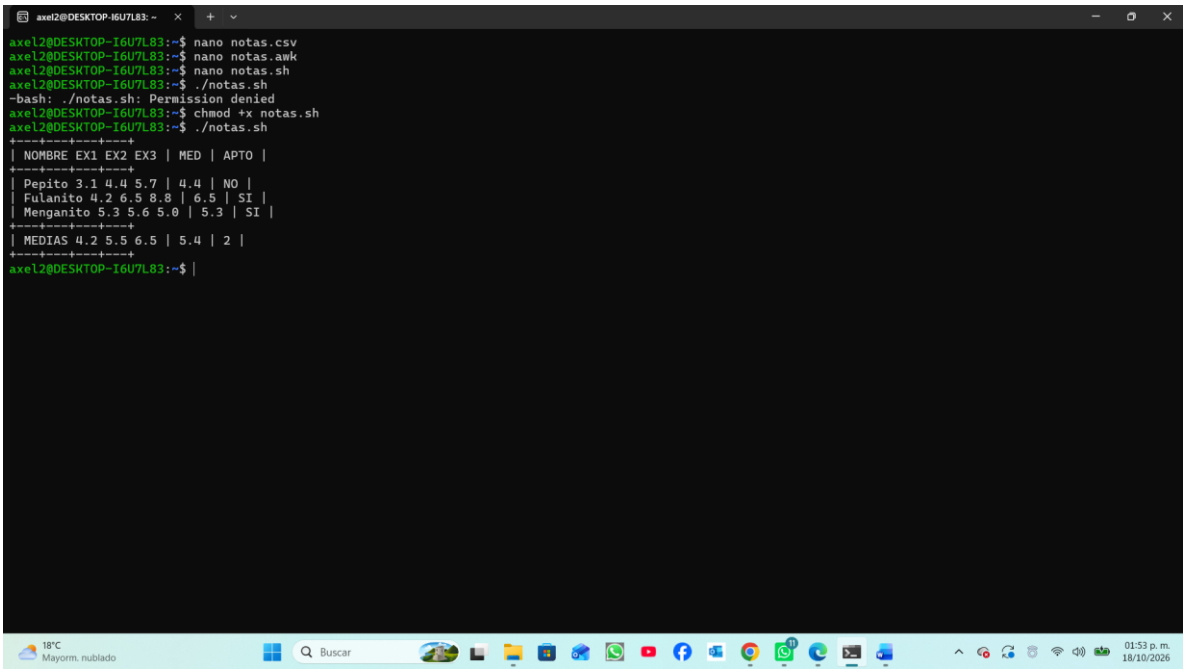
EJERCICIO 20



Aquí se van a crear 2 archivos el primero es notas.awk

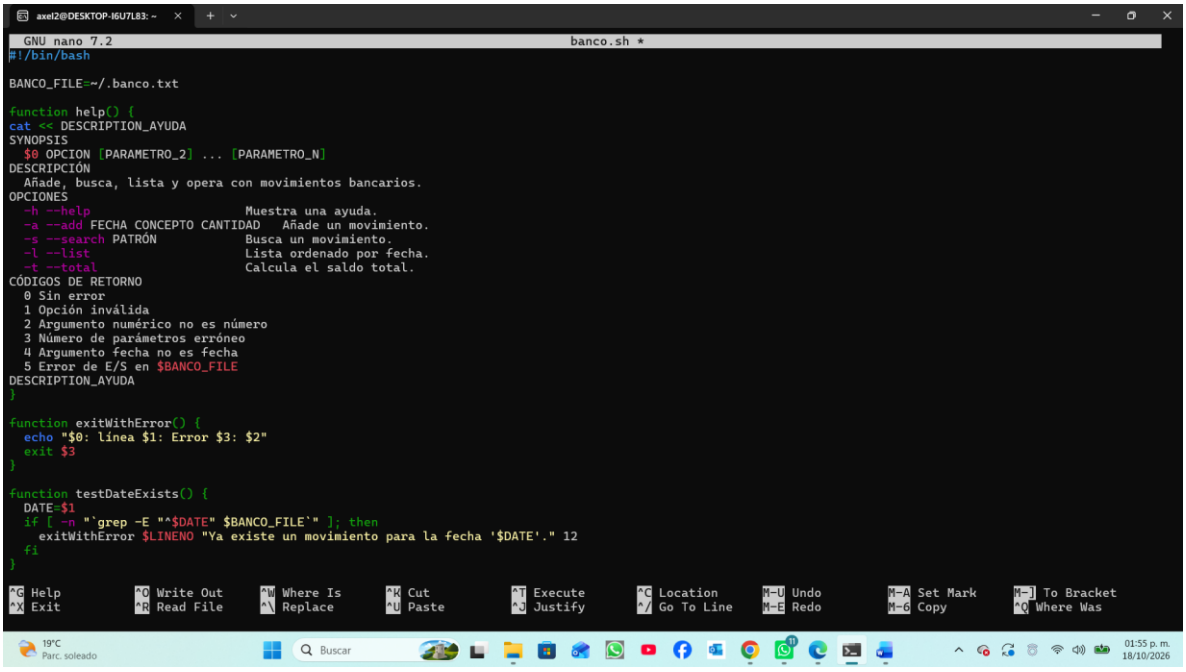


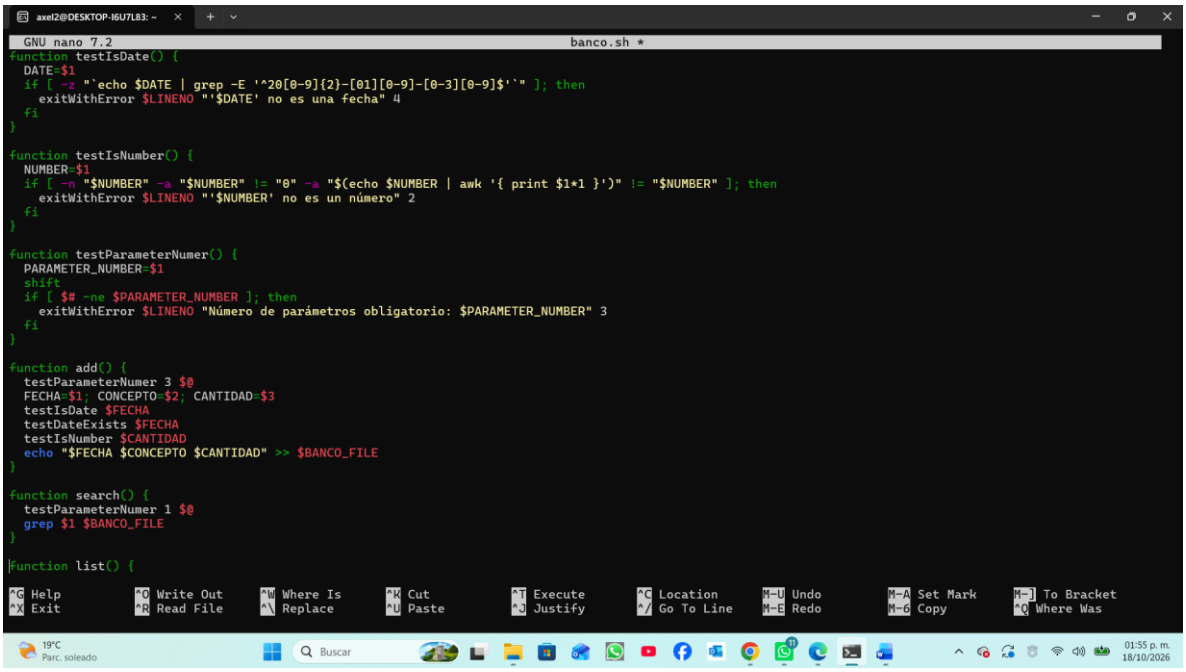
y otro llamado notas.sh ahora vamos a ejecutar y se supine que debería de aparecer una tabla vamos a haber que pasa

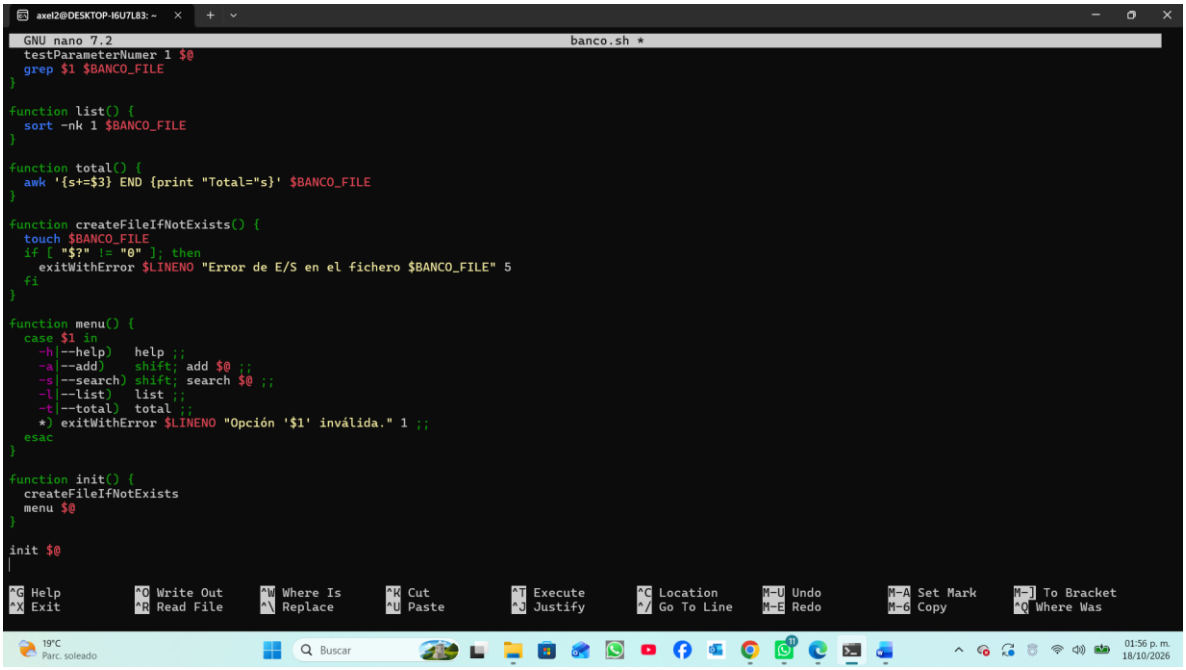


Listo se me olvido darle permisos, pero ahí esta al ejecutar si sale una tabla de lo que habíamos creado en el ejercicio 19, vamos con el siguiente

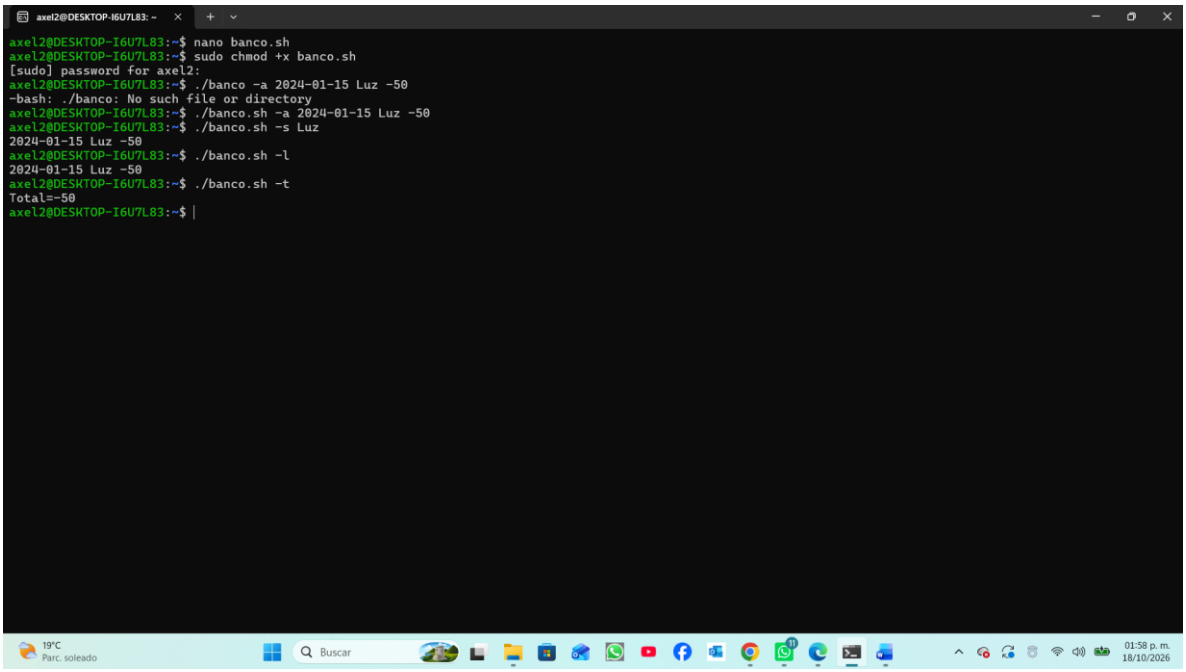
EJERCICIO 21





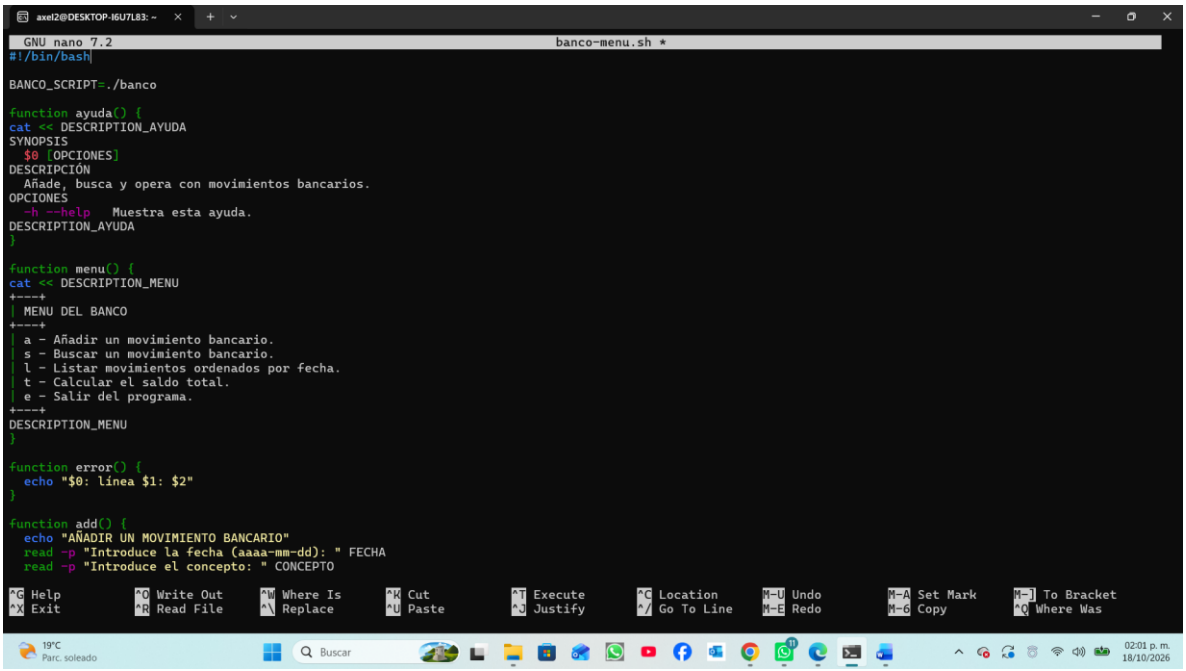


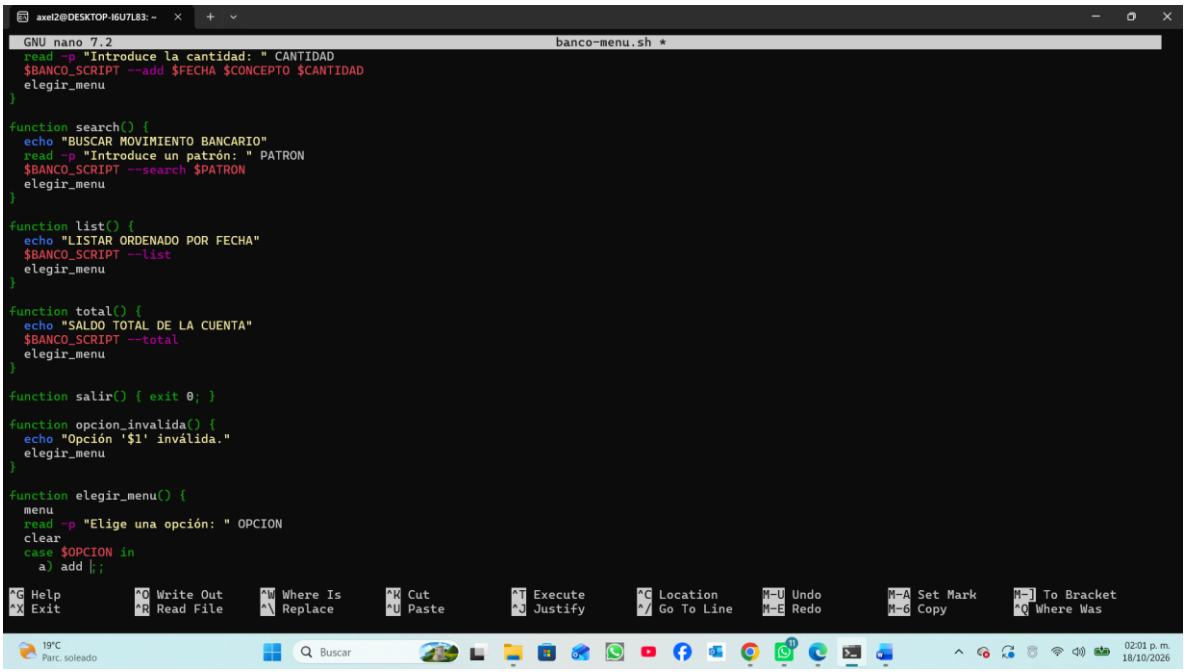
De este ejercicio ahora es de un banco que va a permitir añadir, buscar y listar movimientos del banco y calcular el saldo de la cuenta, vamos a ejecutarlo

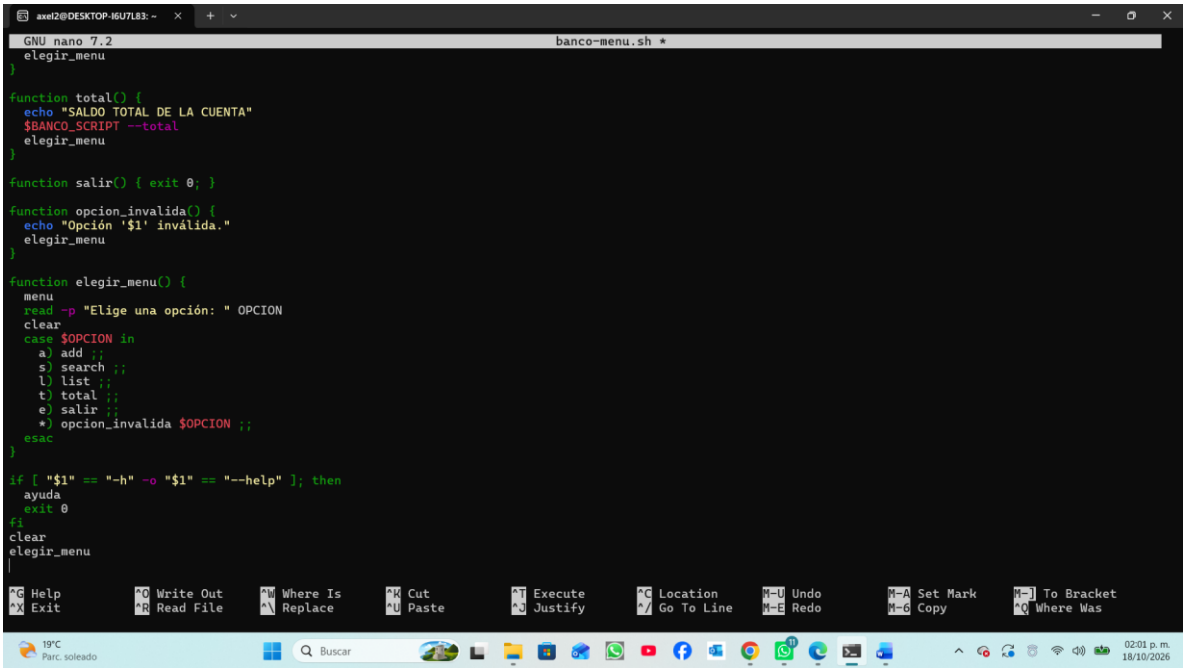


Ahora el ejecutar con el primero de -a es para que se añada un movimiento, el -s es busca ese movimiento, el -l lo lista por la fecha y al final el -t muestra el total de este movimiento

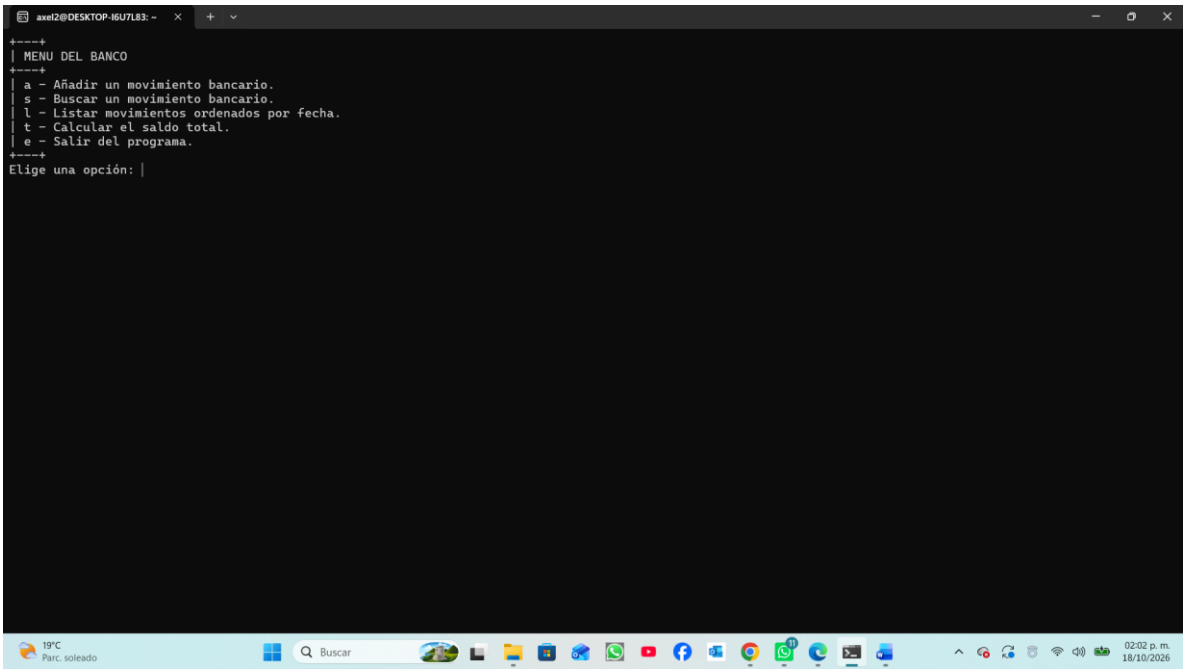
EJERCICIO 22



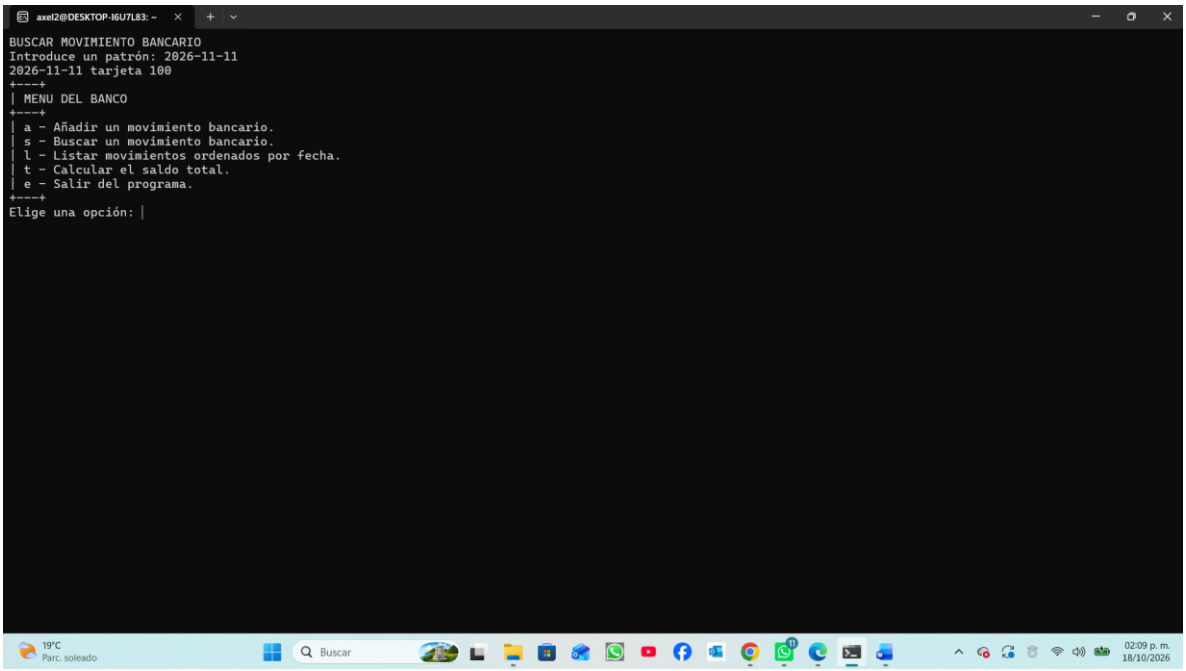




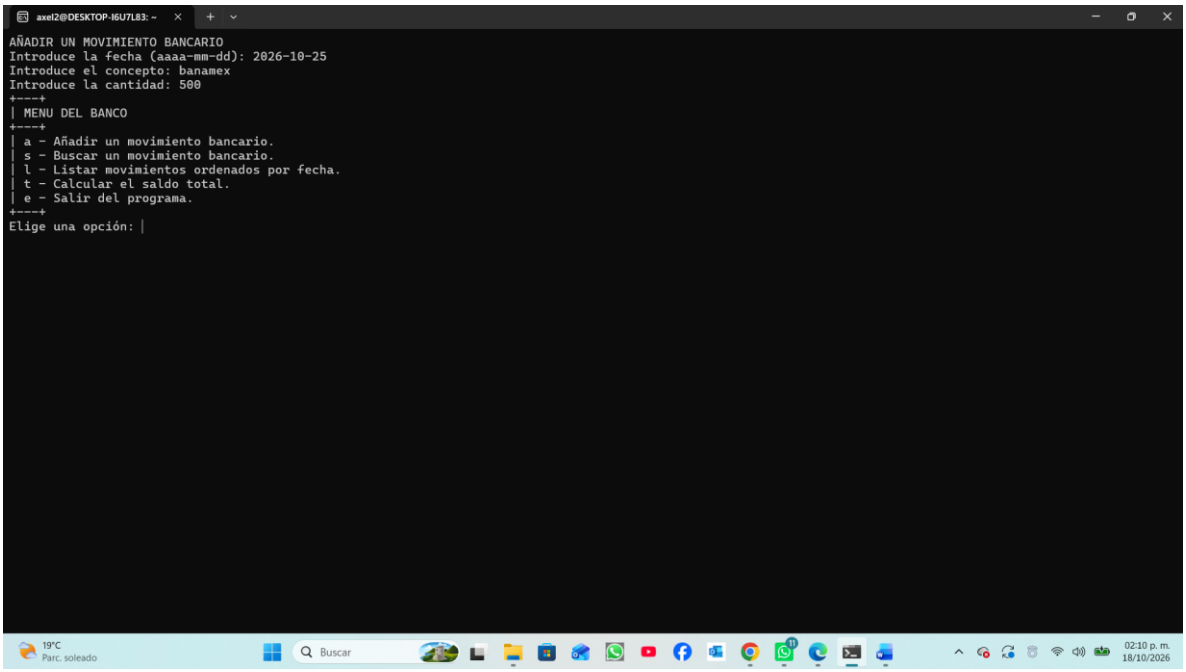
En este ahora va a servir en una interfaz como el anterior ejercicio, vamos a ejecutarlo



Al ejecutar primero aparece esta interfaz ahora vamos a interactuar con ella

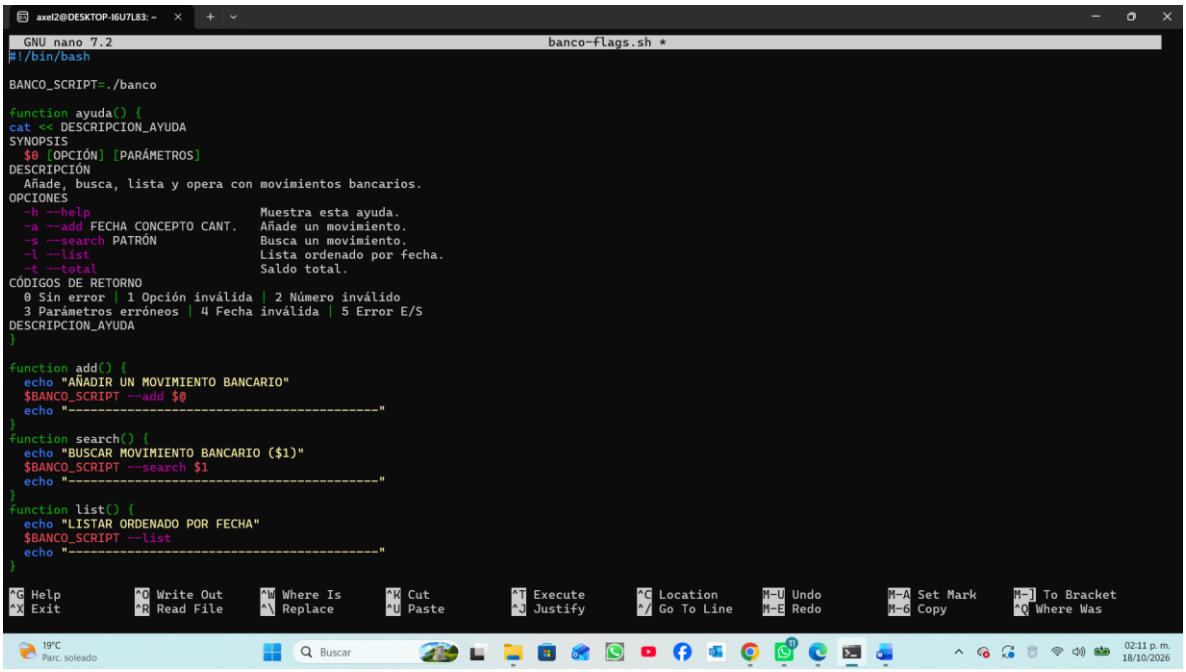


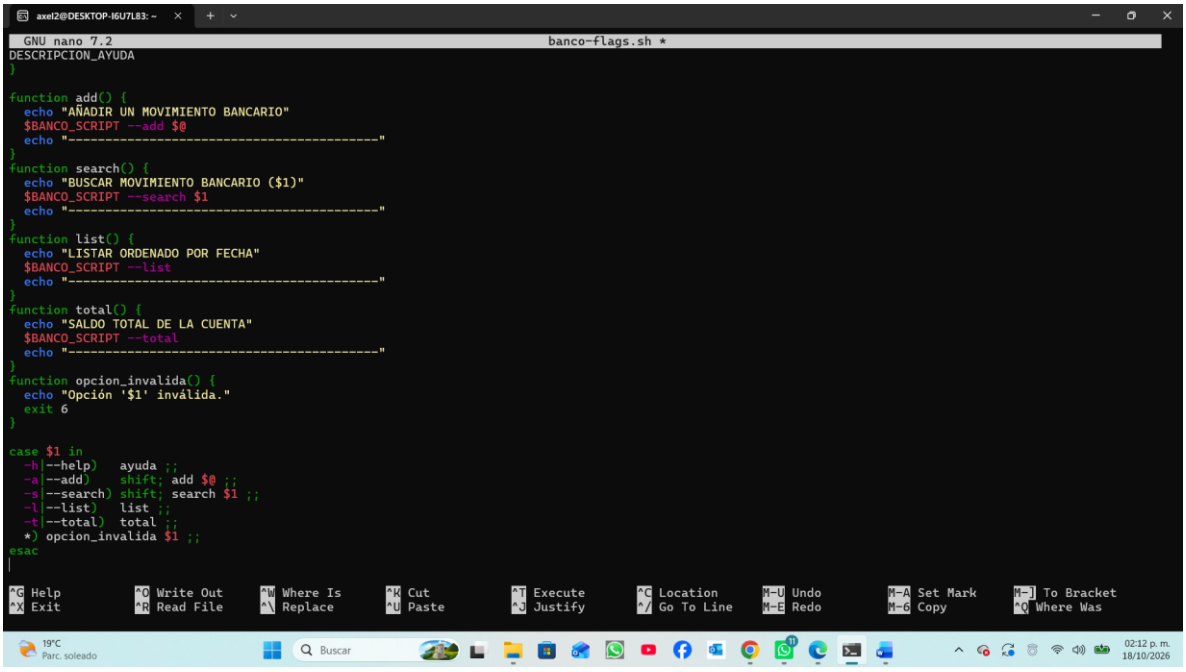
Vemos que ya había guardado uno y al buscar la fecha se pone ese movimiento buscado.



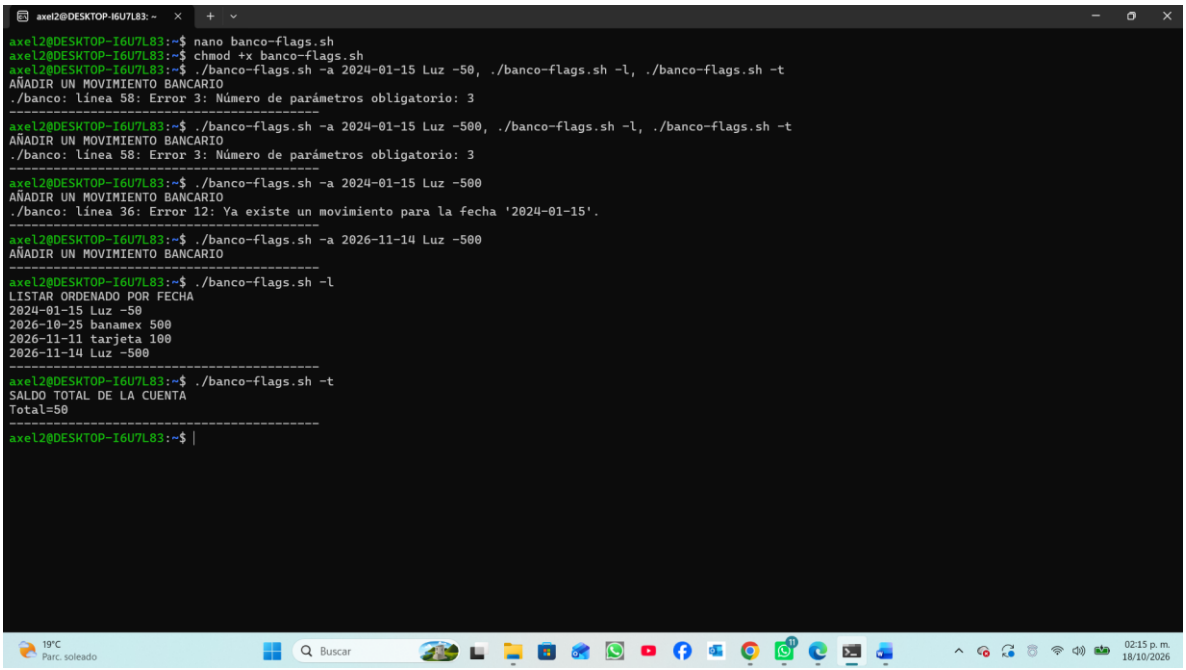
Y en esta imagen añadi un nuevo movimiento con la letra a, así que este script si funciona con esta interfaz

EJERCICIO 23





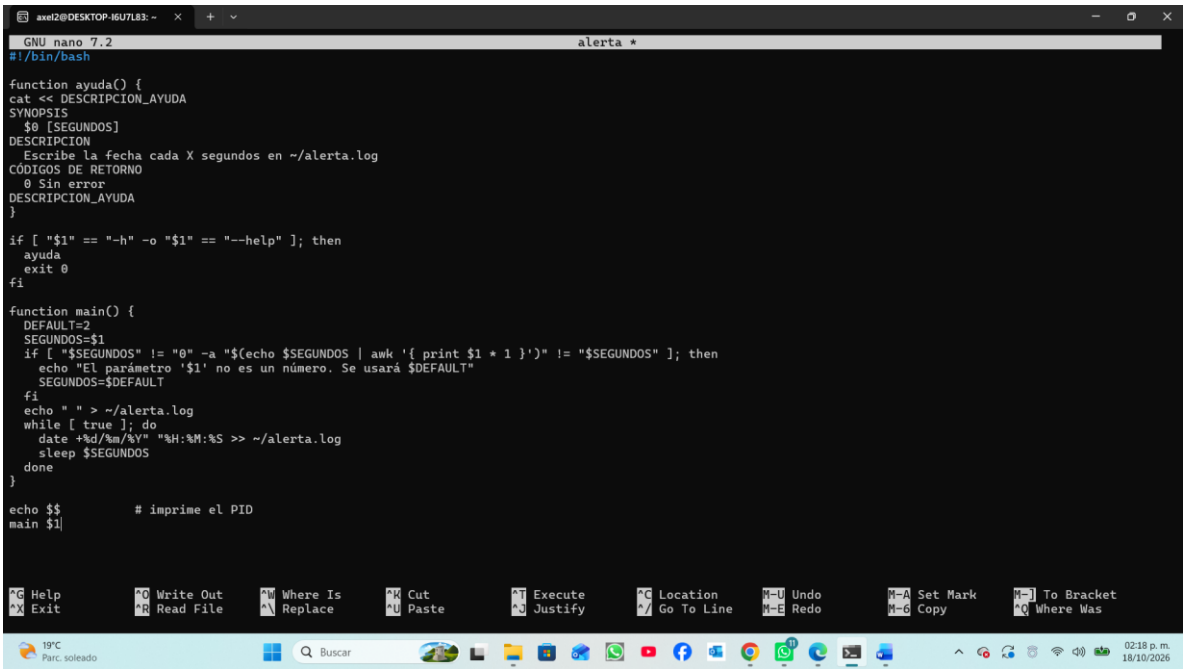
En este ejercicio 23 es para utilizar el banco desde CLI, vamos a ejecutarlo



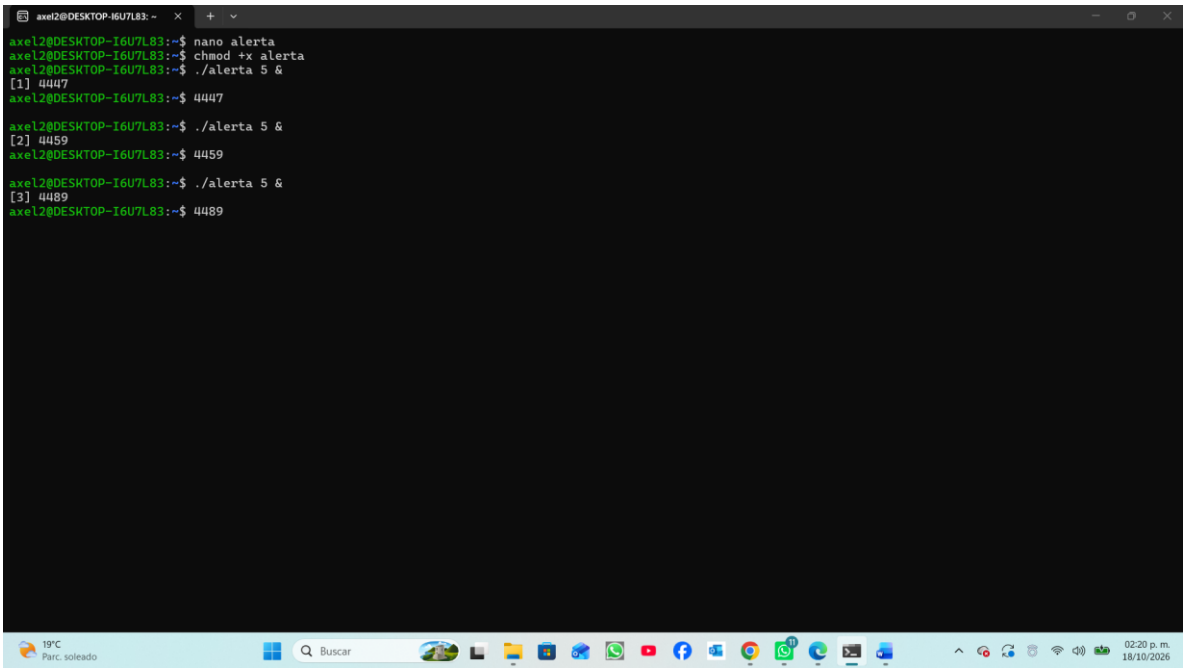
AL ejecutar estos comandos ./banco-flags.sh -a 2026-11-14 Luz -50, ./banco-flags.sh -l, ./banco-flags.sh -t

Se puede ver que las funciones de CLI si funciona y añaden movimientos, ordena y muestra el total

EJERCICIO 24

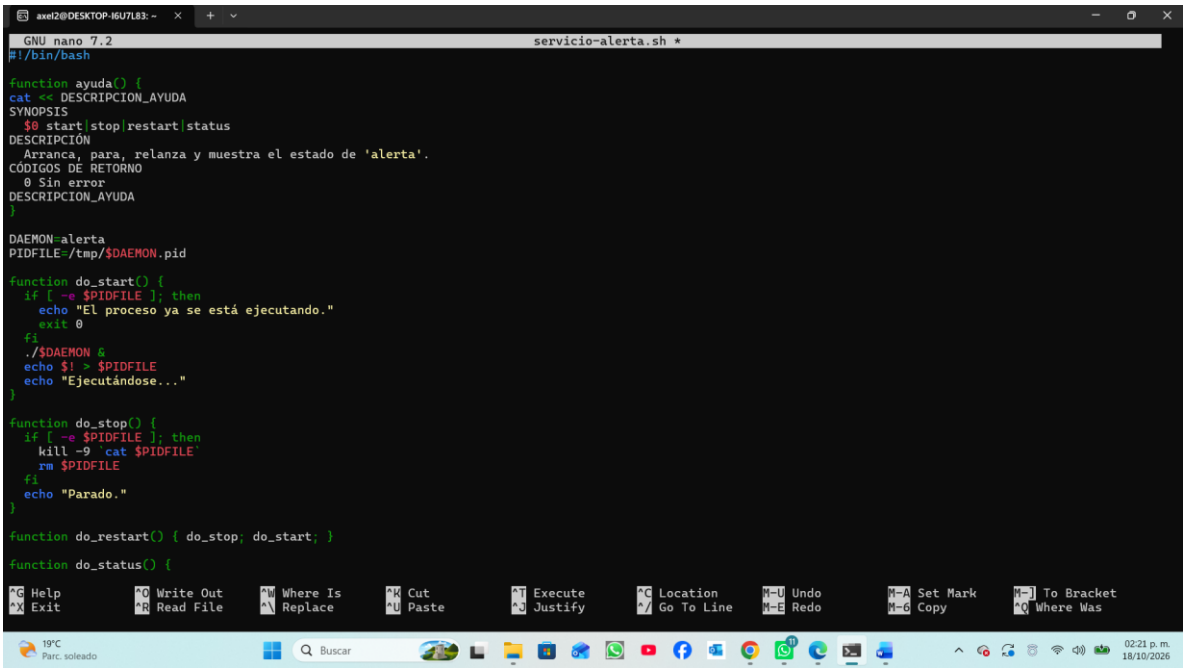


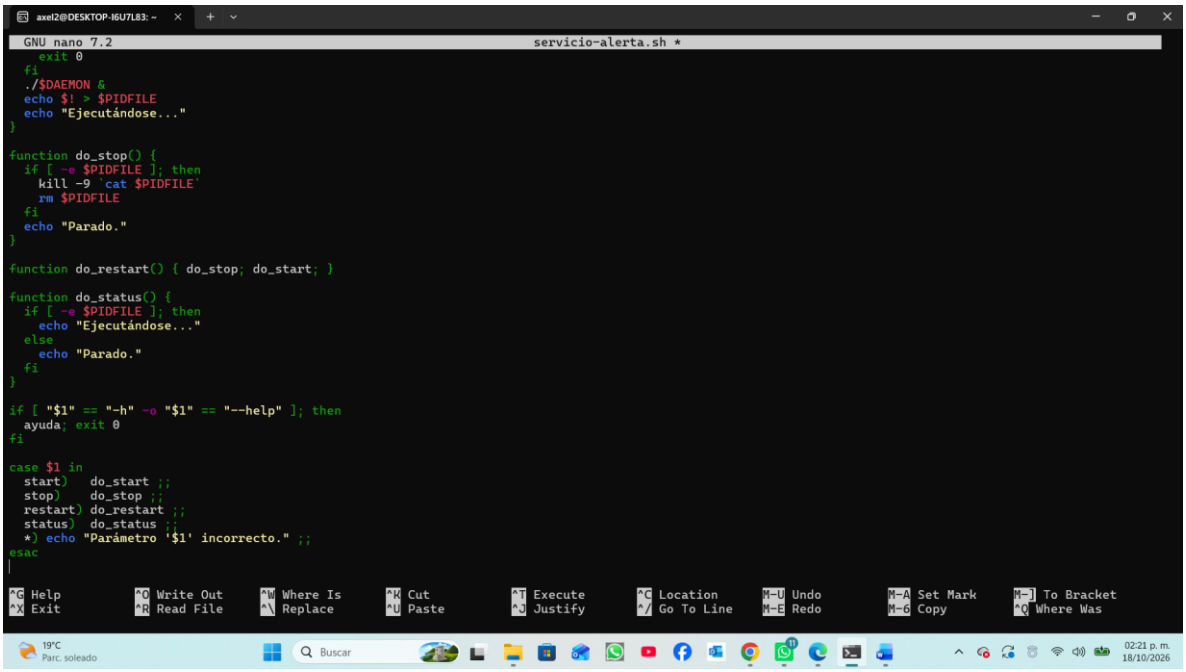
Se crea un archivo llamado alerta es un demonio en donde este que escriba la fecha en un log llamado '~/alerta.log', vamos ejecutarlo



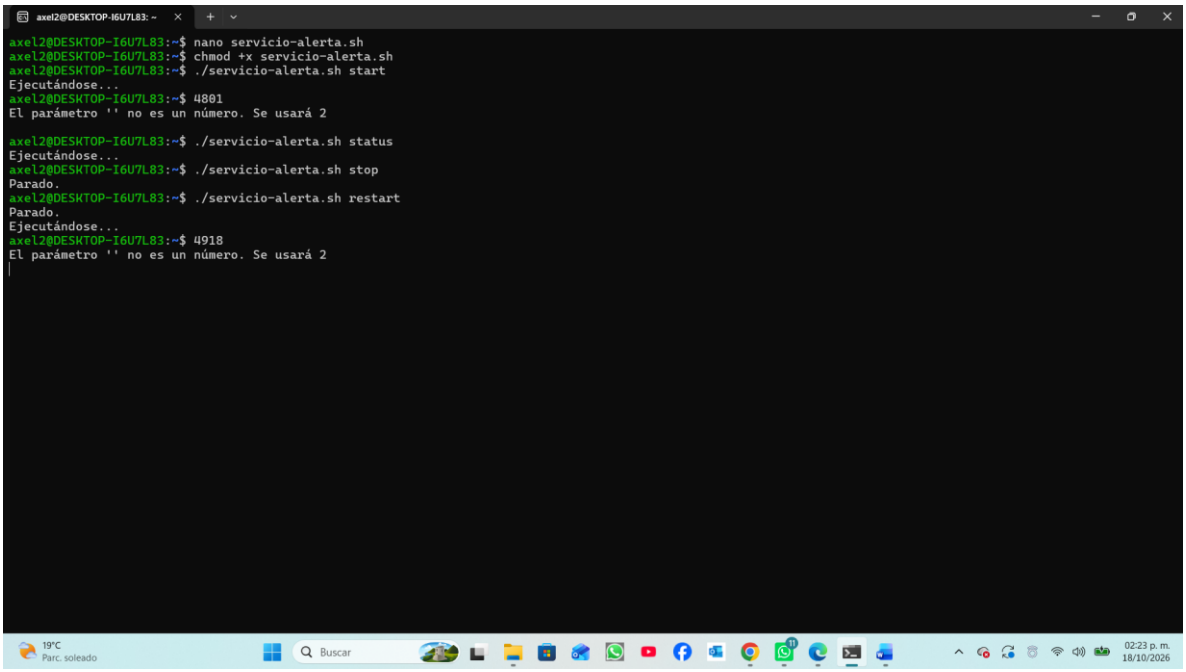
Al ejecutar si muestra la fecha en este caso puse 5 y lo muestra cada 5 segundos

EJERCICIO 25



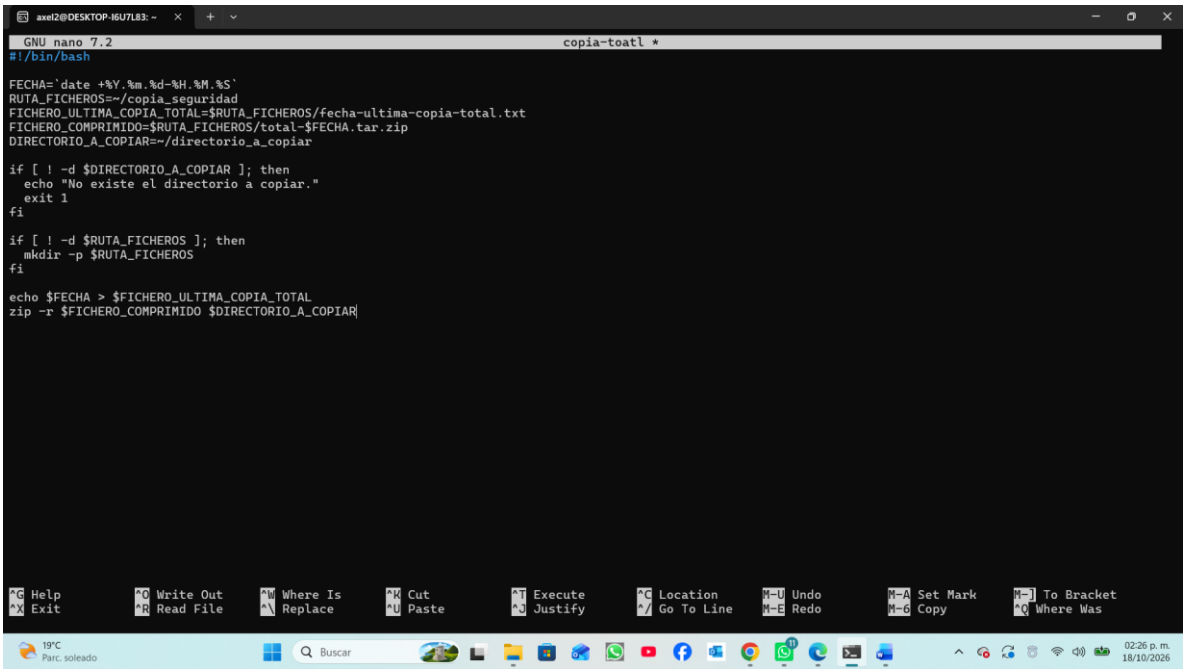


En este ejercicio basado en de alerta, va ahora con interfaces con las opciones básicas, así que vamos a ejecutarlo

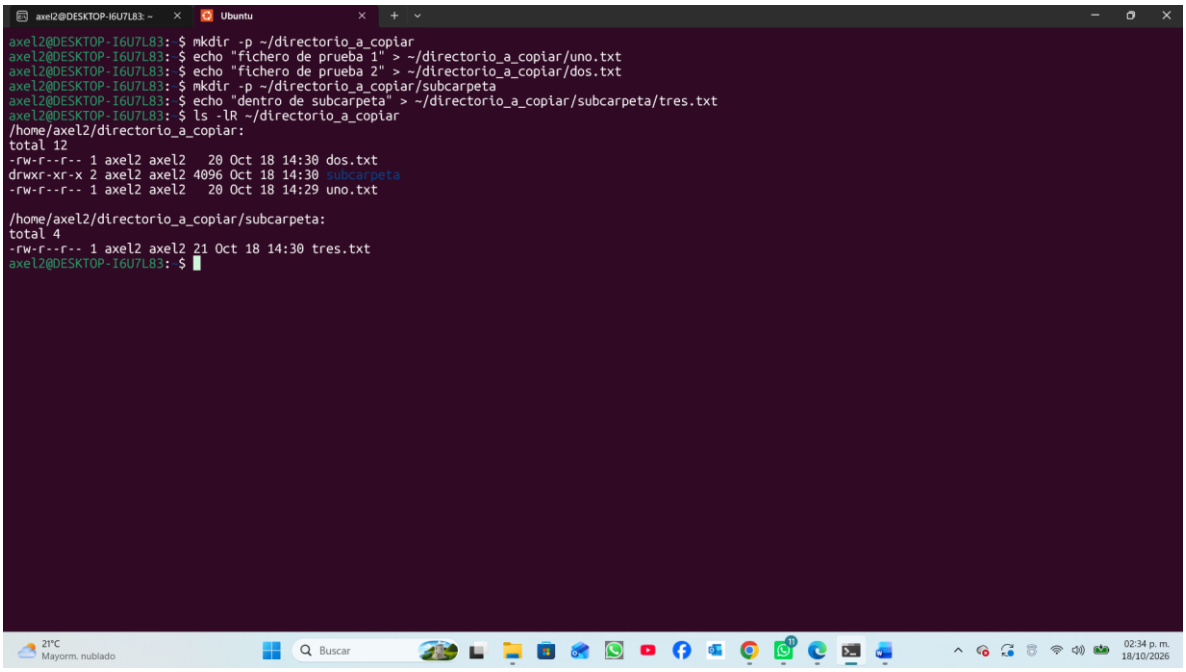


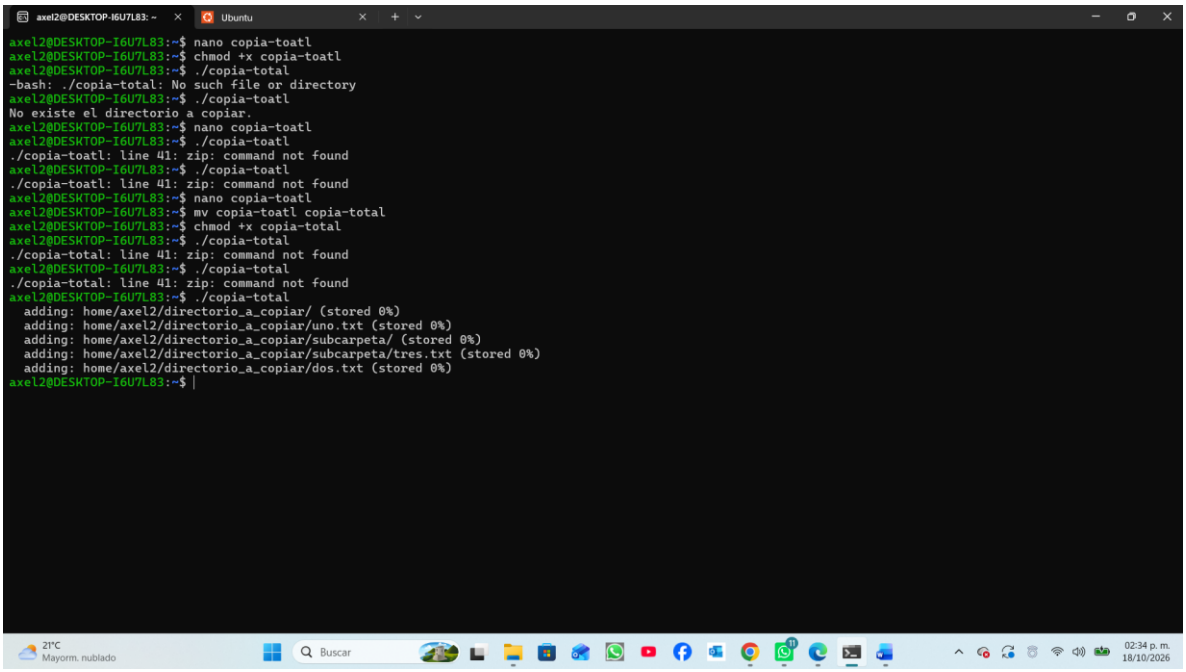
Vemos que los básicos son start, status, stop y restart que cada uno si cumple con su función en este caso jejeje

EJERCICIO 26



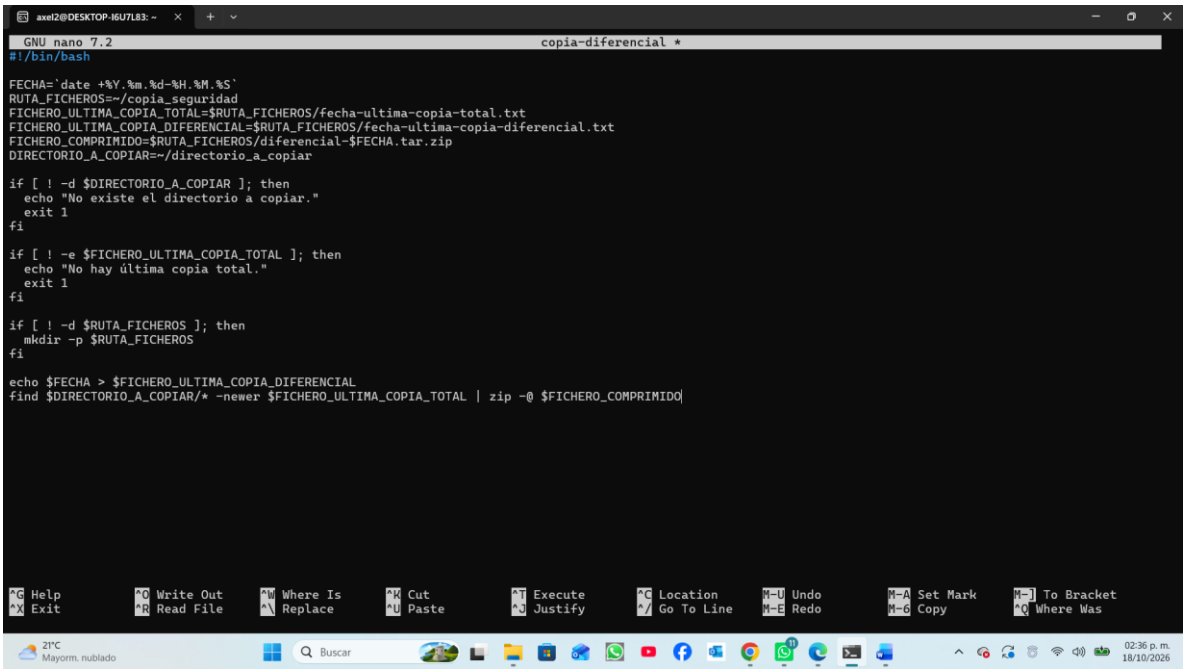
En este va hacer que empaquete y comprima el contenido de la carpeta de total y lo demás es basado en este script ahora vamos a ejecutarlo



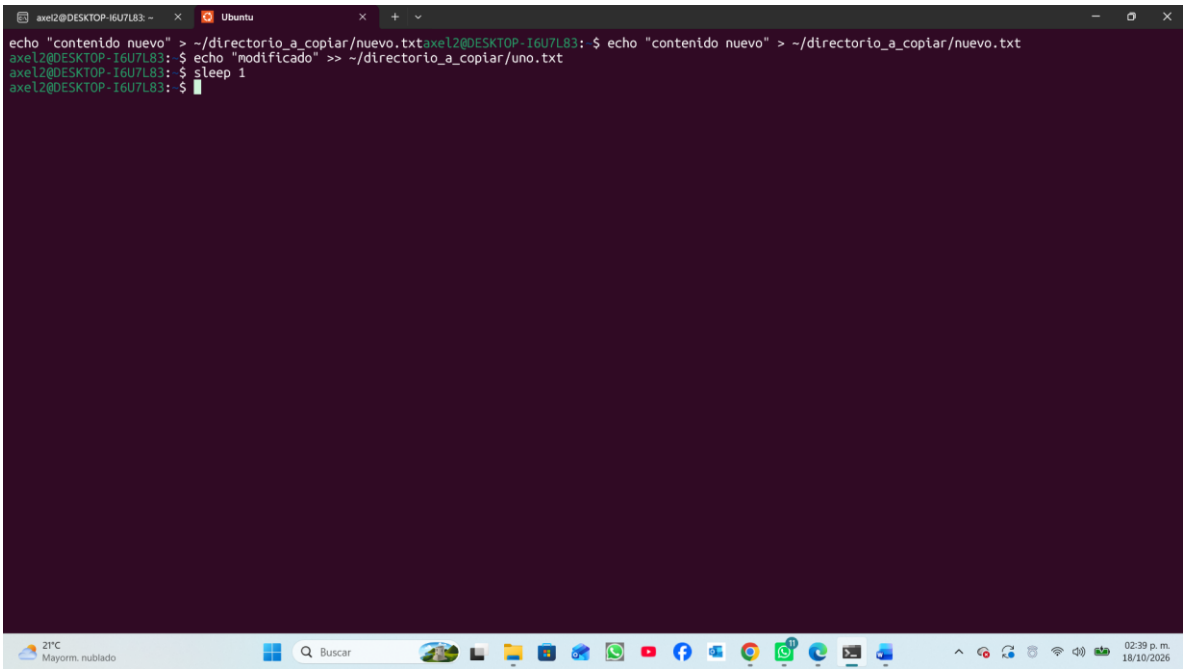


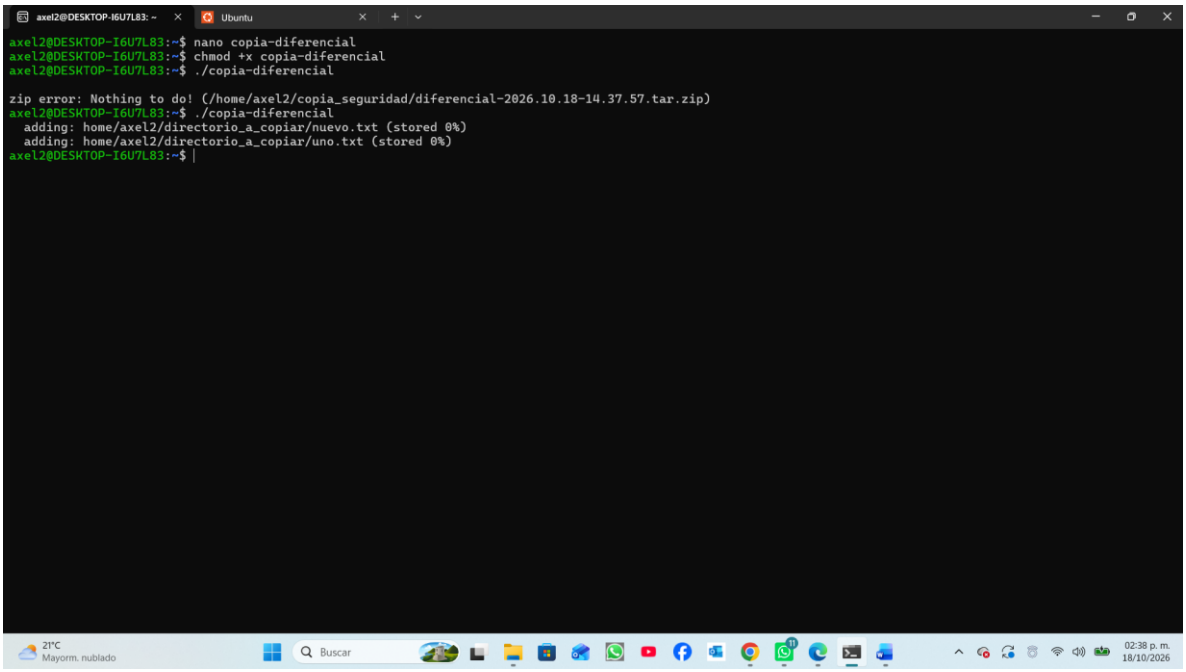
Primeo cree esa carpeta de mkdir para que funcionara y cambiar el nombre al archivo ya que lo puse mal y al ejecutarlo muestra los ficheros

EJERCICIO 27



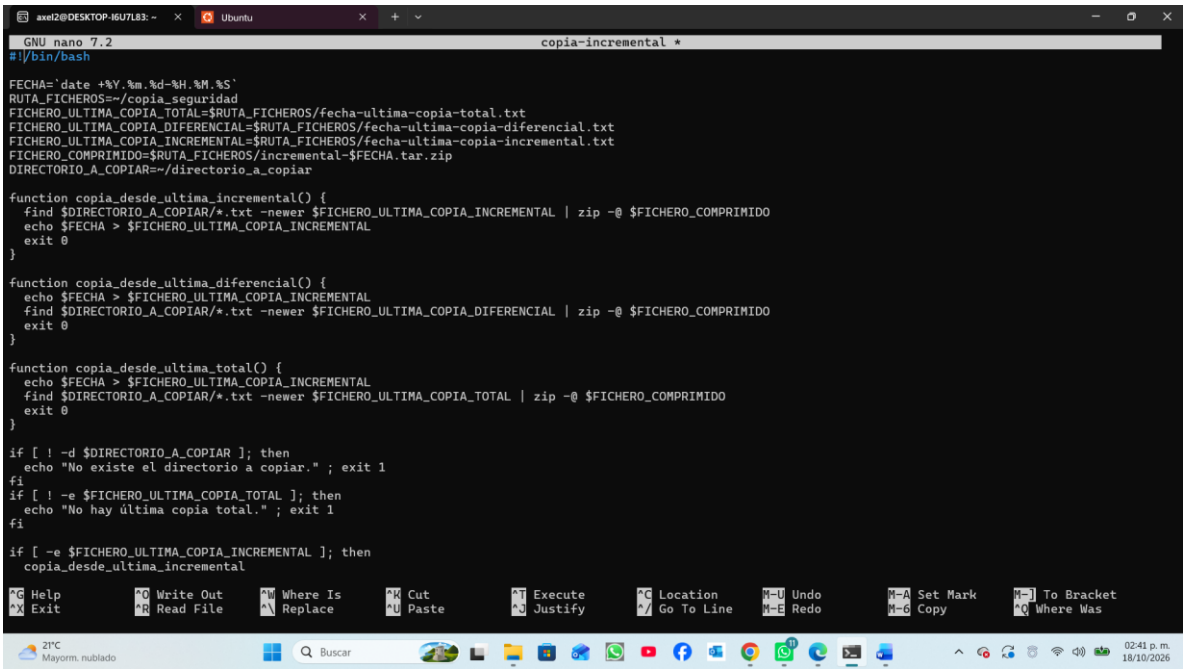
Aquí ahora en el script de archivo diferencial es para modificarlos desde la ultima copia total en un nuevo fichero, vamos a ejecutarlo



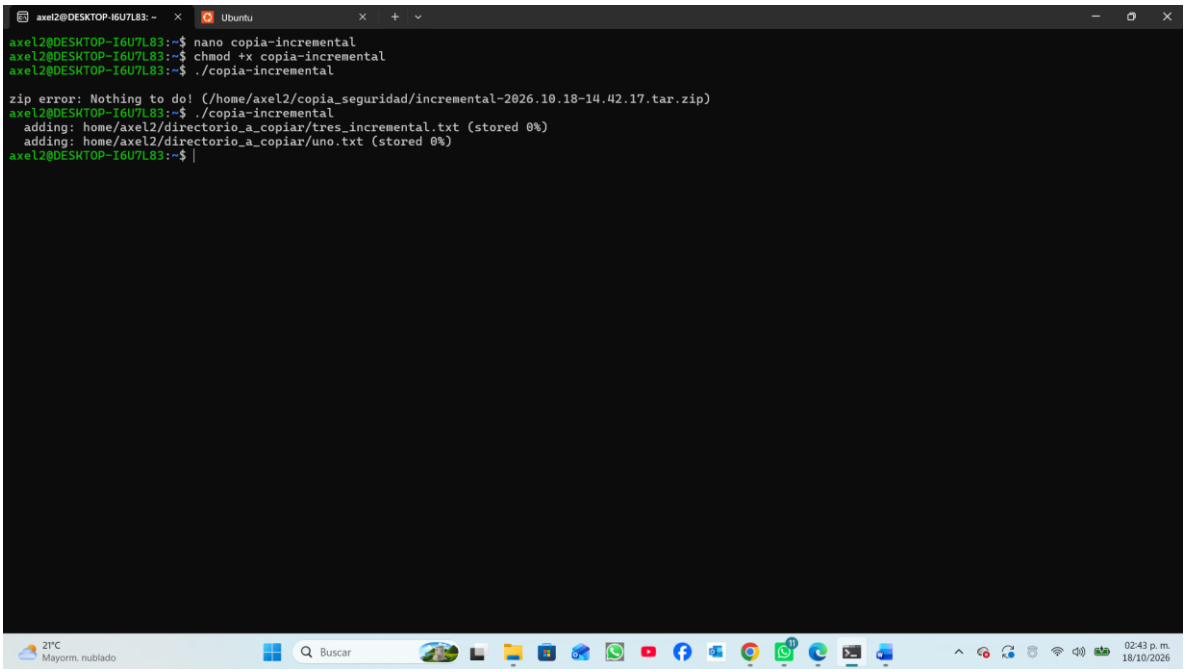


Tuve que crear un echo de nuevo contenido para que se pudiera ejecutar y al ejecutar muestra los otros ficheros con un 0% stored

EJERCICIO 28

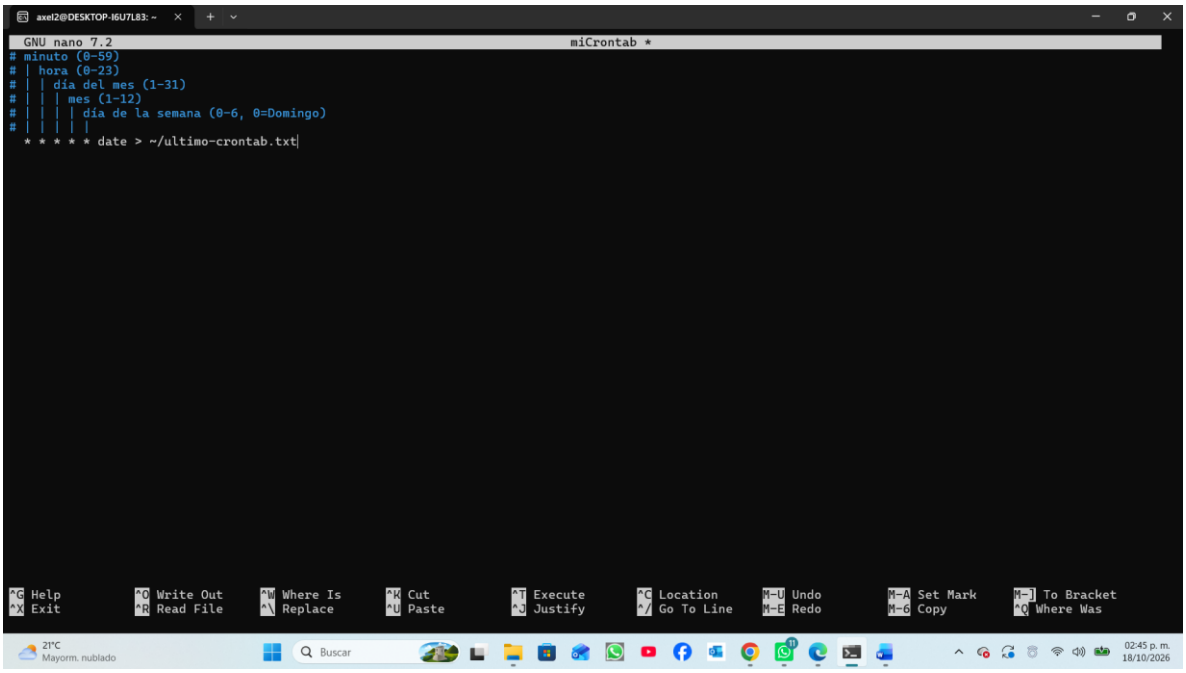


Ahora en este que es de incremental es lo va a modificra desd ela ultima copia y ahora vamos a ejecutar

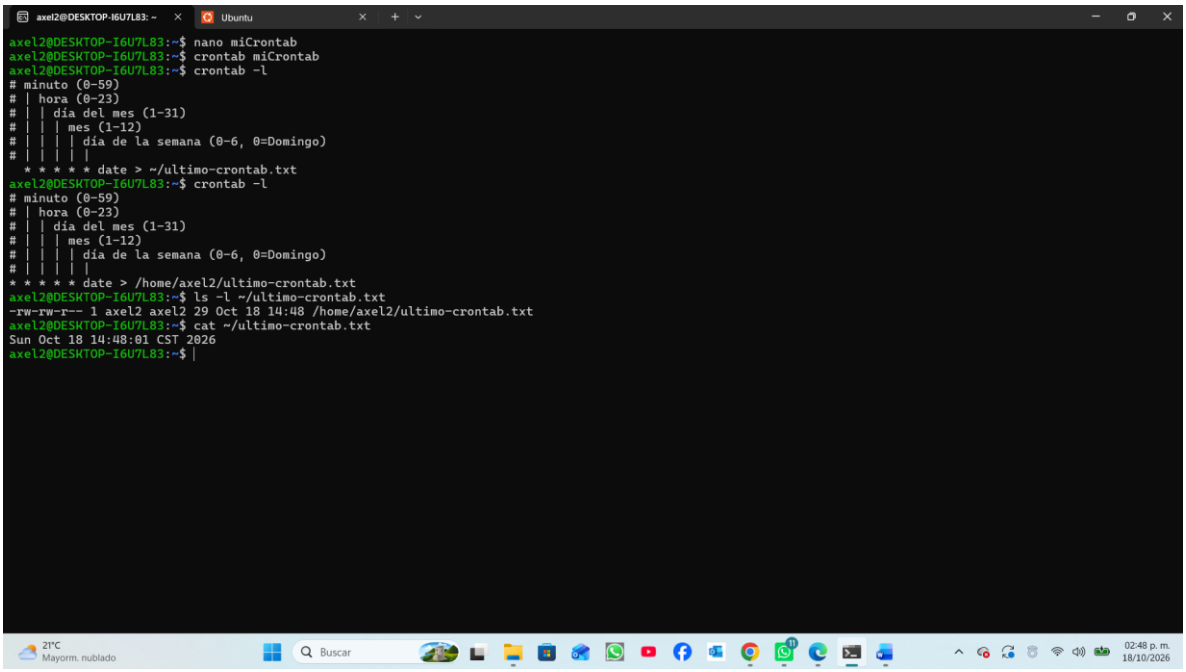


Vemos que si funciono entonces

EJERCICIO 29

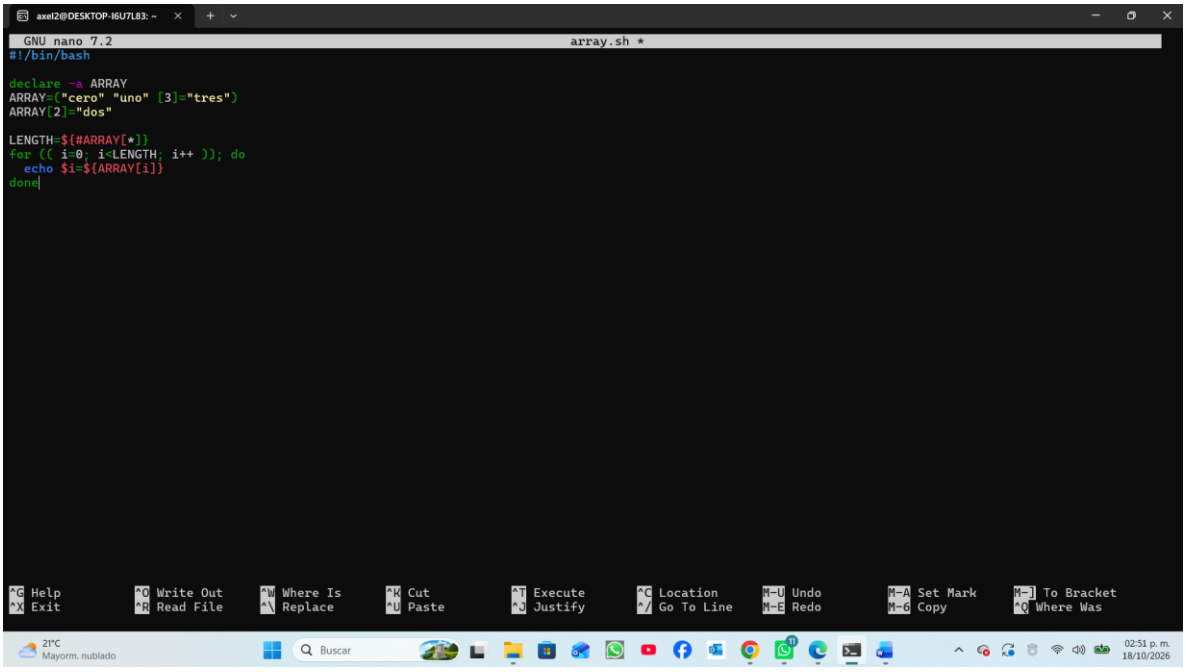


En un archivo miCrontab ponemos lo siguiente para que imprima la fecha un in fichero txt, vamos a ejecutarlo

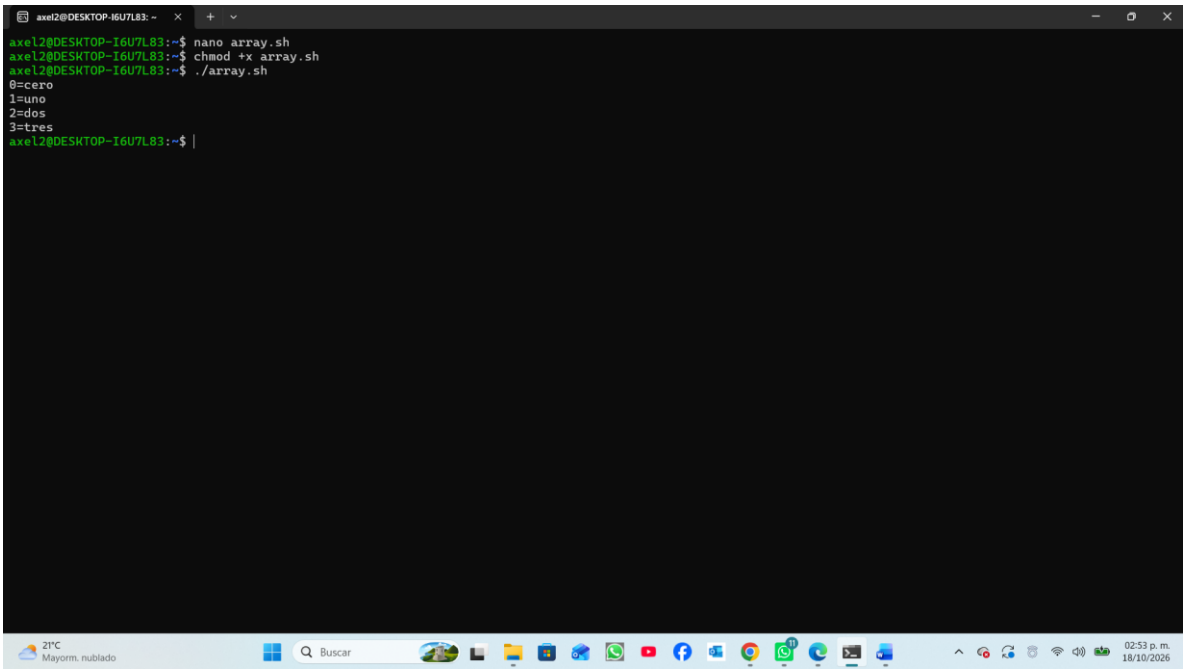


Y vemos que si funciono paso 1 minuto y salió el txt de ultimo-crontab.txt y con el cat se ve su contenido de este txt

EJERCICIO 30

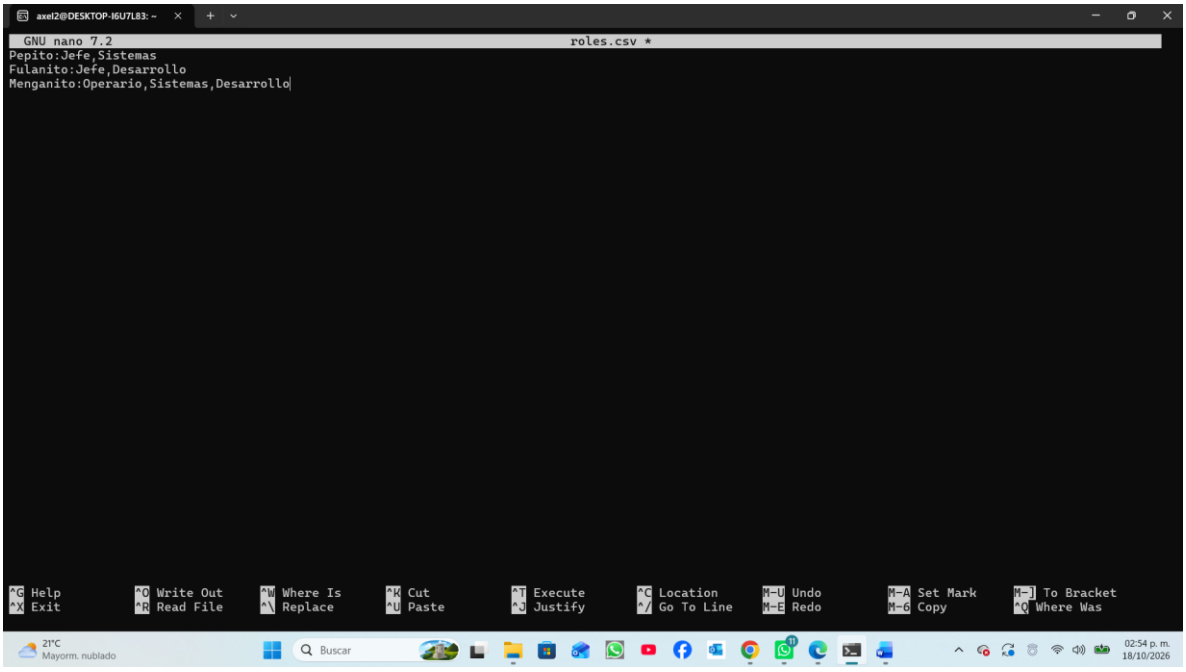


Es crear un script llamado array donde declare el array y relleno datos y itere sobre ese para mostrar los datos así que vamos a ejecutarlo



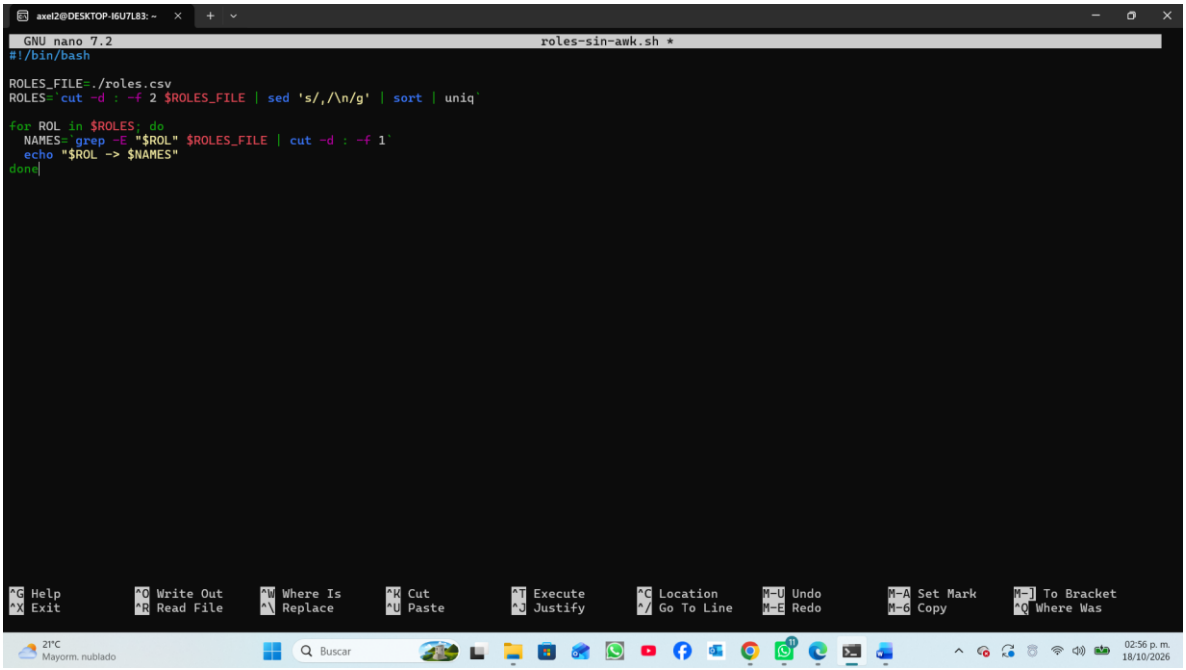
Vemos que imprime estos 4 datos empezando del 0 y termina en 3 y va desde cero a tres, así que esta función del array su funciono

EJERCICIO 31

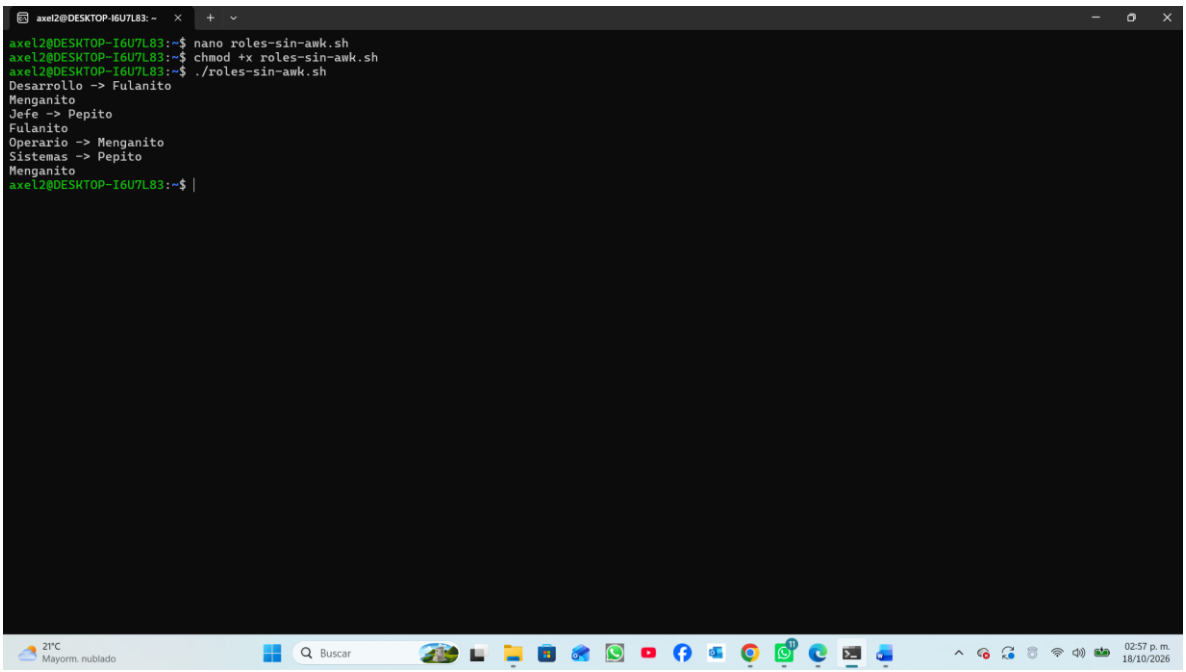


Para el ejercicio 32 y 33 primero se crea un archivo llamado roles.csv que va a contener estos datos

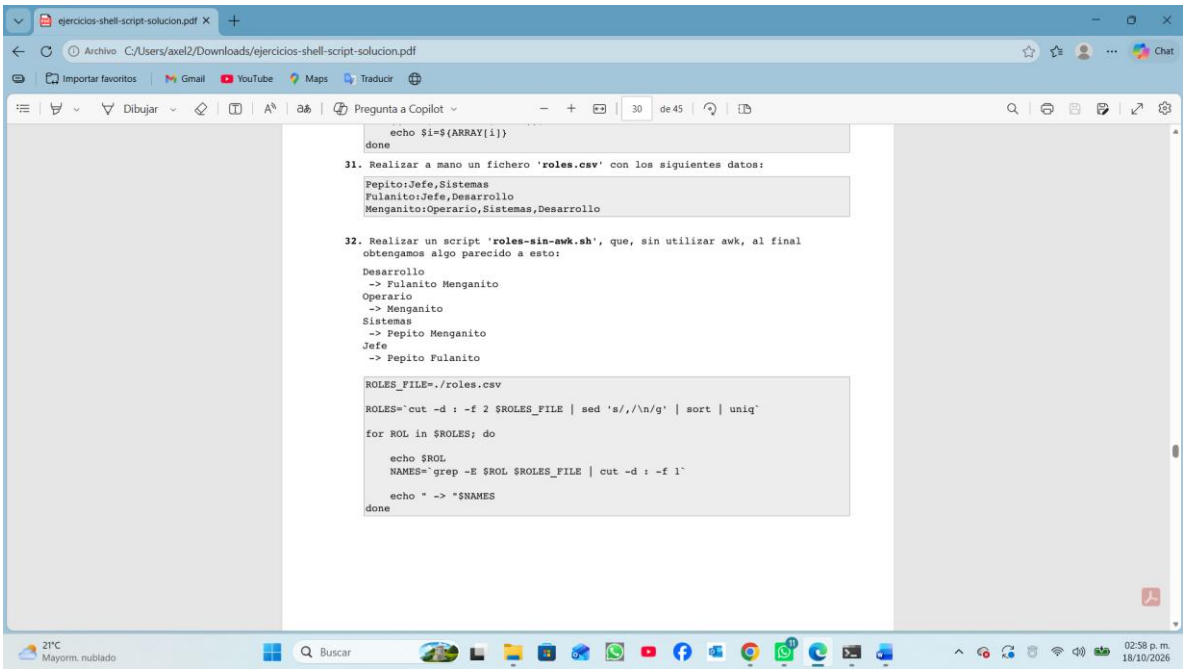
EJERCICIO 32



Con este script de roles-sin-awk.sh se va a obtener lo siguiente

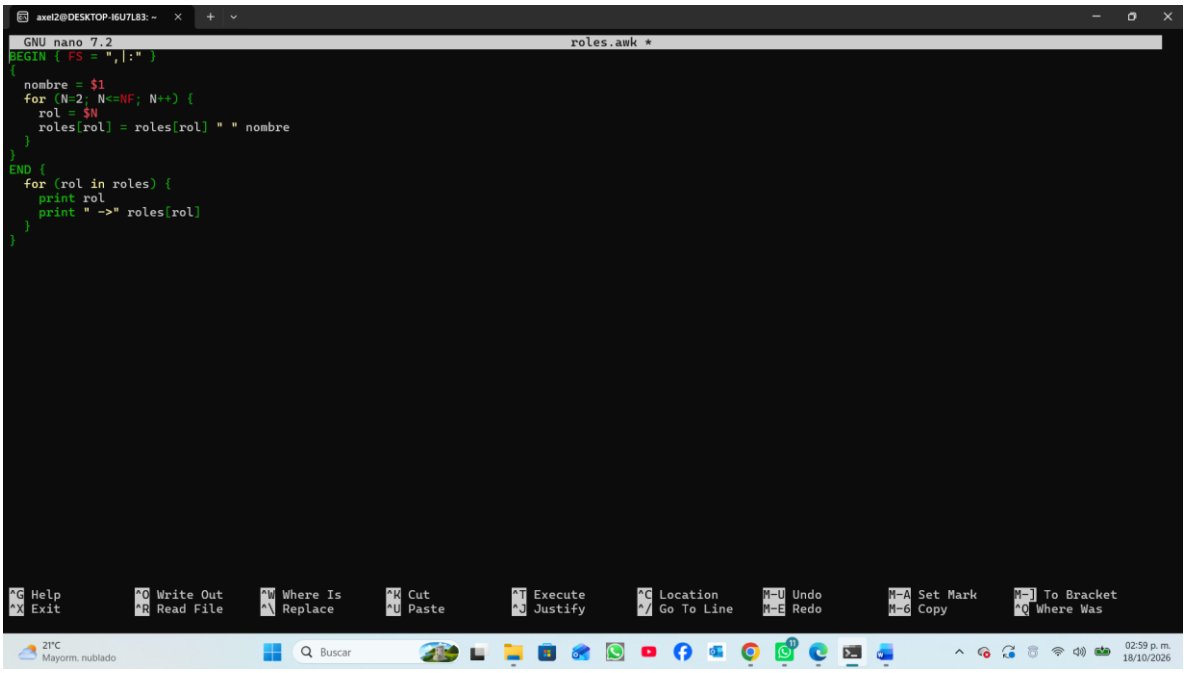


Y si se obtiene eso como lo digo el pdf de estos scripts

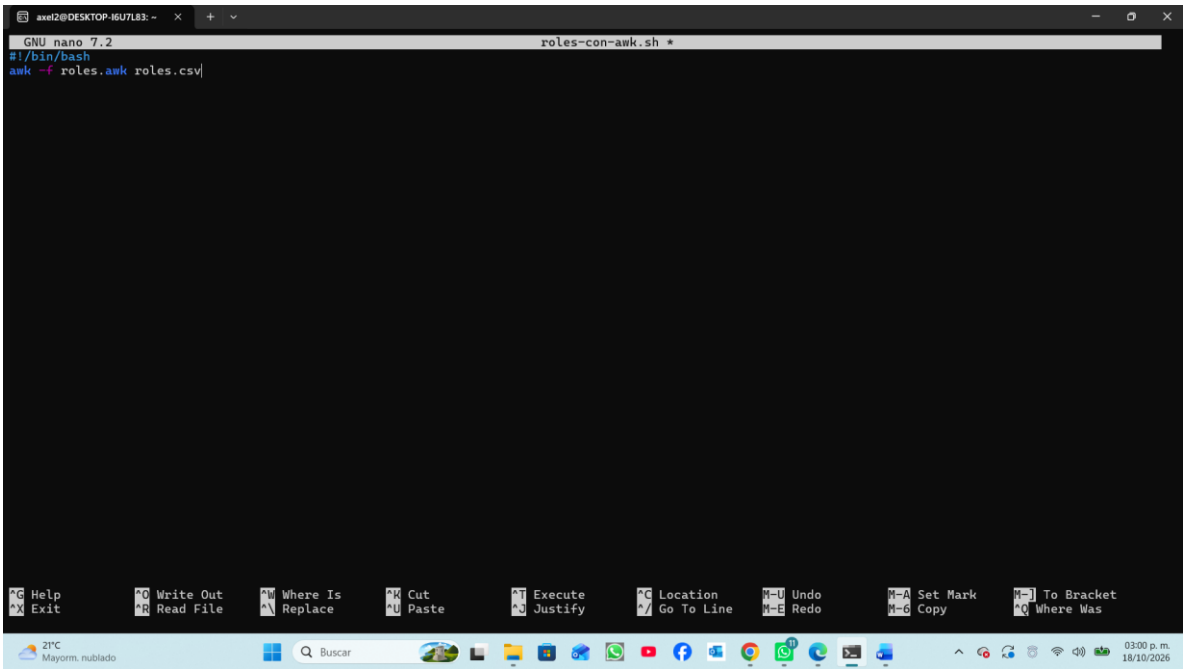


Y ahí esta la prueba a así que si funciona este

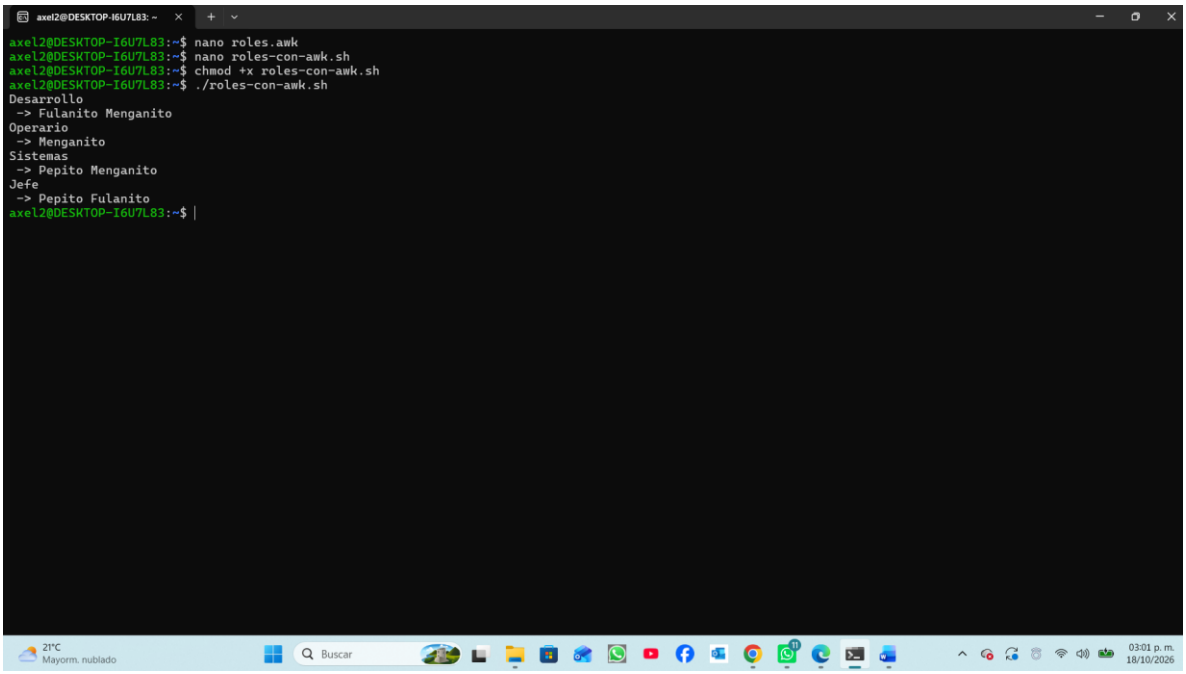
EJERCICIO 33



De este se crea un archivo llamado roles.ask que tiene esto



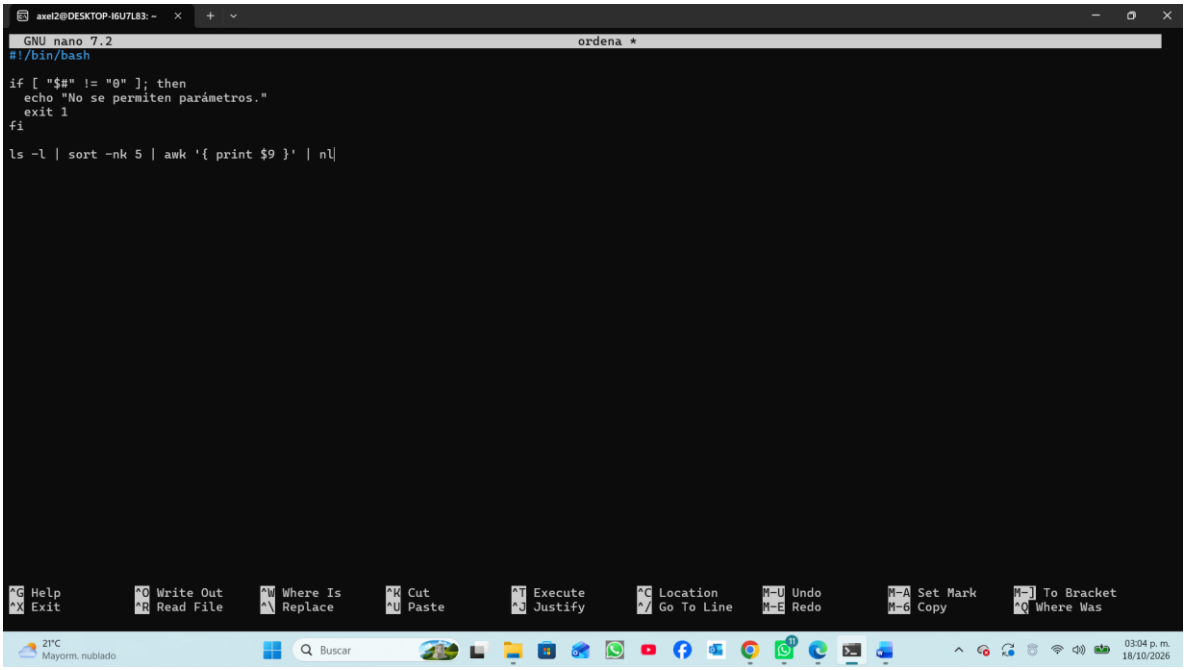
Y también se crea este que el de roles-con-ask.sh, ahora si tiene los awk vamos a ejecutarlo



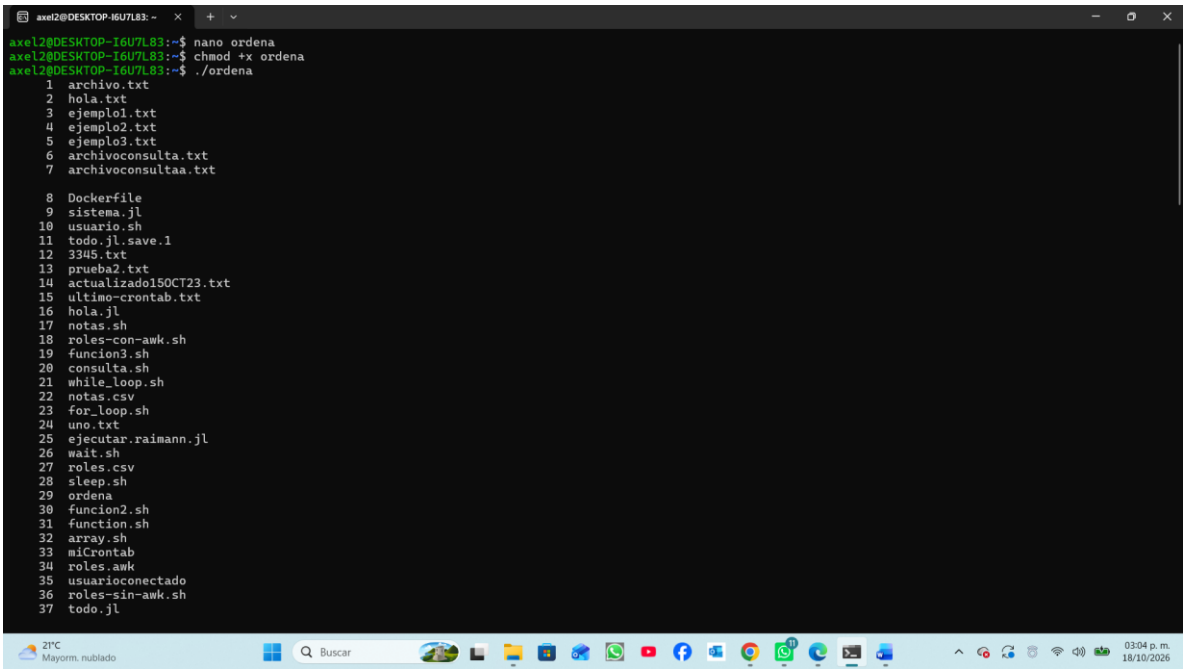
Esto ahora sale al ejecutarlo así que si funciono con awk

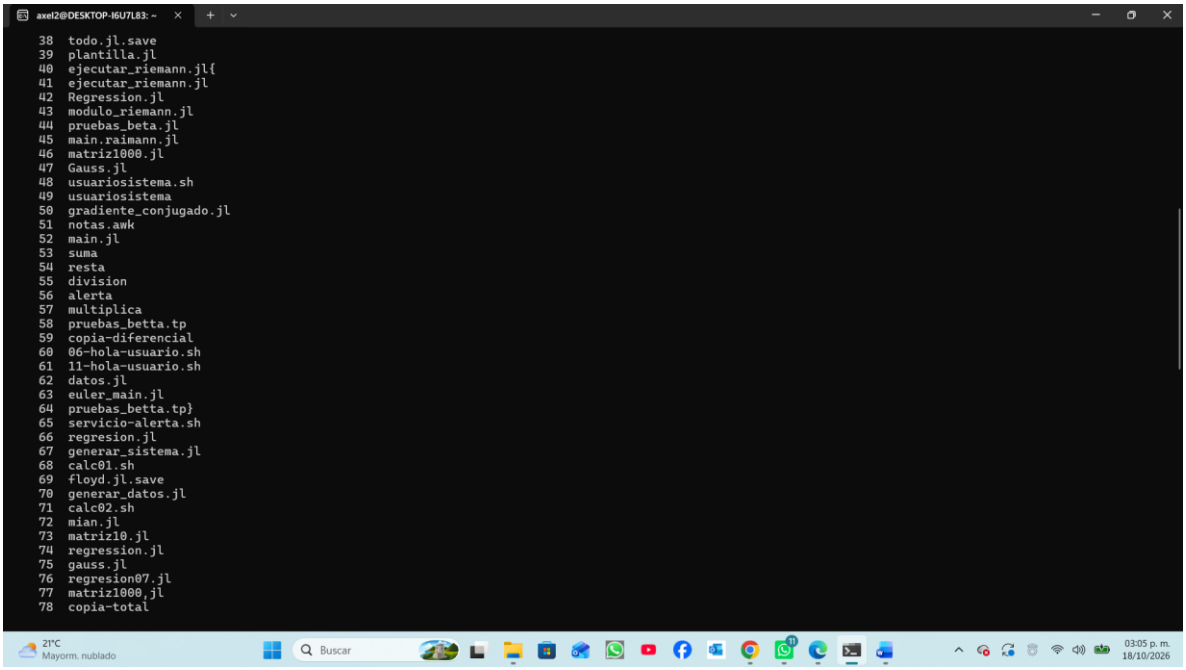
EJERCICIO 34

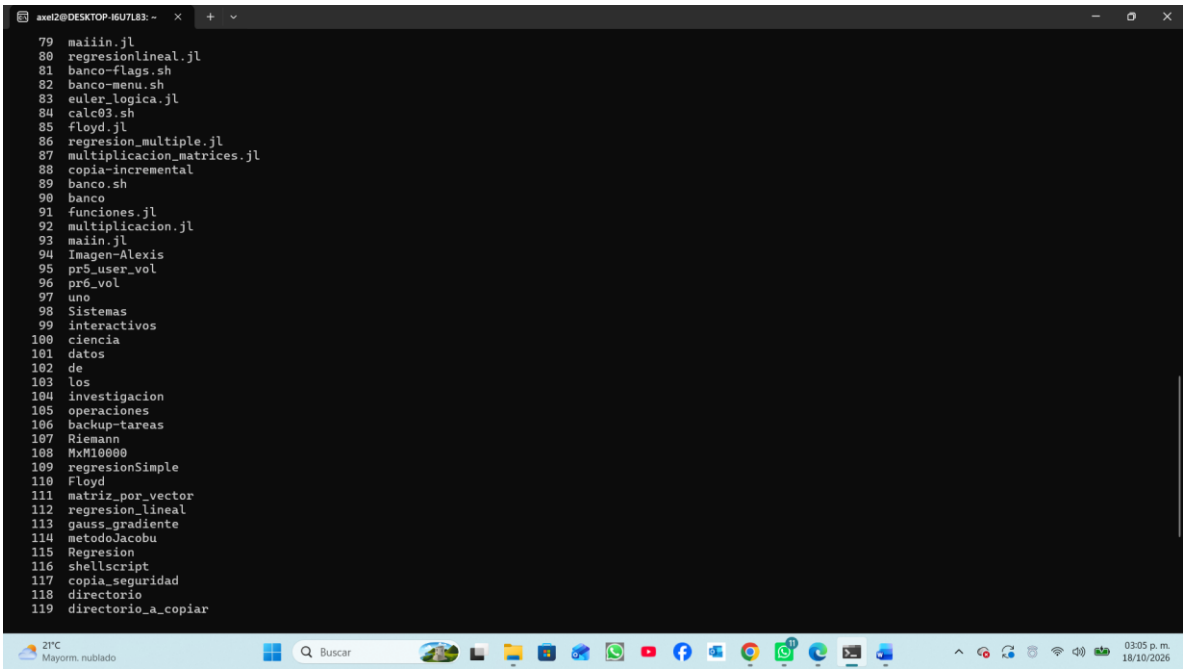
Realizar un script llamado 'ordena' que liste el contenido del directorio actual ordena do por tamaño del archivo de menor a mayor. El listado sólo mostrará el nombre de lo s archivos y el número de línea correspondiente. En el
caso de que se introduzca algún parámetro se mostrará el siguiente mensaje de error : “No se permiten parámetros.” y retornará un código de retorno igual a 1.
ESTE SI LO PUSE TAL EL TEXTO DEL PDF PARA QUE SE ENTIENDA MEJOR JEJE

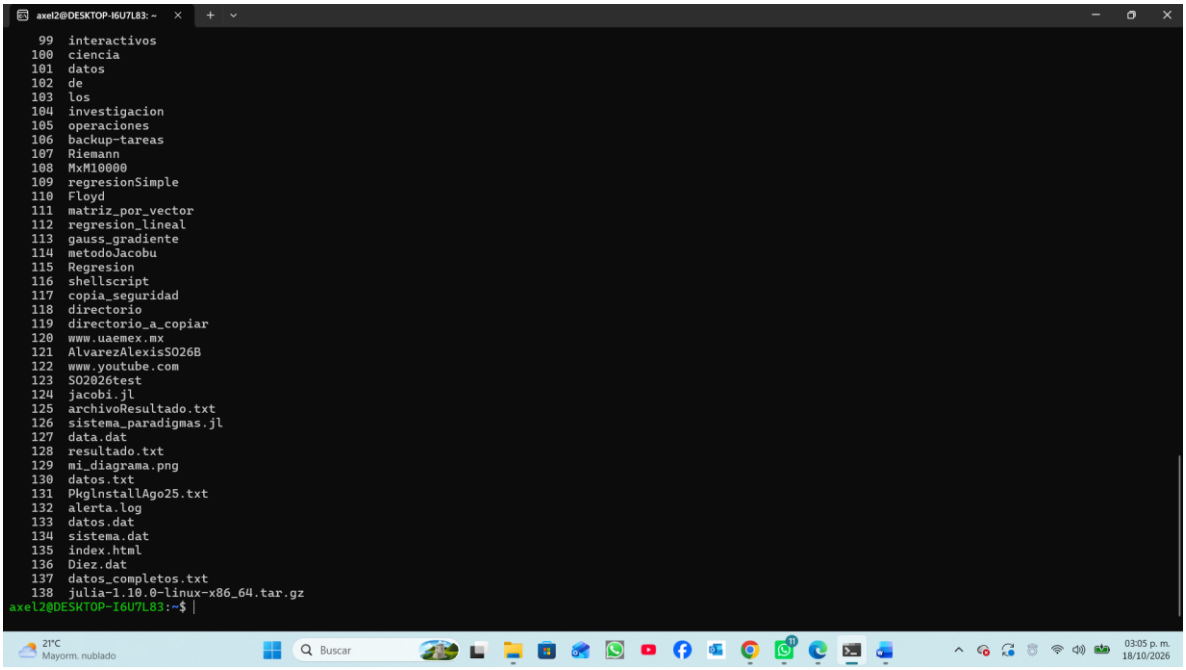


Y así que esta el script ahora vamos a ejecutarlo


axel2@DESKTOP-I6U7L83: ~
axel2@DESKTOP-I6U7L83:~$ nano ordena
axel2@DESKTOP-I6U7L83:~$ chmod +x ordena
axel2@DESKTOP-I6U7L83:~$ ./ordena
1 archivo.txt
2 hola.txt
3 ejemplo1.txt
4 ejemplo2.txt
5 ejemplo3.txt
6 archivoconsulta.txt
7 archivoconsultaa.txt
8 Dockerfile
9 sistema.jl
10 usuario.sh
11 todo.jl.save.1
12 3345.txt
13 prueba2.txt
14 actualizado15OCT23.txt
15 ultimo-crontab.txt
16 hola.jl
17 notas.sh
18 roles-con-awk.sh
19 funcion3.sh
20 consulta.sh
21 while_loop.sh
22 notas.csv
23 for_loop.sh
24 uno.txt
25 ejecutar.raimann.jl
26 wait.sh
27 roles.csv
28 sleep.sh
29 ordena
30 funcion2.sh
31 function.sh
32 array.sh
33 miCrontab
34 roles.awk
35 usuarioconectado
36 roles-sin-awk.sh
37 todo.jl
21°C
Mayorm. nublado
Buscar
03:04 p. m.
18/10/2026

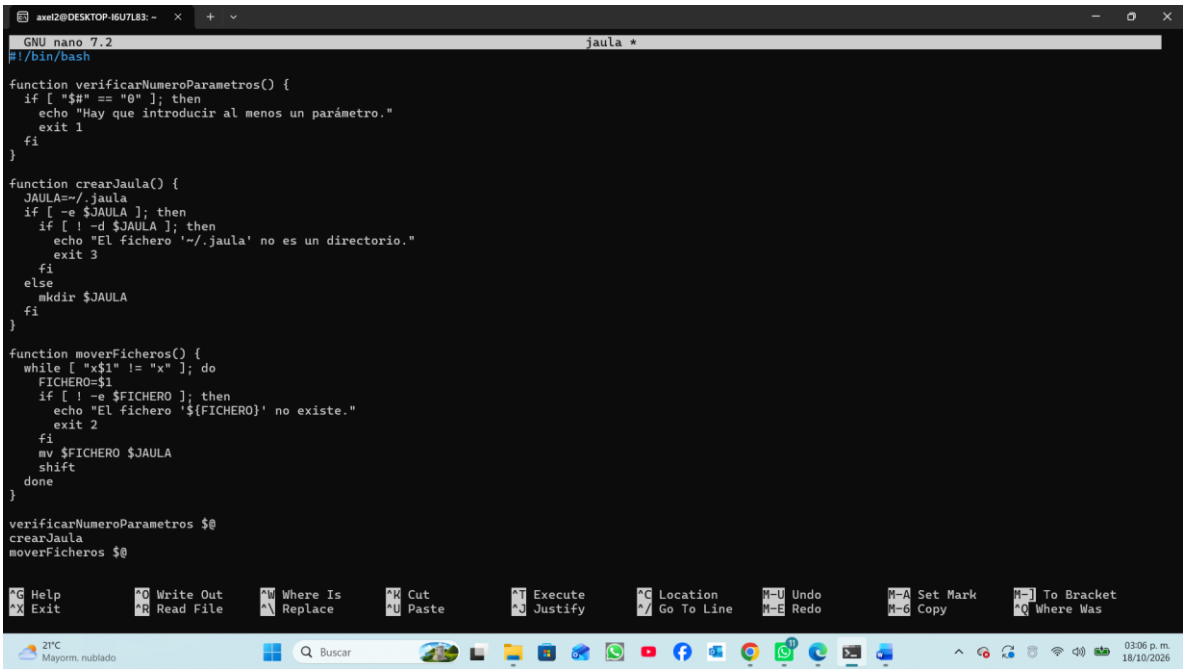

axel2@DESKTOP-I6U7L83: ~
38 todo.jl.save
39 plantilla.jl
40 ejecutar_riemann.jl{
41 ejecutar_riemann.jl
42 Regression.jl
43 modulo_riemann.jl
44 pruebas_beta.jl
45 main.raimann.jl
46 matriz1000.jl
47 Gauss.jl
48 usuariosistema.sh
49 usuariosistema
50 gradiente_conjugado.jl
51 notas.awk
52 main.jl
53 suma
54 resta
55 division
56 alerta
57 multiplica
58 pruebas_betta.tp
59 copia-diferencial
60 06-hola-usuario.sh
61 11-hola-usuario.sh
62 datos.jl
63 euler_main.jl
64 pruebas_betta.tp}
65 servicio-alerta.sh
66 regresion.jl
67 generar_sistema.jl
68 calc01.sh
69 floyd.jl.save
70 generar_datos.jl
71 calc02.sh
72 mian.jl
73 matriz10.jl
74 regression.jl
75 gauss.jl
76 regresion07.jl
77 matriz1000.jl
78 copia-total
21°C
Mayorm. nublado
Buscar
03:05 p. m.
18/10/2026





Y vemos que ordena mis ficheros de mi directorio cuales tengo en total 138

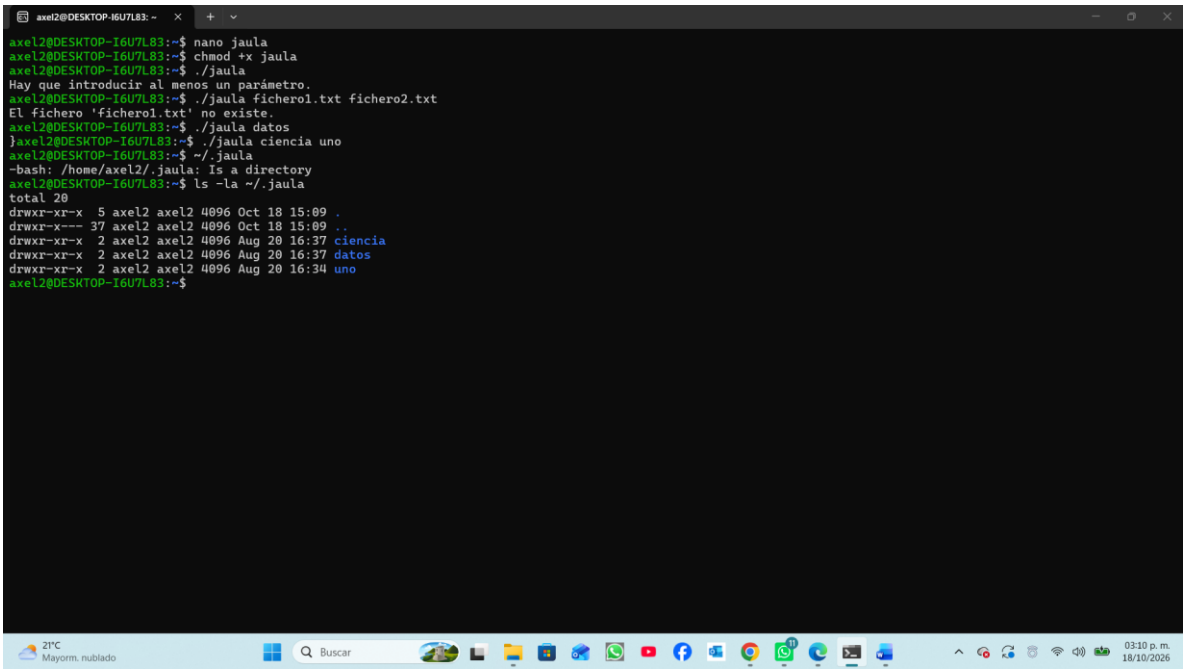
EJERCICIO 35



Este llamado jaula

Realizar un script llamado 'jaula' que cree, sólo si no existe, el directorio . jaula en la $HOME del usuario y mueva los ficheros pasados por parámetro a dicho directorio. En el caso de que no se le pase ningún parámetro se mostrará el siguiente mensaje de error: “Hay que introducir al menos un parámetro.” y retornará un código de retorno igual a 1. En el caso de que algún fichero introducido por parámetro no exi sta se mostrará el siguiente mensaje de error: “El fichero '$FICHERO' no existe.” y reto rnará un código de retorno igual a 2. Si el fichero . jaula existe en la $HOME del usuari o pero no es un directorio mostrará el siguiente mensaje de error: “El fichero '$HOME /.jaula' no es un directorio.” y retornará un código de retorno igual a 3
TAMBIEN LO COPIE PARA QUE SE EXPLIQUE MEJOR JEJE

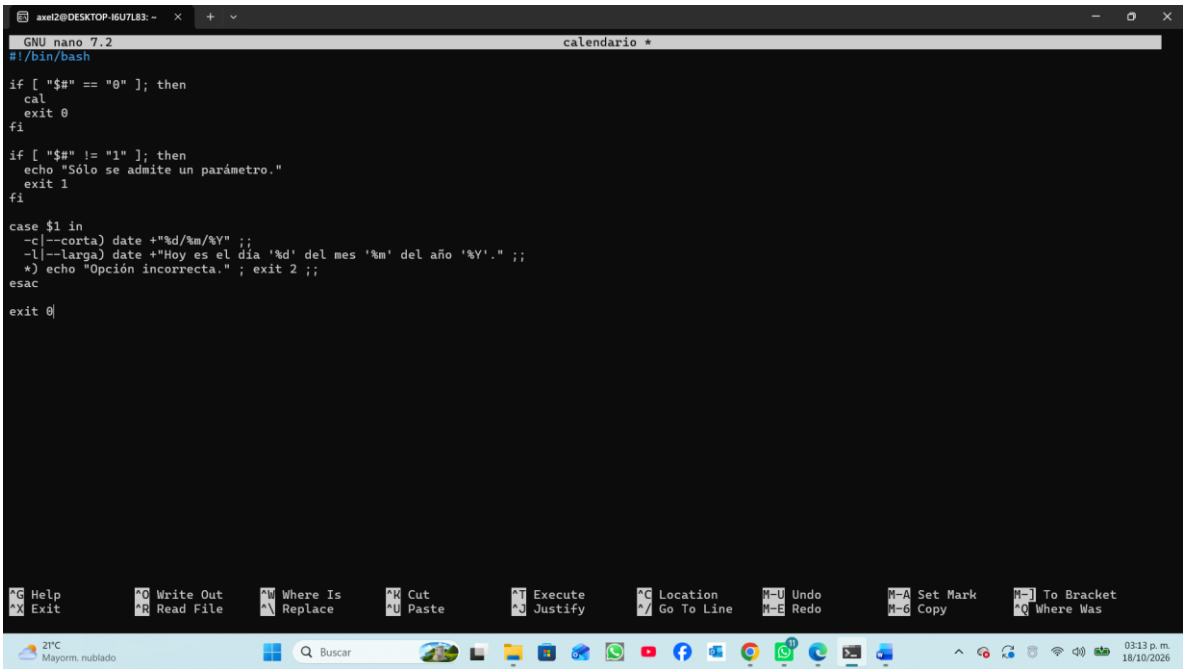
A ejecutarlo entonces



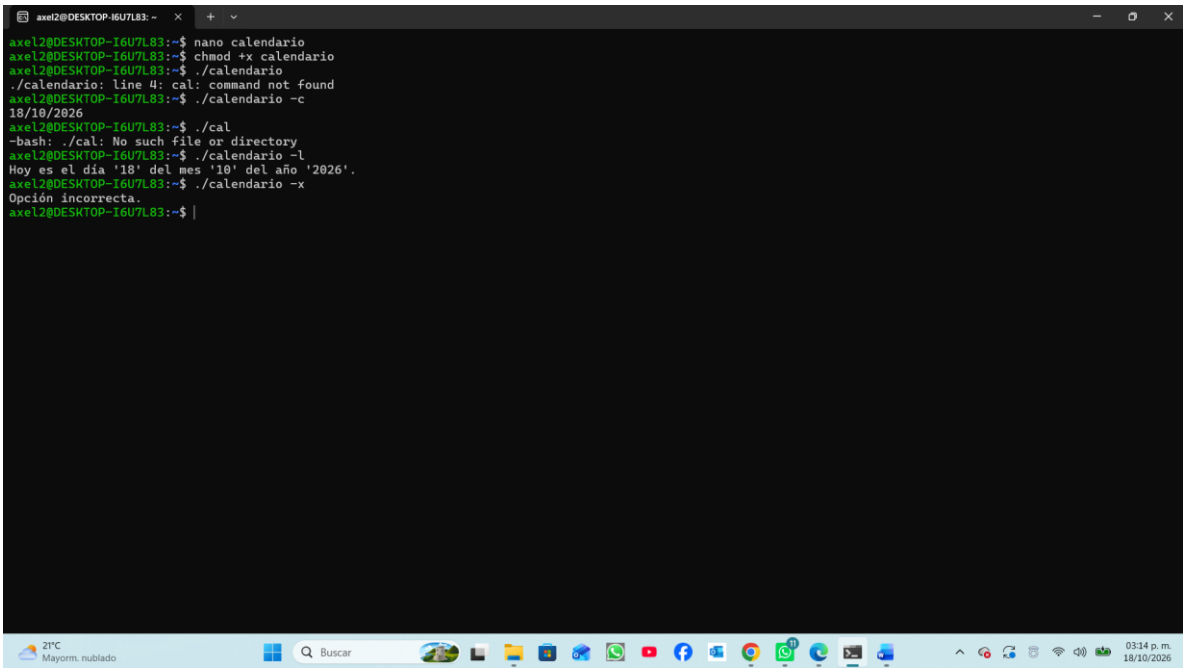
Al ejecutar vemos si pongo ficheros que no tengo manda el mensaje de no existe pero si pongo ficheros que si tengo entonces si crea jaula y en el comando de ls -la /.jaula se que que ciencia, datos y uno si se crearon ya que son ficheros que si tengo en escritorio

EJERCICIO 36

Realizar un script llamado 'calendario' al que si pasamos el parámetro c o el parámetro corta mostrará la fecha de hoy con el formato “$DIA/$MES/$AÑO” y si le pasamos el parámetro l o larga mostrará la fecha de hoy con el formato “Hoy es el día '$DIA' del mes '$MES' del año '$AÑO'.”. En el caso de que no se introduzca ningún parámetro se mostrará el calendario del mes actual. En el caso de que el número de parámetros introducidos sea distinto de
1 se mostrará el siguiente mensaje de error: “Sólo se admite un parámetro.” y retornará un código de retorno igual a 1. Si pasamos otra cosa que no sea c, corta, l o larga mostrará el siguiente mensaje de error: “Opción incorrecta.” y retornará un código de retorno igual a 2
TAMBIEN LO MISMO PARA QUE SE ENTIENDA ESTE ES EL DE CALENDARIO

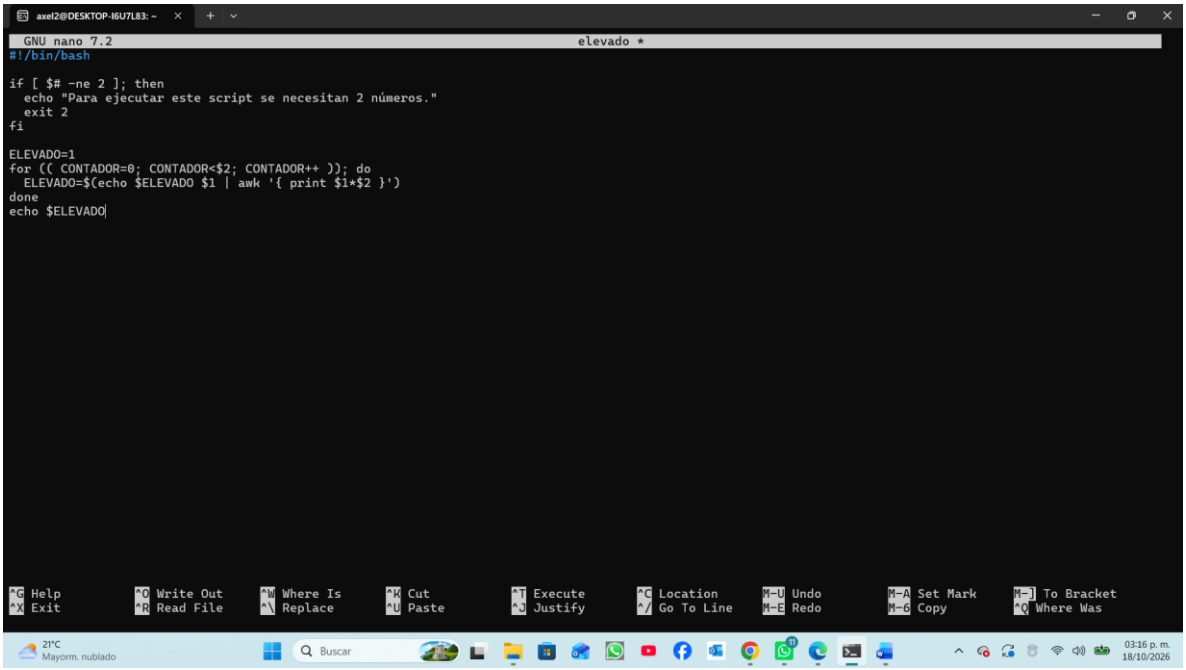


Vamos a ejecutarlo



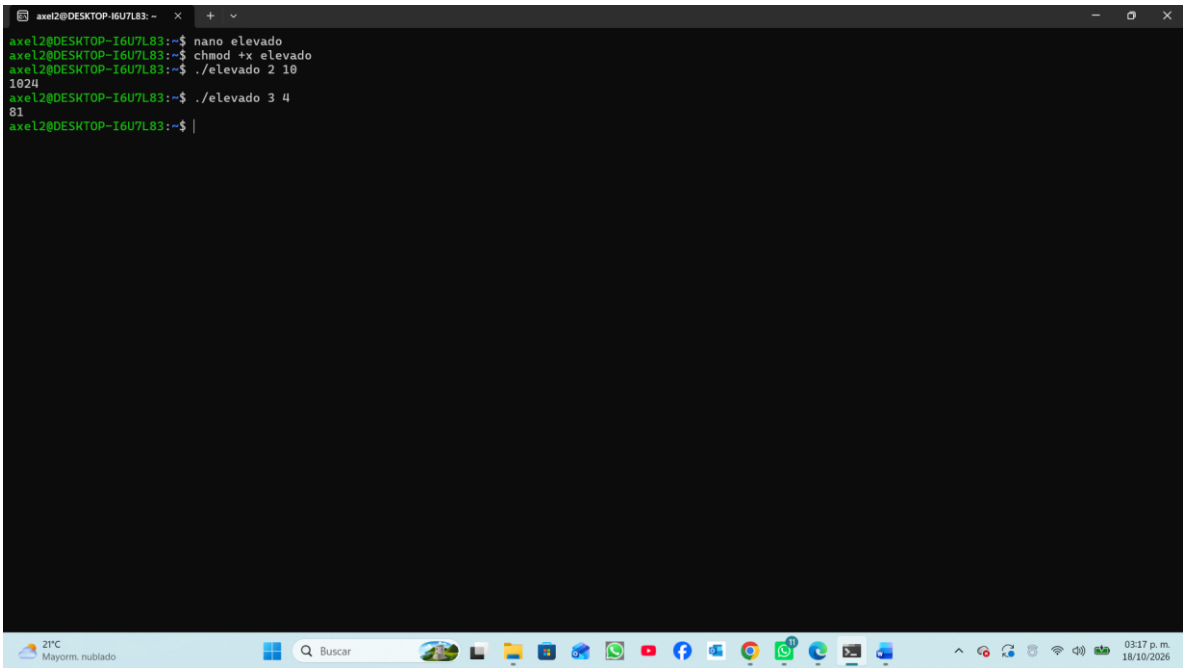
Al ejecutar primero solo da un error porque no es correcto el comando, pero al ejecutar *c da la fecha de este día 18/10/2026, con el ./cal da también error, ahora con el de -l da mas especifico el día y con -x da una opción incorrecta

EJERCICIO 37



Realizar un script llamado 'elevado' que calcule “a^b”, osea “a elevado a b”, donde “a” será el primer parámetro y “b” el segundo parámetro. En el caso de que el número de parámetros introducidos sea menor que 2 se mostrará el siguiente mensa je de error: “Para ejecutar este script se necesitan 2 números.” y retornará un código de retorno igual a 2.

A EJECUTARLO

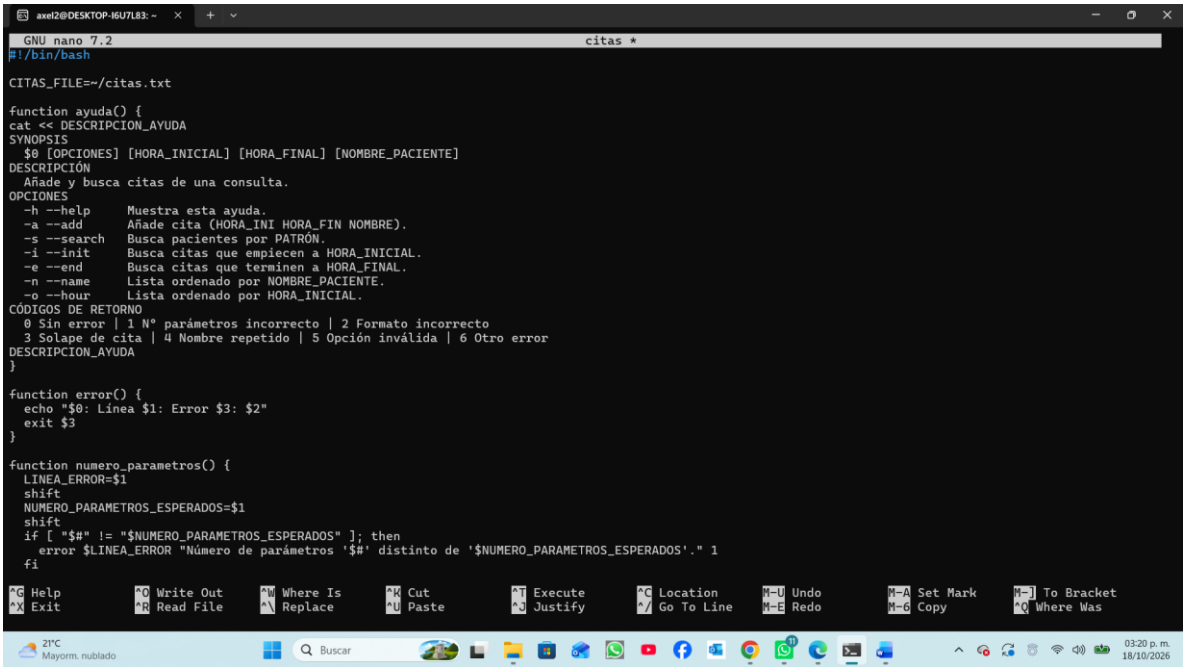


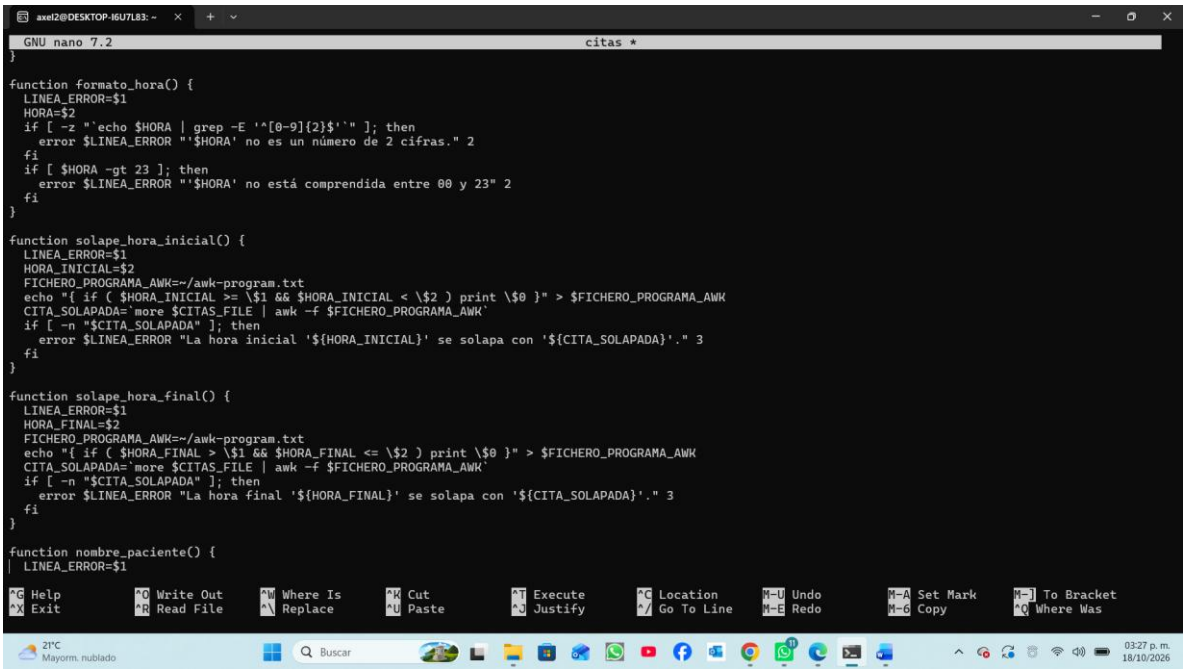
Al ejecutar vemos que al poner el de 2 y 10 pasa a 1024 y el de 3 4 pasa 81, y los leva bien

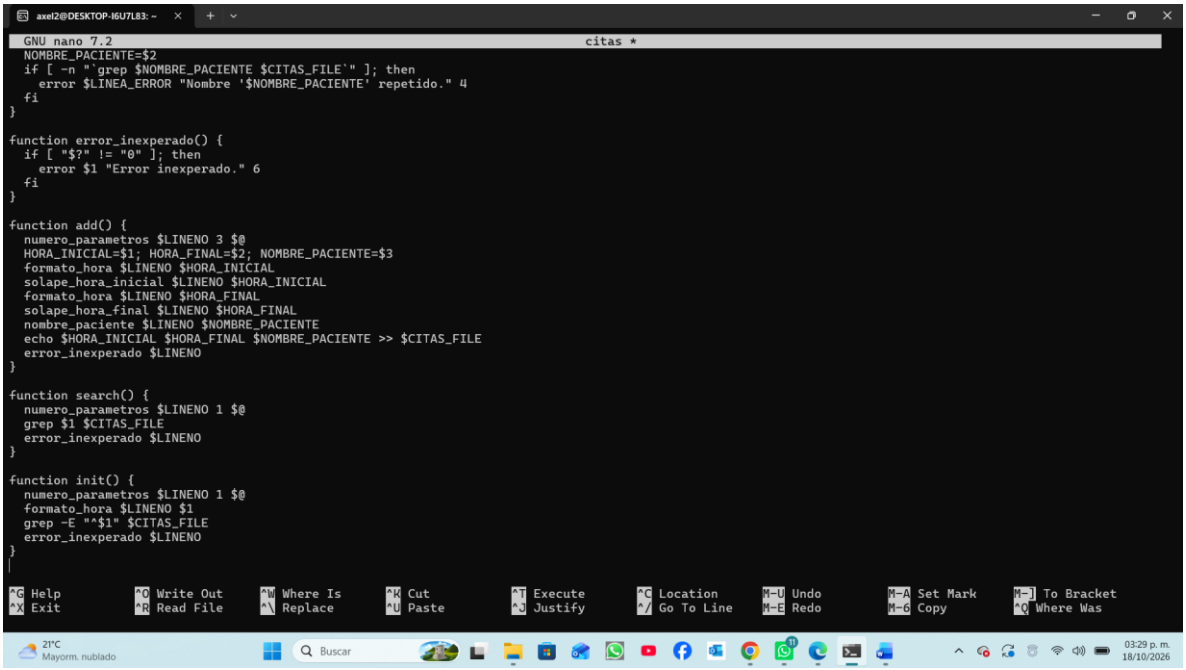
EJERCICIO 38

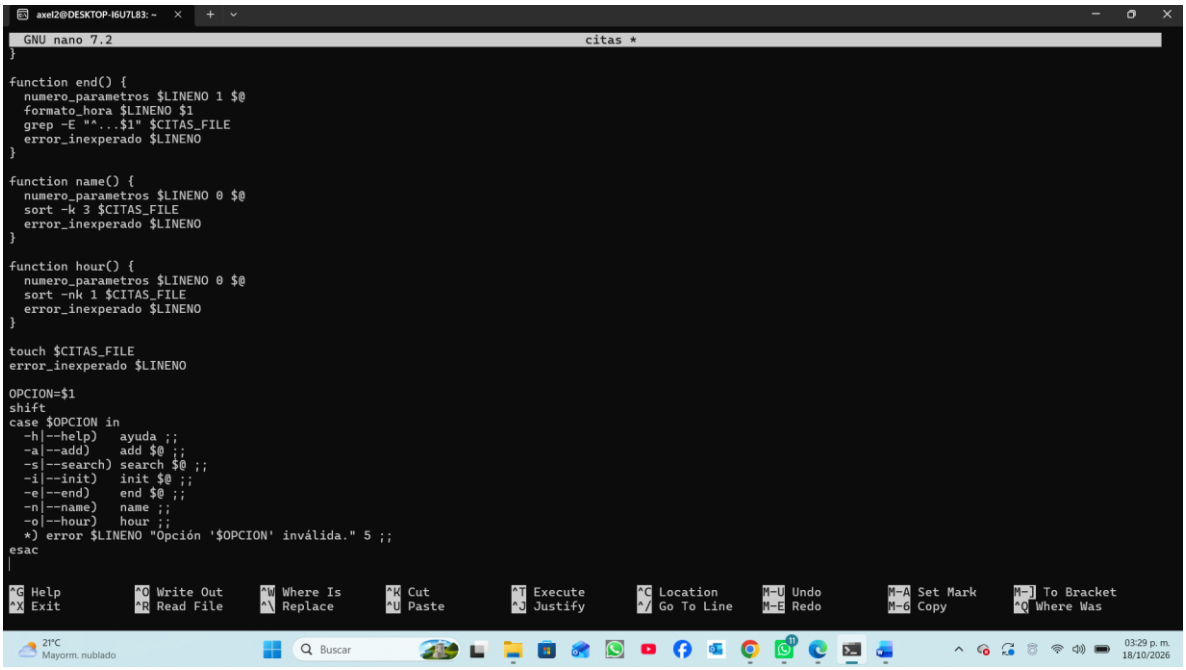
Ahora para mi este es el interesante de todos los scripts el de citas voy a poner lo que dice el pdf y después vamos a ejecutarlo

Realizar un script llamado 'citas' en el que se puedan utilizar las siguientes opciones: h help Para mostrar un texto de ayuda. a add Para añadir una cita con HORA_INI, HORA_FIN, y NOMBRE_PACIENTE. s search Para buscar los pacientes que contenga n PATRÓN. i init Para buscar las citas que empiecen a HORA_INICIO. e end Para buscar las citas que terminen a HORA_FINAL. n name Para listar todas las citas ord enadas por NOMBRE_PACIENTE. o hour Para listar todas las citas ordenadas por H ORA_INICIO. Para cada una de las opciones se comprobará que se introducen el nú mero de parámetros correctos y con el formato correcto. HORA_INCIO y HORA_FIN AL serán números enteros comprendidos entre 00 y 23. Al introducir una cita nueva s e comprobará que no se solape con otra ya introducida. Se comprobará también qu e no se repita ningún nombre de paciente.

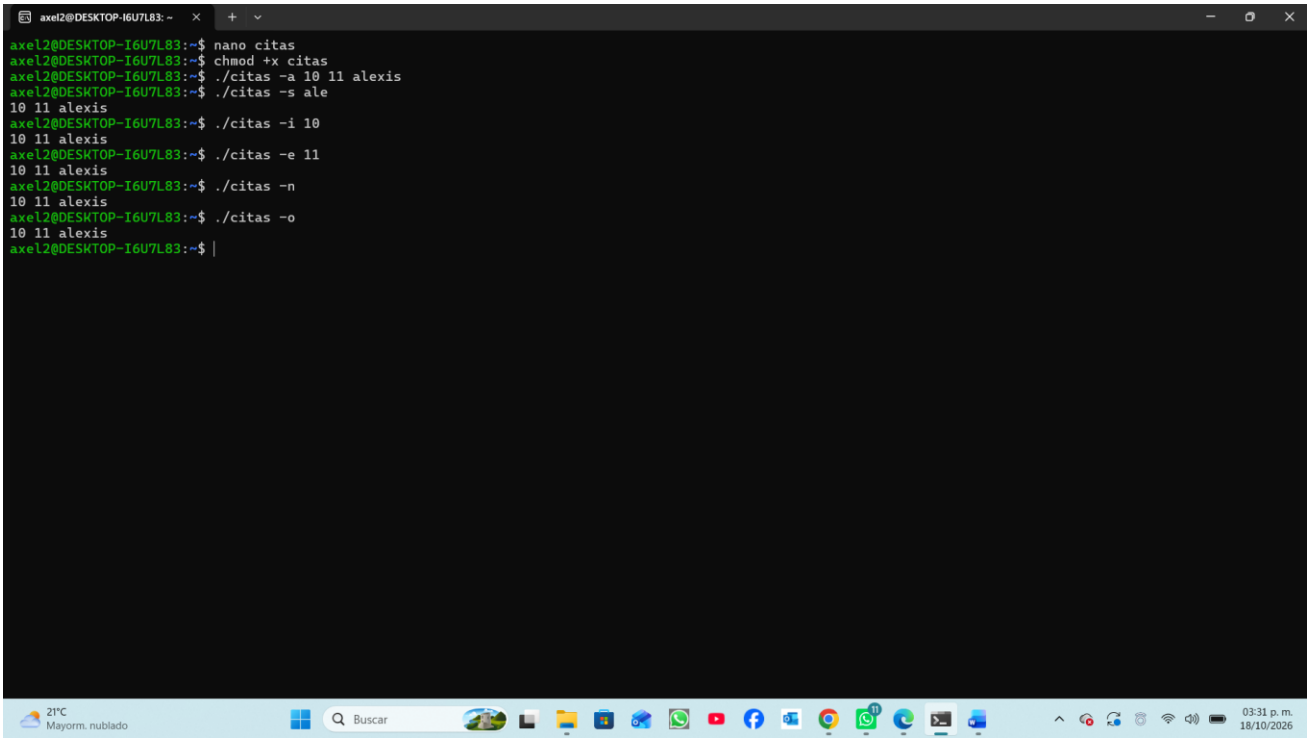



axel2@DESKTOP-I6U7L83: ~
GNU nano 7.2 citas *
}
function formato_hora() {
LINEA_ERROR=$1
HORA=$2
if [-z "`echo $HORA | grep -E '^[0-9]{2}$'`"]; then
error $LINEA_ERROR "'$HORA' no es un número de 2 cifras." 2
fi
if [$HORA -gt 23]; then
error $LINEA_ERROR "'$HORA' no está comprendida entre 00 y 23" 2
fi
}
function solape_hora_inicial() {
LINEA_ERROR=$1
HORA_INICIAL=$2
FICHERO_PROGRAMA_AWK=~/awk-program.txt
echo "{ if ($HORA_INICIAL >= \$1 && $HORA_INICIAL < \$2) print \$0 }" > $FICHERO_PROGRAMA_AWK
CITA_SOLAPADA=`more $CITAS_FILE | awk -f $FICHERO_PROGRAMA_AWK`
if [-n "$CITA_SOLAPADA"]; then
error $LINEA_ERROR "La hora inicial '${HORA_INICIAL}' se solapa con '${CITA_SOLAPADA}'." 3
fi
}
function solape_hora_final() {
LINEA_ERROR=$1
HORA_FINAL=$2
FICHERO_PROGRAMA_AWK=~/awk-program.txt
echo "{ if ($HORA_FINAL > \$1 && $HORA_FINAL <= \$2) print \$0 }" > $FICHERO_PROGRAMA_AWK
CITA_SOLAPADA=`more $CITAS_FILE | awk -f $FICHERO_PROGRAMA_AWK`
if [-n "$CITA_SOLAPADA"]; then
error $LINEA_ERROR "La hora final '${HORA_FINAL}' se solapa con '${CITA_SOLAPADA}'." 3
fi
}
function nombre_paciente() {
LINEA_ERROR=$1
^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark M-] To Bracket
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E Redo M-6 Copy ^Q Where Was
21°C Mayorm. nublado
Buscar
03:27 p. m. 18/10/2026


axel2@DESKTOP-I6U7L83: ~
GNU nano 7.2 citas *
NOMBRE_PACIENTE=$2
if [-n "`grep $NOMBRE_PACIENTE $CITAS_FILE`"]; then
error $LINEA_ERROR "Nombre '$NOMBRE_PACIENTE' repetido." 4
fi
}
function error_inexperado() {
if ["$?" != "0"]; then
error $1 "Error inexperado." 6
fi
}
function add() {
numero_parametros $LINENO 3 $@
HORA_INICIAL=$1; HORA_FINAL=$2; NOMBRE_PACIENTE=$3
formato_hora $LINENO $HORA_INICIAL
solape_hora_inicial $LINENO $HORA_INICIAL
formato_hora $LINENO $HORA_FINAL
solape_hora_final $LINENO $HORA_FINAL
nombre_paciente $LINENO $NOMBRE_PACIENTE
echo $HORA_INICIAL $HORA_FINAL $NOMBRE_PACIENTE >> $CITAS_FILE
error_inexperado $LINENO
}
function search() {
numero_parametros $LINENO 1 $@
grep $1 $CITAS_FILE
error_inexperado $LINENO
}
function init() {
numero_parametros $LINENO 1 $@
formato_hora $LINENO $1
grep -E "^$1" $CITAS_FILE
error_inexperado $LINENO
}
^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark M-] To Bracket
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E Redo M-6 Copy ^Q Where Was
21°C Mayorm. nublado
Buscar
03:29 p. m. 18/10/2026



Vamos a ejecutarlo

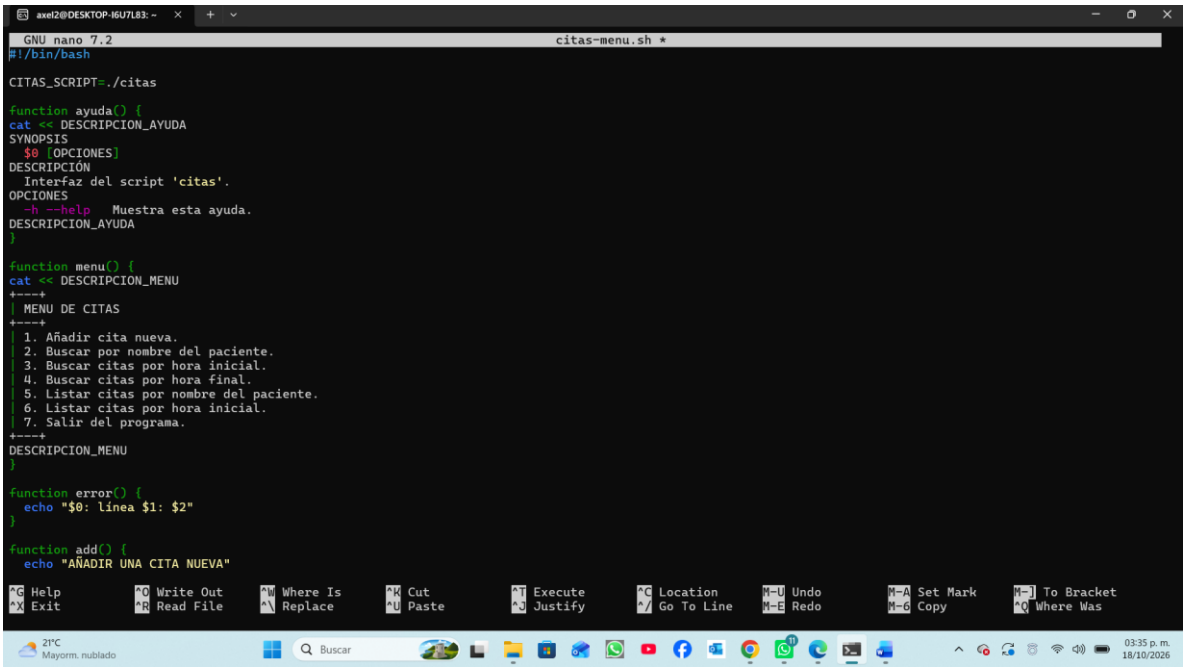


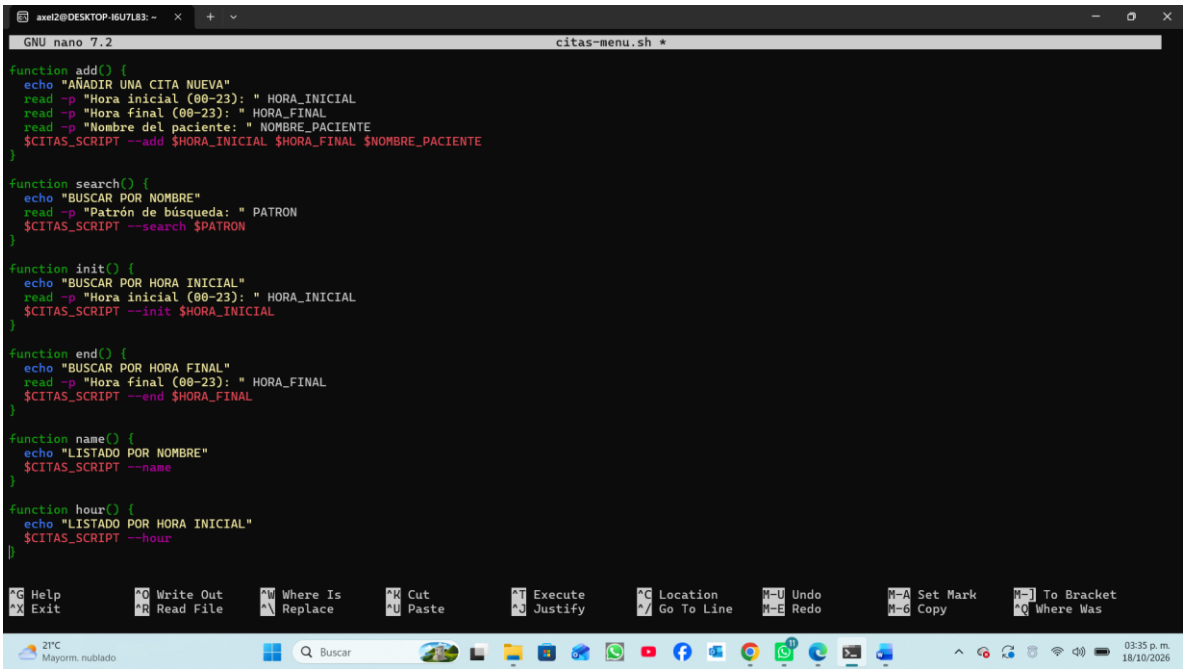
Al ejecutar primero con -a añade una cita en este caso yo Alexis, con la -s busca por patrón, la -i 10 las citas que empiezan a las 10 y la -e las citas que terminan a las 11, la de -n orde por nombre y al final la de -o ordena por hora

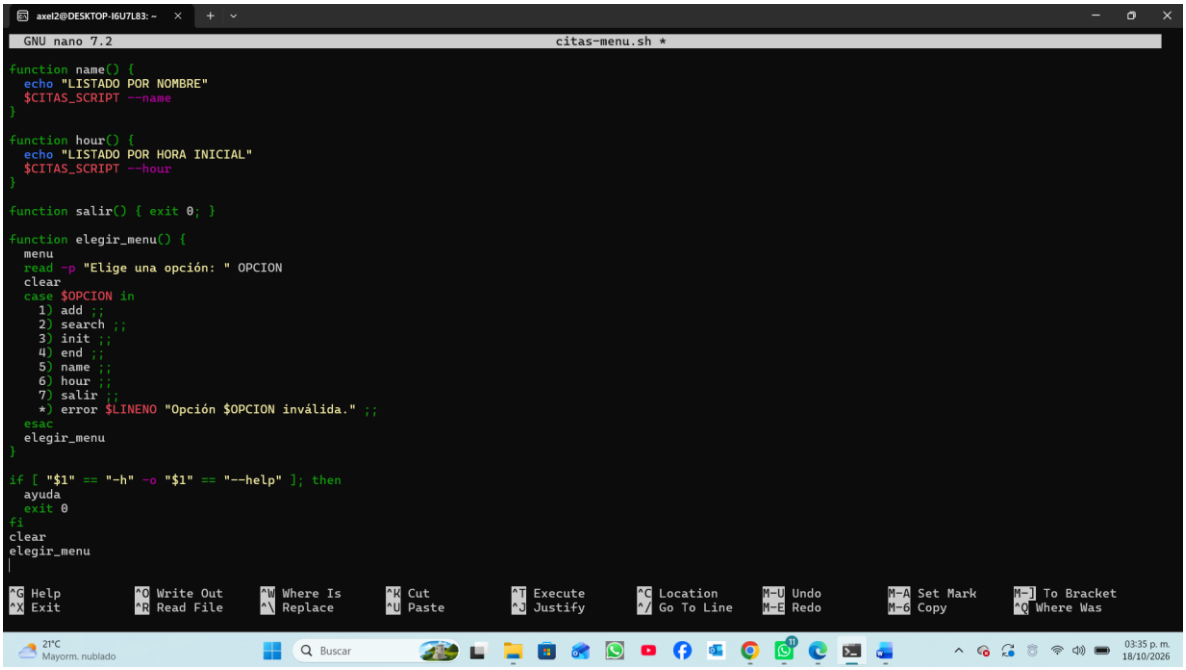
EJERCICIO 39

Realizar un script llamado 'citas menu.sh' que sea una interfaz del script 'citas' motrando un menú con las siguientes opciones: 1.Añadir cita nueva. 2.Buscar por nombre del paciente. 3.Buscar citas por hora inicial. 4.Buscar citas por hora final. 5.Listar las citas ordenadas por nombre del paciente. 6.Listar las citas ordenadas por hora inicial. 7.Salir del programa.

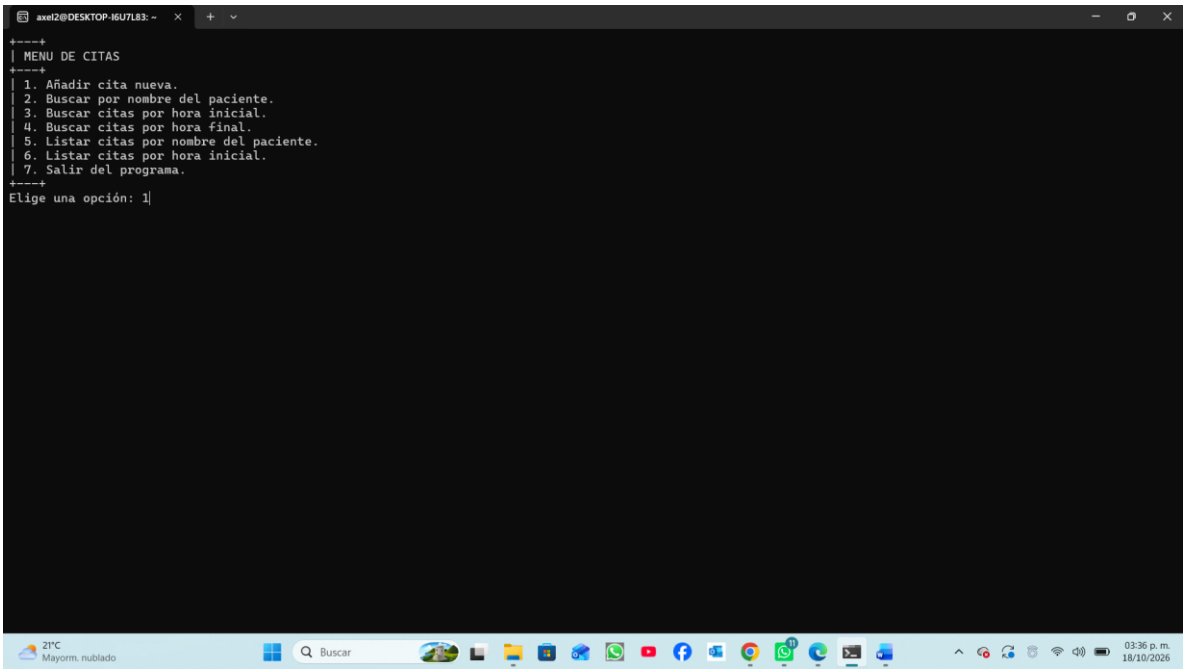
VAMOS A EJECUTARLO Y AÑADIR EL SCRIPT



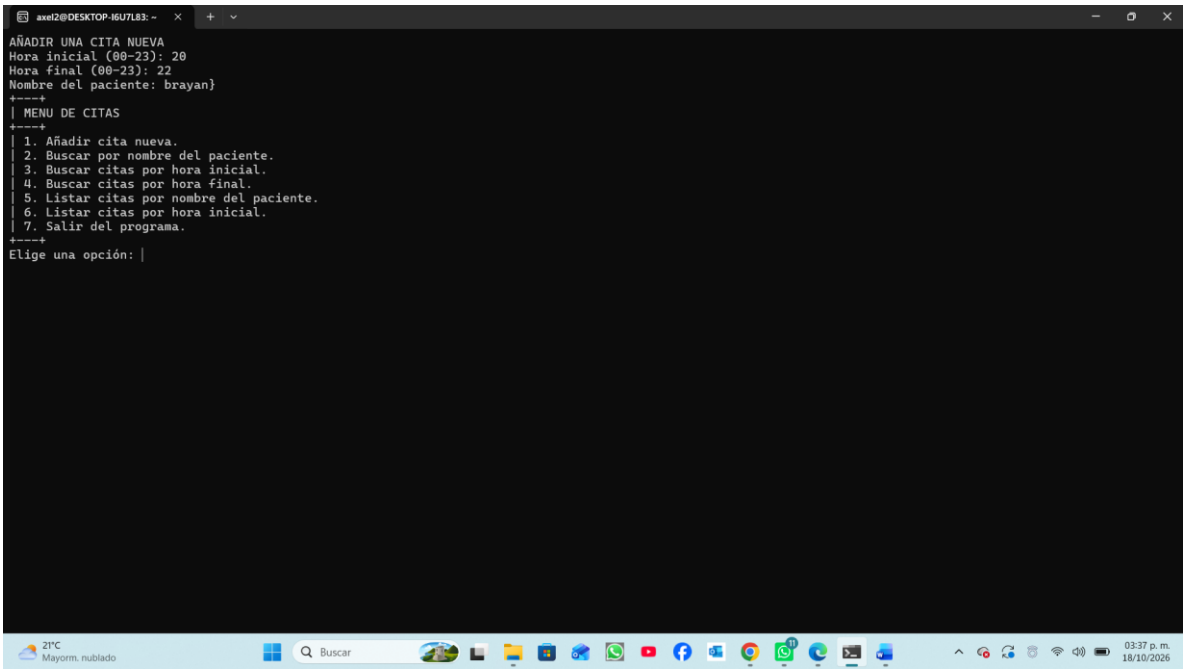




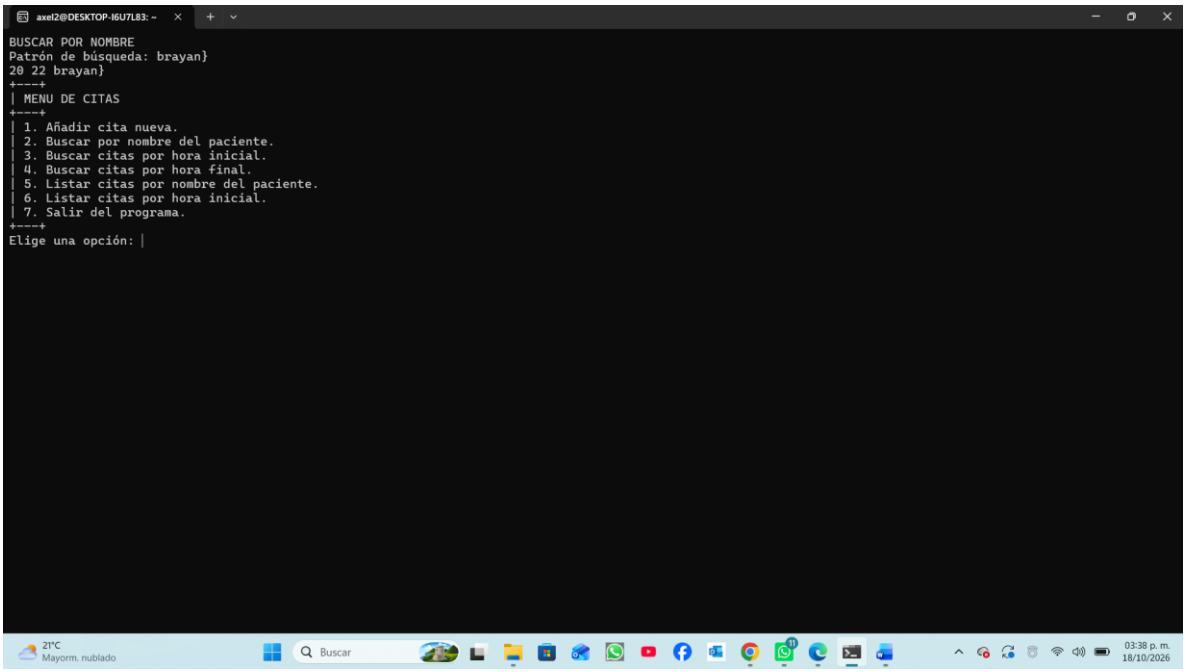
A ejecutarlo



Aparece este menú vamos a probarlo con la opción 1



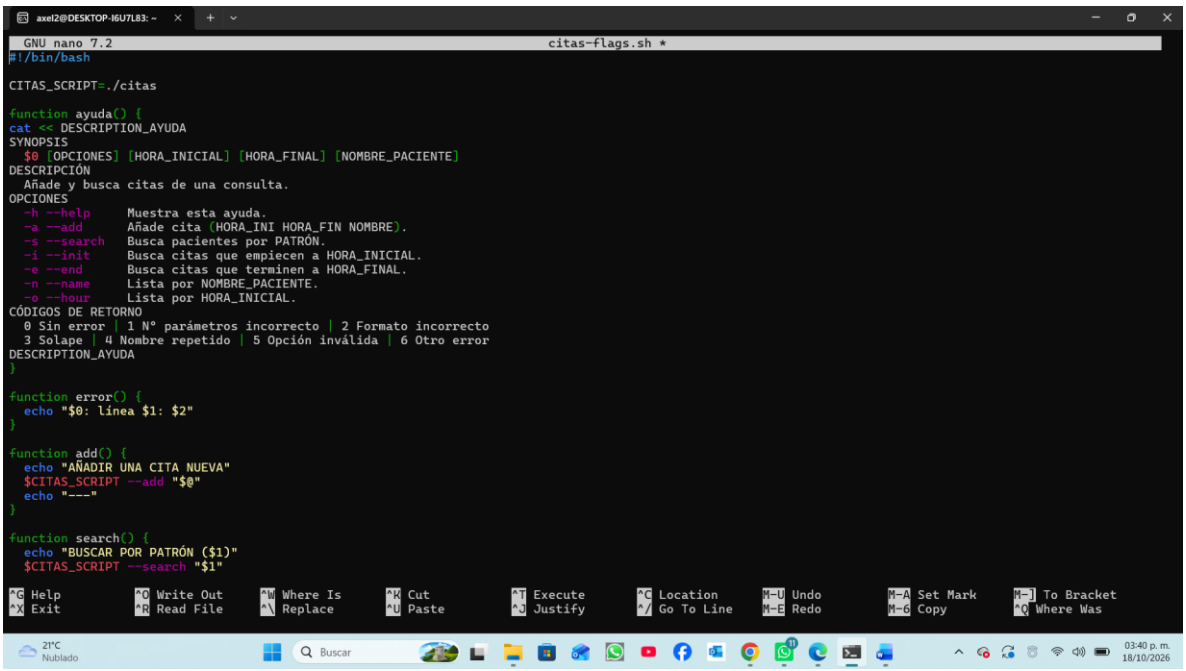
Vemos que la opción 1 funciono

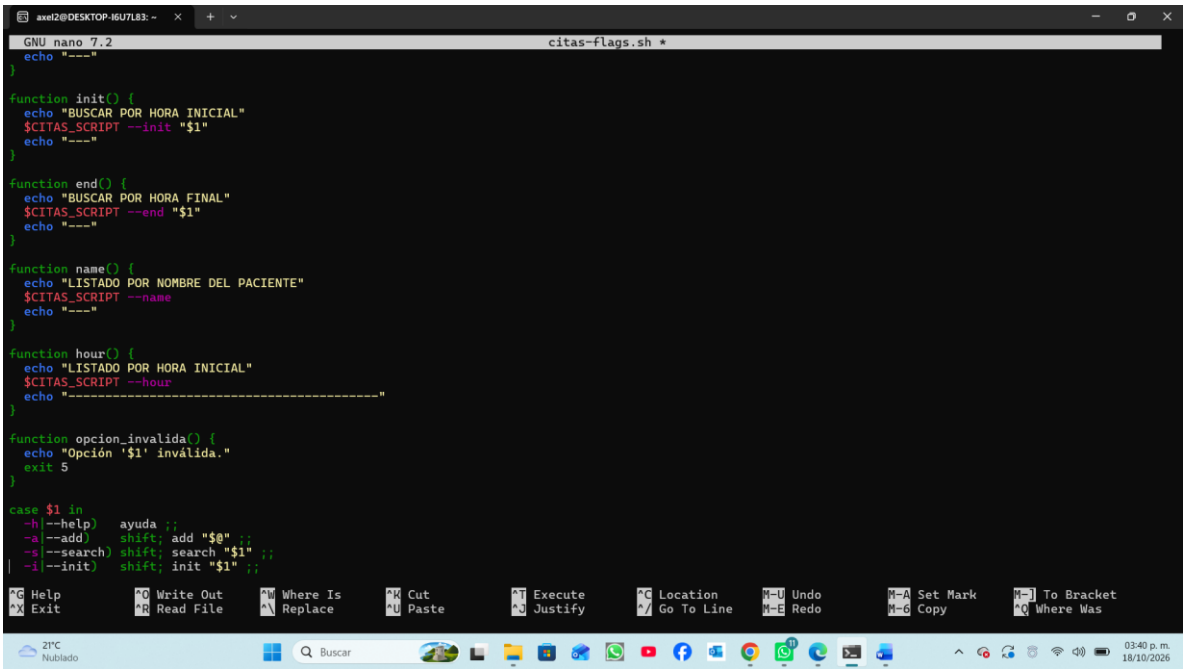


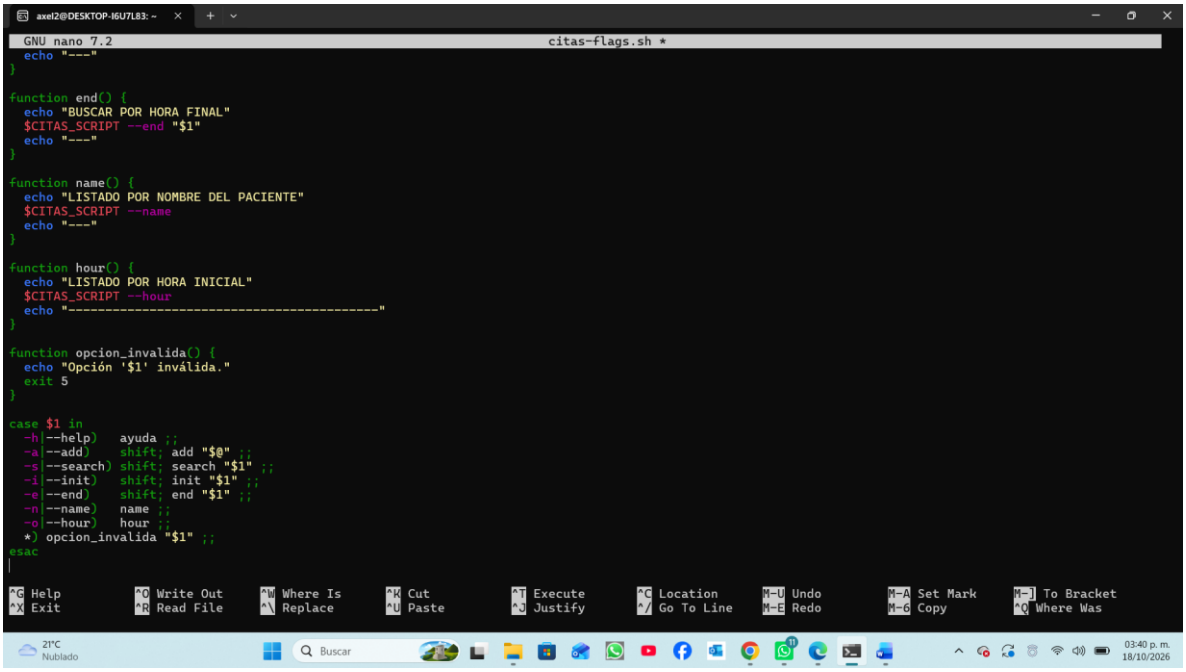
y si ocupamos la opcion 2 si busca al paciente en este caso al brayan} jeje así lo escribi me comi el } jajaja

EJERCICIO 40 EL ULTIMO

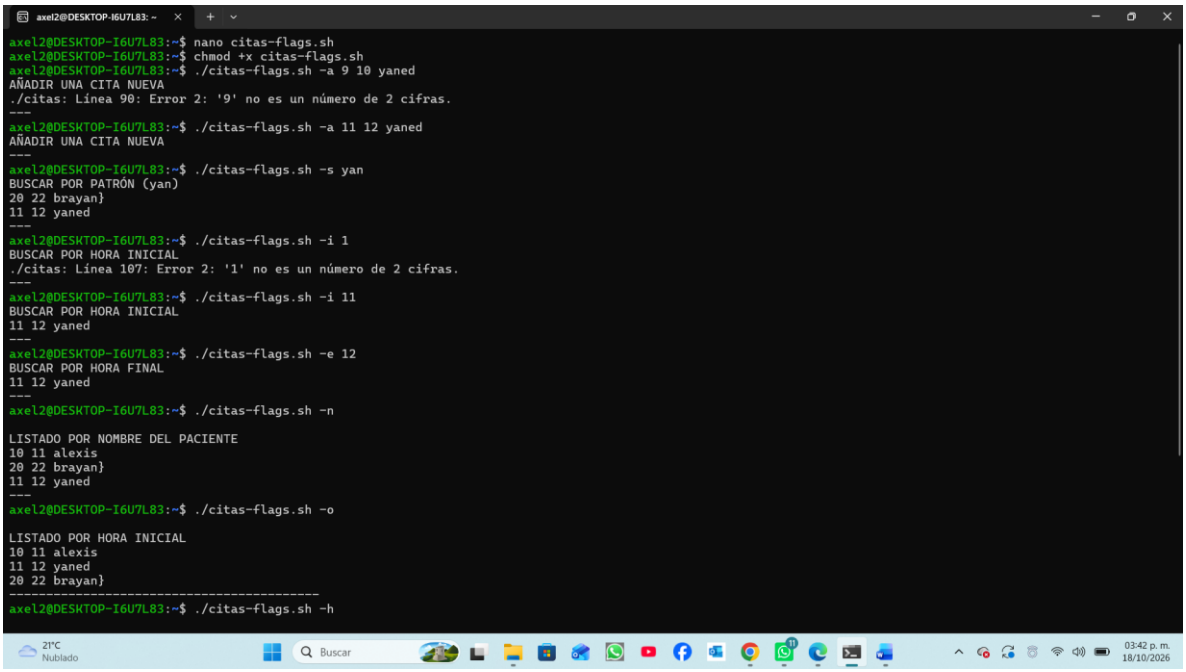
Realizar un script llamado 'citas flags.sh' para poder usar el script 'citas' mediante CLI.

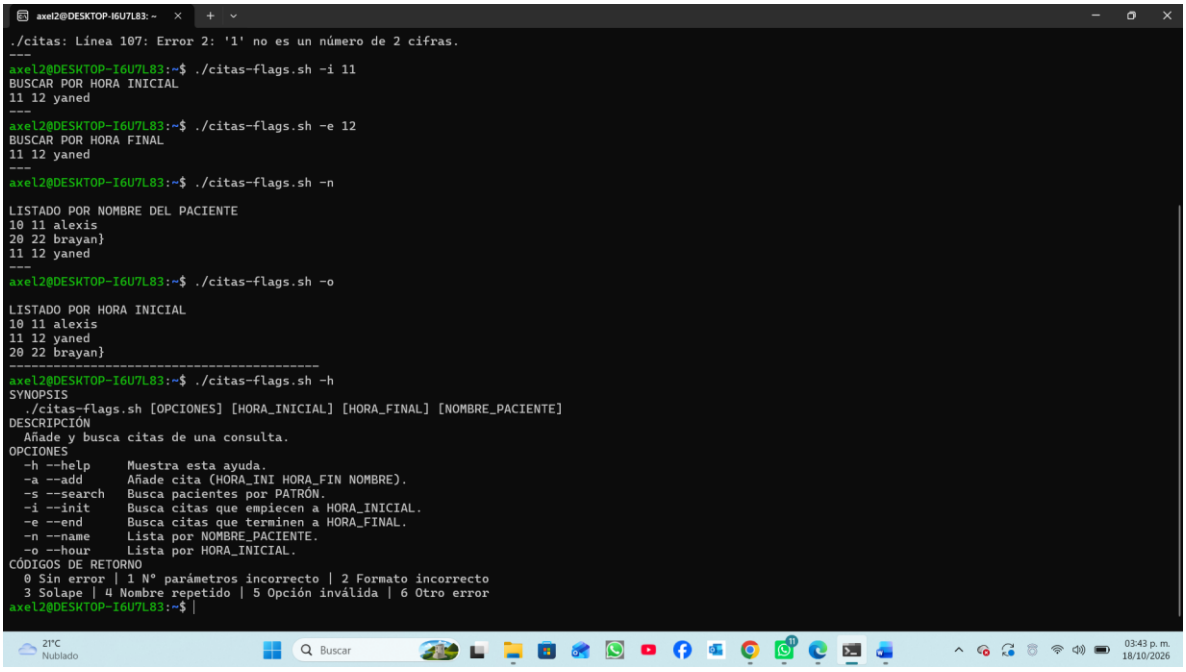






Vamos a ejecutarlo





Vemos que si funciona de hecho en la parte de -s se ve que el que hicimos de brayan hace unos momentos, en el anterior script y el de -n sale el mio el de Alexis el primero que se creo jeje, muestra las hora inicial, nombre del paciente, cita nueva, lo busa por patrón, así que funciona bien este sistema jeje

Así llegamos al final de este pdf de los 40 scripts que para mi ejemplo el de las citas es perfecto para cualquier hospital, incluso creando un script mas llamativo para otras empresas para que interactúen personas y trabajadores y es uno de los mas llamativos del semestre en la materia